

Senior Kubernetes -> AI Infrastructure Interview Handbook

Kubernetes / Linux / Distributed Systems / GPU / LLM Inference / Agent Runtime

2026 Research Edition - 面向 Senior Infrastructure / Platform / AI Infra 面试与转型



能力地图：保留 Kubernetes/Linux 深度，在其上增加 GPU、推理、分布式通信与 Agent Runtime。

定位 目标不是“从 Infra 转 ML Scientist”，而是把 Linux、Kubernetes、网络、分布式系统能力延伸到 GPU Fleet、Model Serving、Distributed Inference 与 Agent Sandbox。

0. 如何使用这本手册

这份手册按“面试能讲清 -> 源码/机制能深入 -> 生产问题能排查 -> AI Infra 能迁移”的顺序组织。每一章都包含知识架、机制、常追、建和系接点。

目标	建议重点	检验标准
Senior K8s / Platform	第 2-10 章	能从 API Server 一直讲到内核，能做生产故障树
GPU Platform / Fleet	第 2-12 章	能解释 GPU 从发现、分配、隔离、监控到故障处理
Inference Infra	第 2-14、16 章	能解释 TTFT/TPOT/KV Cache/批处理/多 GPU 通信与扩缩容
Agent Runtime	第 2-10、15、16 章	能设计沙箱隔离、凭证、网络、快照、容量与审计

推荐复习方法

- 第一遍：看图 + 每章“必”的，建立主路。
- 第二遍：对每个机制写 3-5 分钟口述答案；任何出现“就是这样”的地方继续追问为什么。
- 第三遍：把知识连接到真实故障：Pending、OOM、throttling、网络超时、GPU Xid、NCCL hang、TTFT。
- 第四遍：完成一个可展示的 Kubernetes AI Inference 目，形成可的工程故事。

1. 2026 岗位 Research : Kubernetes 工程师为什么适合转 AI Infra

2026 年的 AI 基础设施岗位已经明显从“纯 ML”分化出 Compute/Fleet、Kubernetes/Orchestration、Inference Serving、High-Performance Networking、Agent Infrastructure 等方向。当前公开岗位和官方文档显示，这些方向与传统 Senior Kubernetes / Infrastructure 的能力重合度很高。

观察来源	公开信息中的重点	对准备方向的含义
OpenAI - Compute Infrastructure	Kubernetes、scheduling、accelerators、NCCL/collective communication、networking、storage、fleet health、agent infrastructure	K8s 深度不是旁支，而是 AI compute control plane 的核心能力。
Anthropic - ML Infrastructure	大规模 distributed backend、production serving、SLO、observability、on-call；明确表示 transformer 经验并非前置	分布式系统和生产基础设施经验可以先进入，再补模型 serving。
CoreWeave Careers	Compute Services (Kubernetes, Bare Metal) 等 AI cloud 角色	Kubernetes + bare metal + fleet lifecycle 是 GPU cloud 主线。
Lambda Careers	Managed Kubernetes、Fleet Engineering、Compute、Storage、SDN	AI cloud 需要完整 cloud control plane，而不只 GPU kernel。

2. 面架：Senior Infra 会被怎么考

Senior/Staff Infrastructure 面试通常不按“背概念”打分，而是观察候选人是否能跨层定位问题、解释 trade-off、写可代、理模糊需求建立期可的平台。

面试维度	常见考察	Senior 级答题信号
Coding	Go/Python、并发、数据结构、可维护性	不仅 AC；解释复杂度、并发安全、错误处理、测试和可观测性
Linux/Systems	CPU、memory、IO、network、process、kernel	能从指标追到底层机制，并区分症状与根因
Kubernetes Internals	API Server、Informer、Scheduler、Kubelet、CNI/CSI	能描述调用链、缓存、一致性、失败重试与源码入口
Distributed Systems	一致性、复制、leader、queue、backpressure	明确 failure model 与 correctness invariant
System Design	多集群、调度、推理、agent sandbox	先需求/SLO，再容量、数据面/控制面、故障、成本
Troubleshooting	真实 incident	有假设 -> 证据 -> 收敛 -> 修复 -> 防复发闭环
Behavior/Leadership	项目 ownership、冲突、优先级	讲 trade-off、影响、复盘和组织级改进

2.1 一条通用追问链

What happens? -> Why? -> Failure mode? -> How do you observe it? -> How do you mitigate it? -> What trade-off did you choose?

例如“CPU limit 如何实现”不能停在 cpu.max。完整链路应该从 PodSpec -> kubelet -> CRI -> OCI LinuxResources -> cgroup -> CFS bandwidth -> throttling 指标 -> latency 影响 -> 是否应该设置 limit 的策略。

3. Linux Systems : Infra 面试的底座

AI Infra 故障排查分层

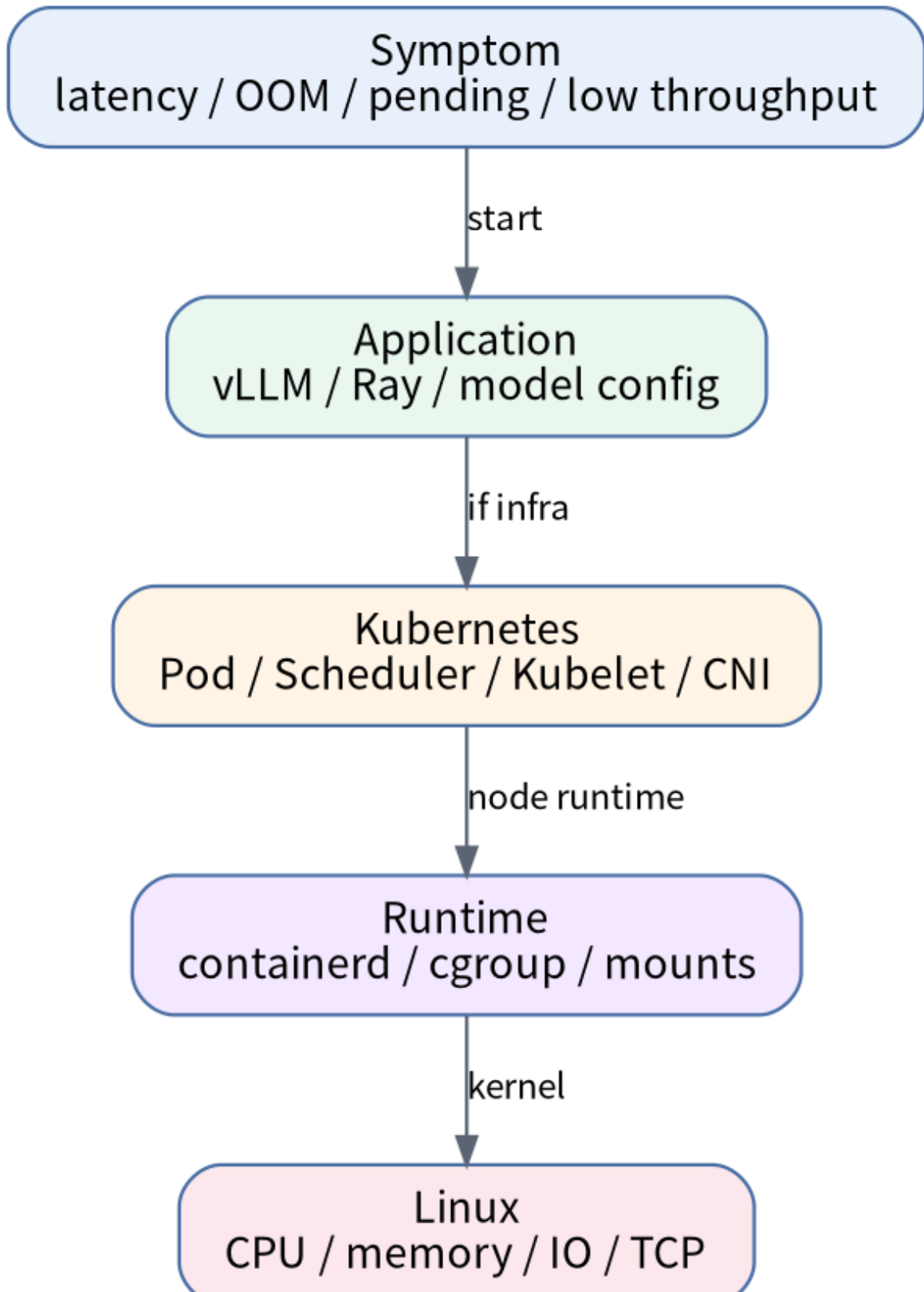


图 2：排障时从应用逐层下钻到 kernel、GPU、fabric 与硬件。

3.1 Process / Thread / Scheduler

- 程是源地址空容器；程共享地址空但有立度、和寄存器上下文。
- fork() 通过 copy-on-write 延迟复制页；execve() 替程地址空加新程序。
- 进程状态重点：R、S、D、T、Z。D 通常表示不可中睡眠，常于 block IO / kernel wait。
- context switch 包含保存/恢复 CPU 上下文，进程切换还可能带来地址空间/TLB/cache 影响。
- Linux CFS 目标是公平分配 CPU；nice/weight 改相。代核度持演，面注公平性、run queue、抢占与负载均衡念。

高频追问

- load average 为什么高但 CPU usage 不高？要想到 runnable + uninterruptible tasks。
- Zombie 为什么 kill -9 无效？因行已退出，只剩父程需要 wait 回收的 task 元据。
- 线程多为什么不一定吞吐更高？考虑锁竞争、context switch、cache locality、NUMA。
- 如何定位单核 100%？top -H、pidstat -t、perf top/record、火焰图。

3.2 CPU : utilization、load、steal、iowait、throttling

指标/概念	含义	误区	排查工具
user/sys	用户态/内核态 CPU 时间	sys 高不等于 kernel 有 bug，可能 syscall/网络/IO 很重	top, mpstat, pidstat
load average	可运行 + 不可中断任务压力	不是 CPU 百分比	uptime, ps state, vmstat
iowait	CPU 空闲但系统有 IO pending 的时间视角	不能直接等价“磁盘慢”	iostat, pidstat -d, blktrace
steal	虚拟化环境 CPU 被 hypervisor 占用	容器内看到高 steal 需看宿主层	mpstat
throttling	cgroup CPU quota 限制触发	节点空闲也可能发生	cpu.stat, container_cpu_cfs_throttled _*

面试要点 看到 latency 抖动时，不要只看 CPU utilization。run queue、throttling、CPU frequency、NUMA remote access、IRQ/softirq 都可能造成尾延。

3.3 Memory : Virtual Memory 到 OOM

- 地址通表映射物理；TLB 存地址果。minor fault 不需要磁盘 IO，major fault 往往需要存取。
- RSS 是当前驻留物理页；VSZ/virtual size 可能很大但并不等于实际占用。匿名页通常来自 heap/stack；file-backed 页和 page cache 可以被回收。
- Linux 向利用空存做 page cache，所以 free 很小并不等于内存不足；更应看 available、reclaim、swap、PSI。
- 内存压力下 kernel 进行 reclaim；无法满足分配时可能触发全局 OOM。cgroup memory.max 到达极限则可能触发 memcg OOM，影响范围不同。
- OOM victim 选择受到 badness score、oom_score_adj 等影响；Kubernetes QoS/requests 会间接影响节点压力管理和 eviction。

必须会区分

概念	核心区别
Kubernetes OOMKilled	容器进程被内核 OOM killer 杀死，status 显示 OOMKilled 是结果。
kubelet eviction	kubelet 根据 node pressure eviction policy 主动驱逐 Pod，不等同 kernel OOM。
memory.max	硬上限；接近/超过时 reclaim，无法回收可能 memcg OOM。
memory.high	cgroup v2 throttling/reclaim pressure 边界，不是最终硬限制。
memory.min/low	内存保护语义，可用于更精细 QoS；实际启用取决于环境/策略。

3.4 Filesystem / IO

- VFS 提供统一 inode/dentry/file 抽象；open 返回 file descriptor，fd 是进程表中的索引。
- unlink 删除目录项，不会立即释放仍被进程打开文件的 inode/data blocks；ls -l +L1 是经典排查方式。
- df 看 filesystem block 使用，df -i 看 inode；“No space left”可能是 blocks、inode、quota 或 thin provisioning 问题。
- 读路经常经过 page cache；direct IO 绕过部分 page cache 语义，数据库/存储系统常有不同策略。
- Block layer 排查关注 latency、queue depth、IOPS、throughput、utilization，而不是只看 MB/s。

3.5 Network : 从 socket 到 packet

- 理解 TCP 三次握手、四次手、重、滑口、塞控制、RTO/RTT。
- TIME_WAIT 通常在主动关闭方，保护旧连接报文；CLOSE_WAIT 大量存在通常表示应用没有 close 已被对端关闭的 socket。
- Linux packet path 涉及 socket buffer、TCP/IP stack、routing、netfilter/nftables、qdisc、NIC driver、IRQ/NAPI。
- 容器网络增加 veth/bridge/overlay/eBPF datapath；Kubernetes Service 可能由 iptables/IPVS/eBPF 实现，具体取决于网络方案。
- conntrack table 满会导致新连接异常，是高连接数 K8s 集群典。

Network troubleshooting 顺序

1. DNS：名字是否解析、search domain/ndots 是否导致额外查询。
2. L3：route、ARP/neighbor、MTU。
3. L4：ss -s/-ti、SYN retransmit、listen backlog、conntrack。
4. Policy/NAT：NetworkPolicy、iptables/nft、SNAT、LB。
5. 应用：connection pool、timeout、retry、TLS handshake。
6. 跨节点/跨 AZ：overlay、underlay、ENI/NIC、丢包和 RTT。

4. Container Runtime : Namespace、cgroup、CRI、OCI



图 3：Pod 从 kubelet 到 CRI、containerd、OCI runtime 与 Linux Kernel 的完整链路。

4.1 Namespace

Namespace	隔离对象	容器中的典型意义
PID	进程 ID 与进程树	容器内看到自己的 PID 1
Mount	mount table	不同 rootfs / bind mount
Network	网卡、route、socket stack	独立 eth0、route、port
UTS	hostname/domainname	容器 hostname
IPC	SysV IPC/POSIX message queue	共享内存隔离
User	UID/GID 映射	rootless / 降低 host root 风险
Cgroup	cgroup namespace view	隐藏/重定位 cgroup 路径

4.2 cgroup v2 必须掌握

K8s 语义	cgroup v2 常见文件	作用
CPU request	cpu.weight	竞争时相对权重
CPU limit	cpu.max	CFS bandwidth 上限
Memory limit	memory.max	内存硬上限
Memory QoS	memory.high / memory.min / memory.low	回收压力与保护
IO weight/limit	io.weight / io.max	block IO 权重与带宽/IOPS 限制
PIDs	pids.max	防止 fork bomb
Pressure	cpu.pressure / memory.pressure / io.pressure	PSI 压力信息

重要：requests 首先是 Kubernetes scheduling/QoS 语义，不能简单认为每个 request 都必然一对一映射成一个内核硬保证。CPU request 常映射为权重；memory request 的内核保护取决于 MemoryQoS/运行时策略。

4.3 CRI 与 OCI 的职责边界

- CRI 是 kubelet 与 container runtime 的 gRPC 接口：RuntimeService + ImageService。
- containerd CRI plugin 把 Pod sandbox/container 生命周期翻译成 containerd task/image/snapshot 等对象。
- OCI Runtime Spec 描述 process、mounts、Linux namespaces、LinuxResources 等；runc/crun 将 spec 落到内核。
- sandbox 的网络通常由 CNI 配置；pause/infra container 作为网络 namespace 点是常。

5. Kubernetes Control Plane Internals

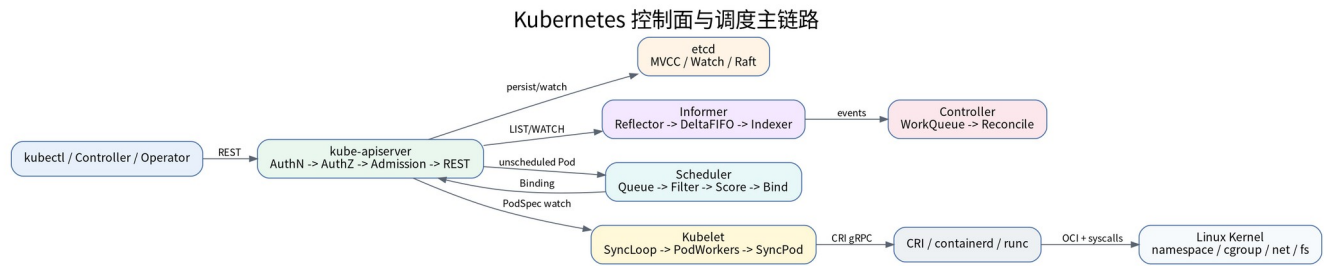


图 4: 控制面、Informer、Scheduler、Kubelet 与 runtime 的核心协作。

5.1 kube-apiserver 请求链

HTTP -> Authentication -> Authorization -> Admission -> Validation/Defaulting -> REST storage -> etcd

- Authentication 验证 Authorization 决定“你能做什么”；Admission 在对象持久化前执行策略/变更。
- Watch 是 Kubernetes 控制器事件驱动模型的基础。resourceVersion 是理解 watch continuity、cache、relist 的。
- API Priority and Fairness 用于在高负载下隔离/公平处理不同请求流；面试可从 control plane overload 延伸。

5.2 etcd

- etcd 使用 Raft 保证复制日志一致性；通常奇数成员以容忍故障并维持 quorum。
- Kubernetes 依赖 etcd 的 MVCC/Watch；compact 会回收旧 revision，watcher 太落后可能需要重新 List。
- 性能瓶颈常来自磁盘 fsync latency、网络 RTT、大对象/高写入速率、碎片与数据库大小。
- 务必理解 snapshot、revision、集群身份；生产事故中“能恢复”比“有备份”更重要。

5.3 Informer 源码主链

Reflector.ListAndWatch -> DeltaFIFO -> Controller.processLoop -> Indexer/Store -> sharedProcessor -> ResourceEventHandler

- Reflector 负责 List 初始状态并持续 Watch；/expired resourceVersion 后进行恢复。
- DeltaFIFO 以 key 聚合 delta，减少对同一对象的重复处理，并支持 Replace/Sync。
- Indexer 提供线程安全本地缓存和索引，业务代码应优先从 lister/cache 读取，减少 API Server 力。
- SharedInformer 允许多个 handler 共享一个 watch/cache；processor listener 为 handler 分发事件。
- Controller 通常不要直接在 event handler 做重活，而是 enqueue key，worker 从 rate-limited workqueue 进行 reconcile。

为什么最终一致而不是强一致？

Informer cache 与 API Server 之存在播延；reconcile 逻辑必须幂等、可重试，并以“观察当前状态 -> 比较期望状态 -> 执行动作”的方式最终收敛。

6. kube-scheduler 深入

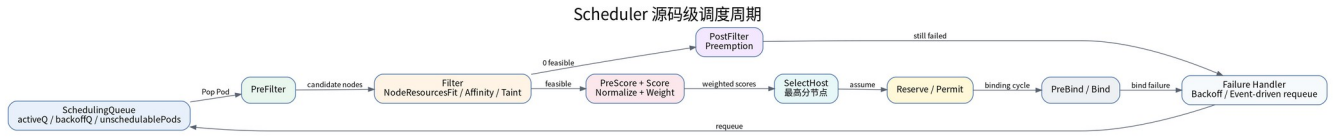


图 5: Scheduler queue、scheduling cycle、binding cycle 与失败重试。

6.1 三类队列/集合

结构	作用	进入条件	离开条件
activeQ	当前可尝试调度的 Pod，按 QueueSort 排序	新 Pod、backoff 到期、事件重新激活	Pop 进入 scheduling cycle
backoffQ	失败后等待指数退避，避免热循环	调度失败且需要 backoff	backoff 到期 -> activeQ
unschedulablePods	当前集群状态下不满足条件	Filter/PostFilter 失败	相关 Node/Pod/PVC 等事件 -> active/backoff

6.2 Scheduling Framework extension points

阶段	问题	代表插件/动作
PreEnqueue/QueueSort	是否入队、谁先调度	SchedulingGates / PrioritySort
PreFilter	预计算 Pod 调度状态	NodeResourcesFit / Affinity / Topology
Filter	Node 能不能放	resources, taints, affinity, volumes
PostFilter	0 feasible 怎么办	DefaultPreemption
PreScore/Score	哪个 Node 更优	NodeResourcesFit, BalancedAllocation, topology, affinity
Reserve/Unreserve	先假定资源	volume/device/自定义状态
Permit	是否等待/批准	gang/co-scheduling 扩展可利用
PreBind/Bind	最终绑定前动作	VolumeBinding / DefaultBinder

6.3 NodeResourcesFit scoring

- LeastAllocated：request 占比越低通常得分越高，偏 spread。
- MostAllocated：已分配比例越高通常得分越高，偏 bin packing，有利于形成可 scale-down 的空节点。
- RequestedToCapacityRatio：自定义 utilization -> score 曲，适合表非性策略。
- 最终节点由多个 Score plugin 的归一化得分 * weight 累加决定，而不是 NodeResourcesFit 定。

6.4 Preemption

当普通 Filter 不到点，PostFilter 可尝试 preemption。核心问题是：在哪些节点上移除哪些低优先级 Pod 可以让当前 Pod 可调度，同时尽量减少 disruption。PDB、priority、victim selection、nominatedNodeName 都是高频追问。

6.5 Scheduler 性能

- `percentageOfNodesToScore` 等配置用于减少大集群评分开销。
- `plugin` 的 `PreFilter/PreScore` 应避免重复昂贵计算；`CycleState` 用于周期内共享状态。
- 大量 `unschedulable Pod` + 高频集群事件会制造 `requeue` 压力，需要 `event hint/列化思`。
- AI workload 还会引入 `gang scheduling`、`topology-aware placement`、稀缺设备和长时间运行任务的公平性问题。

7. Kubelet、CNI、CSI、Node Lifecycle

7.1 Kubelet SyncLoop

- Kubelet 接收来自 `apiserver/其他 source` 的 `Pod update`，维护 `desired vs actual state`。
- `PodWorkers` 把同一 `Pod` 的操作串行化，避免并发生命周期操作互相踩踏。
- `SyncPod` 涉及 `pod cgroup`、`volumes`、`sandbox`、`container`、`probes`、`status` 等多个子系统。
- `PLEG/事件机制` 用于感知 `runtime` 容器状态变化；`runtime/network/storage` 异常最终会反映到 `Pod lifecycle`。

7.2 CNI

`kubelet -> CRI RunPodSandbox -> runtime -> CNI ADD -> create veth/IP/routes/policy -> sandbox ready`

- 必须解释 `Pod IP` 如何被分配、`veth` 如何连接 `host/network namespace`、`route/overlay/underlay` 模型。
- `Service dataplane` 与 `CNI` 不一定是同一组件；`kube-proxy`、`Cilium/eBPF`、云厂商实现差异需要分清。
- 高频故障：IP 枯竭、`MTU mismatch`、`conntrack exhaustion`、`SNAT port exhaustion`、`NetworkPolicy`、`DNS`。

7.3 CSI

`PVC -> scheduler VolumeBinding -> CSI controller provision/attach -> kubelet CSI node stage/publish -> mount`

- 区分 `Controller Service` 与 `Node Service`。
- `WaitForFirstConsumer` 使 `volume provisioning` 考虑 `Pod topology`，避免先建卷后发现节点/AZ 不匹配。
- `Attach/Mount` 慢、`stale mount`、`device busy`、`filesystem error` 是常见 incident。

8. Kubernetes Networking / Storage / Multi-cluster

8.1 Service 与数据面

- `Service` 是稳定 `VIP/名字抽象`，`Endpoints/EndpointSlice` 维护 `backend`。
- `iptables` 模式通过规则 `NAT`；`IPVS` 使用虚拟服务/连接表；`eBPF dataplane` 可在更早 `hook` 点实现 `service/load balancing`。
- `externalTrafficPolicy`、`source IP preservation`、`SNAT`、`hairpin`、`session affinity` 都可能成为追问。

8.2 Ingress / Gateway / Service Mesh

- `Ingress/Gateway` 主要理 `L7/L4 north-south`；`service mesh sidecar/ambient` 处理服务间流量治理。
- `mTLS`、`xDS`、`Envoy cluster/listener/route`、`outlier detection`、`retry budget`、`circuit breaker` 是高级面试点。
- 多集群时要讨论 `service discovery`、`identity/trust`、`east-west gateway`、`failover`、`locality` 与故障域。

8.3 Storage

- 理解 local disk / network block / distributed filesystem / object storage 的性能与一致性差异。
- AI 景外注模型重分、checkpoint、dataset streaming、local NVMe cache 与冷启动。
- 性能指标：throughput、IOPS、latency、metadata ops、cache hit、read amplification。

9. Reliability / Observability / Capacity

9.1 SLI/SLO/Error Budget

- SLI 不是“有监控”，而是用户体验的定量指标；例如 availability、TTFT p99、token streaming error rate。
- SLO 是目标区间；error budget 用于平衡 feature velocity 和 reliability。
- 告警优先从 symptom/SLO burn rate 出发，而不是所有资源利用率都报警。

9.2 Capacity Planning

Demand forecast -> headroom -> failure domain reserve -> bin packing efficiency -> scale-up latency -> cost

- Kubernetes CPU/Memory 注 requests 与实际 utilization 的 gap。
- GPU 场景容量单位更复杂：GPU type、HBM、互联拓扑、MIG profile、model compatibility。
- 推理容量要从 QPS、input/output tokens、TTFT/TPOT、batching efficiency 推导 GPU replicas。

9.3 Incident Response

7. 定义影响面与用户症状；避免先盯某个组件。
8. 建立时间线：deploy/config/traffic/hardware/event。
9. 用 RED/USE 方法确定瓶颈层。
10. 缓解优先：rollback、shed load、failover、disable expensive feature。
11. 修后做 root cause + contributing factors，不把“人为错误”当根因。
12. 产出防复发：guardrail、automation、test、SLO、runbook。

10. Coding 与 Distributed Systems

10.1 Go/Python 面试重点

主题	Go	Python
并发	goroutine/channel/select/context/mutex/atomic	asyncio/thread/process/GIL/queue
资源管理	defer/context cancellation	context manager/finally
网络	net/http, gRPC	aiohttp/httpx, grpc
可靠性	errors.Is/As, retry/backoff	exception hierarchy, retry/backoff
性能	pprof, escape, allocation	profiling, multiprocessing, native libs

10.2 分布式系统必须掌握

- CAP 不是系统设计万能公式；先明确网络分区下你要保持什么 correctness。

- Leader election、lease、fencing token：防止旧 leader 是。
- At-least-once delivery 要求消费者幂等；exactly-once 往往依赖事务/去重界。
- Backpressure、bounded queue、load shedding 是高负载平台稳定性的核心。
- 一致性模型：linearizable、sequential、eventual；K8s controller 常使用最终一致 reconcile。
- 分布式锁、cache invalidation、split-brain、clock skew、retry storm 都是高频追问。

11. GPU / CUDA 基：Infra Engineer 需要到什深度

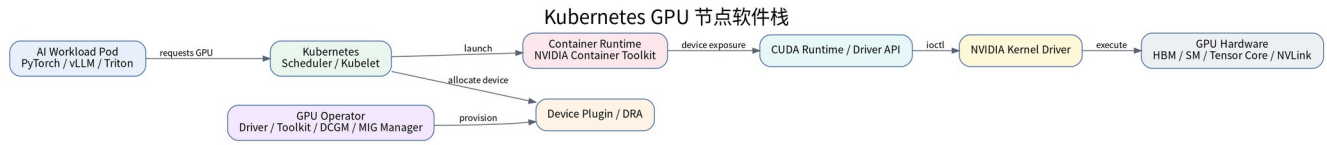


图 6：Kubernetes GPU 节点从 workload 到 hardware 的软件栈。

11.1 GPU 硬件概念

- GPU 的优势来自大量并行执行单元与高带宽 HBM，适合矩阵/向量计算。
- SM 是执行资源组织单位；warp 是 NVIDIA SIMT 执行中的基本线程组概念。
- Tensor Core 矩算加速，/推理使用 FP16/BF16/FP8 等精度以换取吞吐和显存效率。
- HBM 容量决定模型/ KV cache 可容纳量，HBM bandwidth 往往影响 decode 等 memory-bound 段。
- PCIe 是 CPU/GPU 与部分 GPU 间通路；NVLink/NVSwitch 提供更高带宽低延迟 GPU 互联。

11.2 CUDA 软件栈

Application -> Framework (PyTorch/vLLM) -> CUDA runtime/libraries -> NVIDIA driver -> GPU

- Driver 与 CUDA runtime/toolkit 的版本兼容是常见生产问题。
- cuBLAS/cuDNN/NCCL 等库分别服务不同计算/通信场景。
- GPU kernel launch 是异步的；stream/event 影响执行依赖与并发。
- Pinned memory、DMA、GPUDirect RDMA 是理解高性能数据移动的关键。

11.3 GPU 故障信号

信号	可能原因	排查
GPU util 低但请求排队	CPU bottleneck、batching、IO、routing、small batch	DCGM + app metrics + CPU profile
HBM 接近满	模型太大、KV cache、fragmentation	nvidia-smi/DCGM/vLLM KV metrics
Xid error	GPU/driver/hardware/PCIe 等异常	kernel log, DCGM diagnostics, isolate node
ECC error	显存/硬件健康问题	DCGM, vendor diagnostics
NCCL hang	fabric/config/topology/process failure	NCCL_DEBUG, rdma counters, topology, rank logs

12. Kubernetes GPU Scheduling / Operator / DRA



图 7：DRA/Device allocation 与 topology-aware placement 的逻辑链。

12.1 传统 Device Plugin 模型

- GPU 常以 extended resource，例如 nvidia.com/gpu，出现在 Node capacity/allocatable。
- Pod 请求整数 GPU，scheduler 根据展源做 fit；kubelet device manager 调用 plugin Allocate，把 device/env/mount 等信息交给 runtime。
- 优点是简单成熟；不足是表达复杂设备属性、动态配置、共享与拓扑关系的能力有限。

12.2 DRA

Kubernetes 官方文档显示 Dynamic Resource Allocation 自 v1.35 起 stable。它通过 DeviceClass、ResourceClaim、ResourceSlice 等对象，为 accelerator 等设备提供更灵活的过滤、共享和 per-workload 配置。

- DeviceClass：管理员定义面向 workload 的设备类别/策略。
- ResourceSlice：driver 发布可用设备及属性。
- ResourceClaim：workload 对设备的声明，可被 Pod 引用。
- CEL filtering：按设备属性进行更细粒度选择。
- Device sharing / vendor-specific config：比传统 extended resource 更能表达现代 accelerator。

12.3 MIG / Time Slicing / Full GPU

方式	隔离	优点	风险/限制	适用
Full GPU	最高	性能可预测、简单	碎片化、成本高	大模型/高性能
MIG	硬件级 partition	显存/compute 隔离较强、提高利用率	profile 固定、拓扑/重配置复杂	稳定多租户小模型
Time Slicing	时间共享	提高并发与利用率	显存/性能隔离弱，噪声邻居	开发/低敏感 workload

NVIDIA GPU Operator 的 MIG Manager 用于在 Kubernetes 节点管理 MIG 配置。生产环境需要考虑重配置期间 workload drain、节点 label、profile compatibility 与 capacity planning。

12.4 GPU Scheduler 高级主题

- Gang scheduling：分布式 job 需要一组 Pod/GPU 同时满足，否则先启动的成员会占资源但无法工作。
- Topology-aware：优先把 tensor-parallel ranks 放在 NVLink/NVSwitch 连接更好的 GPU 上，跨节点时考虑 RDMA fabric。
- Quota/fairness：不同 team/model 的稀缺 GPU 分配，涉及 priority、preemption、fair share。
- Heterogeneous GPU：H100/B200/MI300 等性能与显存不同，scheduler 需要 capability/成本感知。
- Defragmentation：MIG/大 GPU job 可能因为零散资源无法放置，需要重排/配额/队列策略。

13. LLM Inference Infrastructure

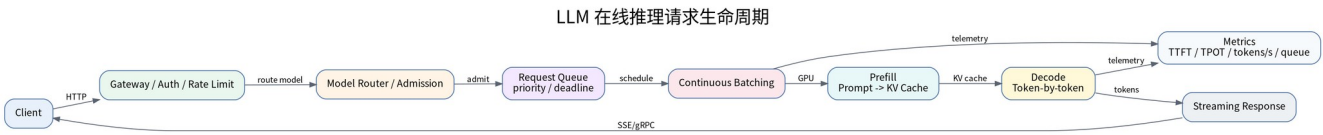


图 8：LLM 在线推理从 gateway 到 continuous batching、prefill/decode 与 streaming。

13.1 推理和传统 Web Service 最大差异

- 请求成本取决于 input/output token 数，不是固定 CPU 时间。
- 模型权重巨大、GPU 显存昂贵，扩容冷启动可能是分钟级而非秒级。
- Prefill 与 Decode 的计算/内存特征不同；decode 需要持续读取 KV cache。
- Continuous batching 会动态把不同请求的 token step 组合，提高 GPU utilization。
- 尾延迟和吞吐存在强 trade-off：batch 越大通常吞吐越高，但 queue/TTFT 可能恶化。

13.2 四个核心指标

指标	定义	主要影响因素	优化思路
TTFT	请求到第一个 token	queue、prefill、冷启动、routing	admission、prefill capacity、warm pool
TPOT	后续 token 间隔	decode、HBM bandwidth、batch size	kernel/quantization/batching
Throughput	tokens/s	batching、GPU、parallelism	continuous batching、spec decode
KV Cache Util	KV cache 占用	context length、并发、page mgmt	PagedAttention、eviction、prefix cache

13.3 KV Cache

Transformer autoregressive decode 会重复使用前序 token 的 key/value。缓存这些结果避免重复计算，但显存消耗随并发和上下文长度增长，因此 KV cache 是推理系统容量与 admission control 的中心资源。

13.4 vLLM

- vLLM 的核心工程价值包括 PagedAttention/高效 KV cache 管理、continuous batching 和 OpenAI-compatible serving。
- Kubernetes 部署时必须考虑 GPU request、shared memory、model cache/PVC、readiness/liveness、rolling update 与 startup time。
- 正的生需要 model routing、autoscaling、capacity guardrail、metrics、灰度、fallback。

13.5 Autoscaling 不能只看 GPU utilization

Signal	优点	缺点
GPU utilization	简单	可能滞后；memory-bound/小 batch 情况误导
QPS	容易获取	不同 token 长度成本差异巨大
running requests	更接近并发	忽略 token work
queue depth / waiting tokens	直接反映压力	需应用暴露
TTFT SLO	用户视角最好	反馈较慢
tokens/s + token backlog	接近真实 work	实现更复杂

推荐 Inference autoscaling 更适合使用 queue/backlog + token workload + SLO 的组合，并加入 GPU

provisioning/model loading 的长冷启动预测。

14. Ray / KubeRay / Distributed Inference

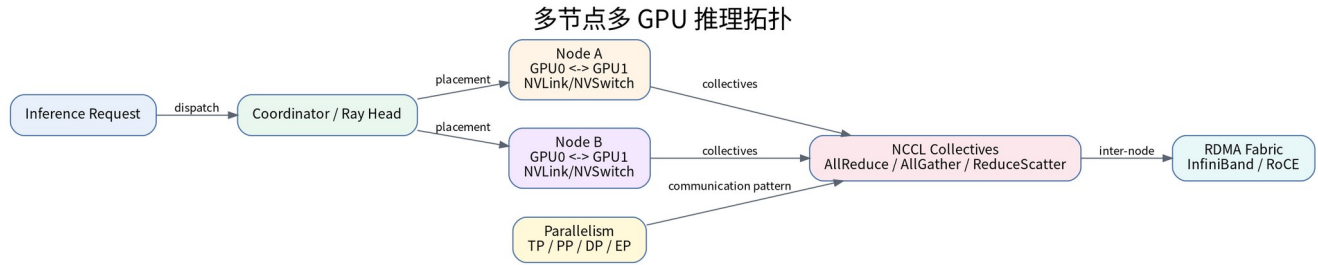


图 9：多节点 GPU 推理的 placement、NCCL collectives 与 RDMA fabric。

14.1 KubeRay

Ray 官方文档推荐在 Kubernetes 上使用 KubeRay operator。KubeRay 提供 RayCluster、RayJob、RayService 三类核心 CRD，并支持异构节点、GPU 和 autoscaling。

CRD	用途	面试关注
RayCluster	直接管理 Ray head + workers	lifecycle、autoscaling、failure
RayJob	提交一次性 job，可管理 cluster 生命周期	job 状态、cleanup、retry
RayService	Ray Serve + RayCluster 的服务化管理	zero-downtime upgrade、HA、serving

14.2 Parallelism

并行方式	思想	通信特征	常见场景
Tensor Parallel	一层权重切到多个 GPU	频繁 collective	单模型放不进一张 GPU / 低延迟
Pipeline Parallel	模型层按 stage 切分	stage 间 activation	跨节点/大模型
Data Parallel	复制模型分不同请求/数据	相对独立，训练时梯度同步	吞吐扩展
Expert Parallel	MoE expert 分布	all-to-all 显著	MoE 模型

14.3 NCCL 与网络

NCCL 提供 AllReduce、Broadcast、AllGather、ReduceScatter、AllToAll 等 GPU collective。性能取决于拓扑、通信算法、GPU Direct RDMA、NIC/PCIe/NVLink 路径和进程布局。

必须理解的通信路径

GPU HBM -> NVLink/NVSwitch (intra-node) -> PCIe/NIC -> RDMA fabric (IB/RoCE) -> remote NIC/PCIe -> remote GPU

- GPUDirect RDMA 目标是减少 CPU bounce buffer 和多余 copy。
- InfiniBand 提供专用 HPC fabric；RoCE 在 Ethernet 上传 RDMA，需要更谨慎处理 loss/congestion/PFC/ECN。

- NCCL hang 排查要按 rank 日志、collective progress、NIC/RDMA counter、topology、peer failure 系统化进行。

15. Agent Runtime / Sandbox Infrastructure

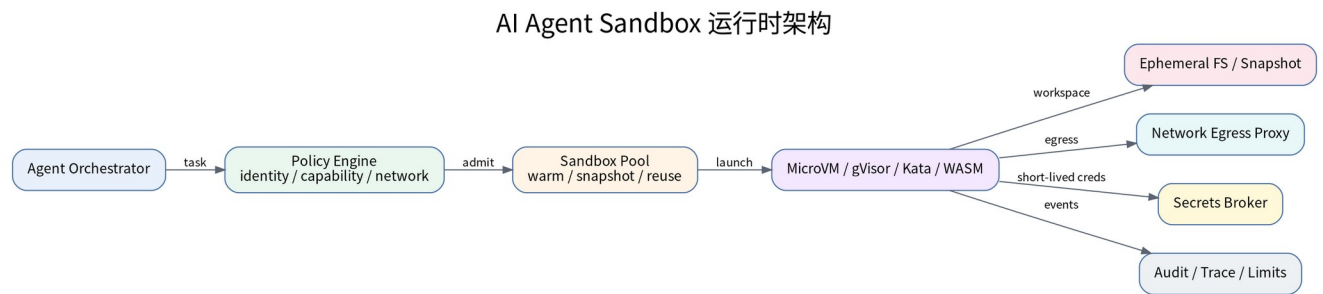


图 10: Agent sandbox 的控制面、隔离 runtime、network、secret 与 audit。

15.1 为什么 Agent 需要不同于普通容器的平台

- Agent 会执行不完全可信的代码、shell、browser、package install，风险模型比普通服务更高。
- 任务通常短生命周期且 bursty，需要毫秒到秒级启动、warm pool、snapshot/restore。
- Agent 需要外部网络和凭证，但权限必须是 task-scoped、短时、可审计。
- workspace 需要可丢弃、可快照、可复制，有时需要保留执行状态以 resume。

15.2 Sandbox 技术选择

技术	隔离强度	启动/密度	兼容性	适合
普通容器	中	极佳	最高	可信 workload
Rootless container	中+	极佳	高	降低 host root 风险
gVisor	中高	好	较高但 syscall 有差异	多租户代码执行
Kata Containers	高	中	高	需要 VM 边界又要 K8s UX
Firecracker/MicroVM	高	中好	完整 Linux guest	Agent sandbox / FaaS
WASM	强 capability sandbox	极佳	系统调用能力受限	插件/脚本/受限执行

15.3 Agent Sandbox System Design 必答

- Admission：谁能运行什么镜像/命令/网络目标。
- Isolation：kernel boundary、user namespace、seccomp、capabilities、SELinux/AppArmor。
- Network：default deny、egress proxy、DNS、domain allowlist、anti-SSRF。
- Secrets：broker 注入短期 token，不把长期凭证写进 sandbox filesystem。
- Resource：CPU/memory/pids/io/ephemeral disk/time quota。
- Lifecycle：warm pool、snapshot、hibernate、garbage collection。
- Audit：command、network destination、file change、secret access、policy decision。

16. AI Infra System Design

16.1 答模板

13. Clarify：用户、模型规模、QPS/tokens、latency SLO、region、GPU 类型、availability、成本。
14. Capacity：单 GPU benchmark -> replicas -> headroom -> failure reserve。
15. Control plane：model registry、deployment、scheduler、autoscaler、quota。
16. Data plane：gateway、router、queue、batching、model server、streaming。
17. State：model weights、KV cache、session、artifact/cache。
18. Failure：GPU/node/AZ、router、model load、network partition、NCCL。
19. Observability：TTFT/TPOT/tokens/s/queue/GPU/HBM/error/SLO burn。
20. Security：tenant isolation、auth、secret、network policy。
21. Cost：GPU utilization、bin packing、right-sizing、spot/preemptible、cold/warm trade-off。

16.2 经典设计题 A：100k QPS LLM Serving

- 不要直接从“多少 Pod”开始。先把 QPS 转成 token workload：input tokens/s + output tokens/s。
- 按模型/GPU 做 benchmark，获得不同 concurrency/batch 下 TTFT/TPOT/throughput 曲线。
- router 需要 model-aware 与 capacity-aware，避免把请求打到 KV cache/queue 已饱和实例。
- autoscaler 采用 token backlog/queue + SLO，并预测 GPU node provisioning + model load delay。
- 部署策略需要 staged rollout，保证新权重加载完成并 warmup 后才切流。
- 跨 region 设计应考虑模型复制成本、数据合规、用户 RTT、故障转移。

16.3 经典设计题 B：GPU Fleet Scheduler

- 资源模型：GPU type、HBM、MIG、NVLink domain、NIC/RDMA、NUMA、健康状态。
- 队列模型：team quota、priority、fair share、gang、preemption。
- placement：先满足硬 topology，再做 bin packing/fragmentation/energy/cost score。
- 故障：GPU unhealthy 应 cordon/quarantine，任务 checkpoint/retry；避免坏 GPU 反回池。
- 可观测：pending reason、allocation latency、GPU fragmentation、preemption、utilization、job success。

16.4 经典设计题 C：Agent Sandbox Platform

- API 接受 task spec，policy engine 生成 capability contract。
- sandbox manager 从 warm pool 分配 microVM/gVisor sandbox，挂载 ephemeral workspace。
- network proxy 和 secret broker 分别控制 egress 与 credential。
- 任务结束后收集 artifact/audit，彻底销毁或安全 snapshot。
- 容量按 concurrent sandboxes、startup latency、CPU/memory/disk/egress。

17. Portfolio：建议做一个可面试的真实项目

17.1 目：Production-Grade Kubernetes LLM Inference Platform

项目目标不是“把 vLLM Deployment 跑起来”，而是展示 control plane、scheduling、observability、autoscaling、reliability 与成本优化。

22. Phase 1：GPU node + GPU Operator + vLLM，记录单卡 benchmark。
23. Phase 2：Prometheus/DCGM + inference metrics，建立 TTFT/TPOT/tokens/s/KV cache dashboard。

- 24. Phase 3：实现基于 queue/token backlog 的 autoscaling，而不是仅 CPU/GPU util。
- 25. Phase 4：加入 KubeRay，多 GPU/多节点推理，做 NCCL benchmark。
- 26. Phase 5：加入 model router、canary rollout、model warmup 与 fallback。
- 27. Phase 6：模拟 GPU Xid/node failure/NCCL hang，写 runbook 与自动隔离策略。
- 28. Phase 7：成本实验：MIG/full GPU、不同 instance、batch size、concurrency 的 \$/1M tokens。

17.2 面试时怎么讲项目

Problem -> Constraints -> Baseline -> Bottleneck -> Design -> Measurement -> Failure -> Trade-off -> Result -> Next step

每个项目准备一个 3 分钟版本和一个 20 分钟 deep-dive 版本。Senior 面试最看重你如何发现瓶颈、为什么选该方案、怎么验证，以及上线后发生了什么。

18. 12 周准备计划

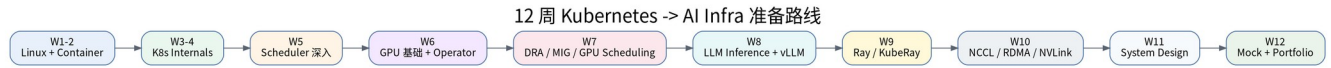


图 11：12 周从 Kubernetes 深化到 GPU/Inference 的学习节奏。

周	主线	输出物
1	Linux CPU/process	解释 load、context switch、throttling；perf 实验
2	Memory/IO/network	OOM、page cache、TCP/contrack 故障实验
3	Container runtime	namespace/cgroup v2/CRI/OCI 调用链笔记
4	API Server/etcd/Informer/Controller	源码流程图 + 自写 controller
5	Scheduler	queue/framework/preemption/bin packing 源码笔记
6	Kubelet/CNI/CSI	Pod startup 端到端流程 + 故障注入
7	GPU/CUDA/GPU Operator	GPU 软件栈 + DCGM dashboard
8	MIG/DRA/GPU scheduling	设备分配/拓扑/fragmentation 实验
9	vLLM inference	TTFT/TPOT/KV cache/batching benchmark
10	KubeRay/NCCL/RDMA	distributed inference + NCCL test
11	AI Infra system design	3 道完整设计题 + capacity spreadsheet
12	Mock/Portfolio	简历项目、STAR、模拟面试、补洞

18.1 每日 3 小模板

时间	内容
60 min	系统/K8s 原理 + 源码

45 min	GPU/Inference 新知识
30 min	Go/Python coding
30 min	System Design / troubleshooting
15 min	口述复盘与 flashcards

19. 高频面试题库（按追问链准备）

Linux CPU / Process

1. load average 的三个数字是什么？为什么 load 高但 CPU 不高？
2. 进程和线程的地址空间/调度区别？
3. fork/exec 的完整过程？Copy-on-write 是什么？
4. Zombie 与 orphan process 的区别？
5. D state 为什么 kill 不掉？
6. context switch 的成本来自哪里？
7. CPU 100% 如何判断 user/sys/softirq/throttling？
8. 节点很空为什么容器仍会 CPU throttling？
9. CPU affinity/NUMA 为什么影响 tail latency？
10. 什么是 PSI，如何用于容量/压力判断？

Memory

1. RSS/VSZ/PSS 区别？
2. page cache 为什么不是内存泄漏？
3. minor/major page fault 区别？
4. memory reclaim 如何工作？
5. swap 对延迟有什么影响？
6. global OOM 与 cgroup OOM 区别？
7. OOM killer 如何选 victim？
8. K8s OOMKilled 与 eviction 区别？
9. memory.max/high/low/min 的区别？
10. 如何排查容器内存持续增长？

Filesystem / IO

1. inode/dentry/file/fd 区别？
2. 删除大文件为什么空间未释放？
3. df 有空间但 No space left 的原因？

4. page cache 与 direct IO 区别？
5. fsync 为什么慢？
6. iowait 高如何排查？
7. IOPS/throughput/latency/queue depth 关系？
8. overlayfs 对容器写放大有什么影响？
9. ephemeral-storage request/limit 如何工作？
10. NVMe local cache 在模型加载中的作用？

Network

1. TIME_WAIT/CLOSE_WAIT 区别？
2. SYN_SENT 多说明什么？
3. TCP retransmission 常见原因？
4. conntrack table 满会怎样？
5. 容器 veth packet path？
6. Service ClusterIP 如何实现？
7. SNAT 为什么会导 port exhaustion？
8. MTU mismatch 的症状？
9. DNS ndots 如何导致延迟？
10. 如何定位跨 AZ latency/packet loss？

K8s API/etcd/Informer

1. APIServer 请求链？
2. resourceVersion 的作用？
3. Watch 后 informer 怎么恢复？
4. DeltaFIFO 为什么不是普通 queue？
5. Indexer 与 informer handler 的关系？
6. 为什么 controller 使用 workqueue？
7. reconcile 为什么必须幂等？
8. etcd Raft 为什么通常奇数节点？
9. etcd compact/defrag 区别？
10. 如何排查 API Server/etcd latency？

Scheduler

1. activeQ/backoffQ/unschedulablePods 的区别？
2. PreFilter/Filter/Score/Bind 各做什么？
3. NodeResourcesFit 三种 scoringStrategy？

4. NodeResourcesBalancedAllocation 解决什么？
5. preemption 的 victim 如何选择？
6. PDB 如何影响 preemption？
7. MostAllocated 为什么利于 autoscaler scale-down？
8. scheduler 为什么主要看 requests 而不看实时 utilization？
9. gang scheduling 为什么对 AI job 重要？
10. GPU topology-aware scheduling 如何设计？

Kubelet/Runtime

1. kubelet 如何得知 Pod 被调度到本节点？
2. SyncLoop/PodWorkers/SyncPod 的关系？
3. RunPodSandbox/CreateContainer/StartContainer 顺序？
4. CRI 与 OCI 的边界？
5. containerd 与 runc 的关系？
6. Pod network namespace 什么时候创建？
7. request/limit 如何映射到 cgroup v2？
8. readiness/liveness/startup probe 区别？
9. Pod Termination 的完整流程？
10. Node NotReady 时 workload 会发生什么？

CNI/CSI

1. CNI ADD 做什么？
2. Pod-to-Pod 跨节点路径？
3. NetworkPolicy 在哪里 enforce？
4. IP exhaustion 怎么处理？
5. CSI Controller/Node service 区别？
6. WaitForFirstConsumer 为什么重要？
7. PVC attach/mount 卡住如何排查？
8. StatefulSet volume topology 怎么处理？

GPU

1. GPU SM/HBM/Tensor Core 分别是什么？
2. CUDA runtime 与 driver 的关系？
3. 为什么 HBM bandwidth 会限制 decode？
4. MIG 与 time slicing 区别？
5. GPU Operator 部署哪些关键组件？

6. DCGM 监控哪些指标？
7. Xid error 如何处理？
8. GPU requests 如何被 scheduler/kubelet/runtime 传递？
9. DRA 相比 device plugin 改善什么？
10. NUMA/PCIe/NVLink topology 为什么影响性能？

Inference

1. TTFT/TPOT 如何定义？
2. Prefill 和 Decode 的差异？
3. KV cache 为什么会成为容量瓶颈？
4. PagedAttention 解决什么？
5. continuous batching 为什么提高吞吐？
6. batch size 越大为什么不一定越好？
7. 推理 autoscaling 用什么 signal？
8. 模型冷启动如何优化？
9. 如何做模型 canary/rollback？
10. 如何估算一个模型需要多少 GPU？

Distributed AI

1. TP/PP/DP/EP 区别？
2. NCCL AllReduce/AllGather/ReduceScatter 用途？
3. NVLink 与 RDMA 的角色？
4. InfiniBand 与 RoCE 区别？
5. GPUDirect RDMA 为什么快？
6. NCCL hang 如何排查？
7. 多节点 inference 的 failure model？
8. gang scheduling 与 distributed job 的关系？

Agent Runtime

1. 容器能否作为不可信代码 sandbox？
2. gVisor/Kata/MicroVM 的 trade-off？
3. 如何做 network egress policy？
4. sandbox 如何安全拿到 cloud credential？
5. warm pool/snapshot 如何降低启动延迟？
6. 如何限制 fork bomb/磁盘写爆/网络滥用？
7. 如何审计 agent 执行？

8. sandbox reuse 有什么数据泄露风险？

System Design

1. 设计一个 100k QPS LLM serving platform。
2. 设计 GPU fleet scheduler。
3. 设计 multi-tenant GPU Kubernetes。
4. 设计 model rollout system。
5. 设计 inference autoscaler。
6. 设计 Agent sandbox service。
7. 设计跨 region model serving。
8. 设计 GPU node health/quarantine system。
9. 设计大模型权重分发/cache。
10. 设计 AI infra observability platform。

19.5 高频核心题参考答案（面试口述版）

下面不是“背诵标准答案”，而是给出一个 Senior 级回答骨架：先定义概念，再解释机制，再讲故障/观测，最后给 trade-off。面试时根据追问深度展开。

1. 为什么 load average 高但 CPU utilization 不高？

Linux load average 统计的核心不是 CPU 百分比，而是系中于可行以及某些不可中等待的任力。CPU 只有 30% 但 load 很高时，应检查 D state、存/NFS 卡顿、锁/资源等待以及 run queue 分布。排查可以从 uptime、vmstat 的 r/b、ps state、pidstat、iostat 开始；如果大量 D state，继续追 block IO、filesystem 或远端存储。

2. 为什么容器在 Node 很空时也会 CPU throttling？

CPU limit 最终会转成 cgroup 的 CPU bandwidth 控制。cgroup v2 常见为 cpu.max，例如 quota=100ms、period=100ms 表示最多约 1 CPU。这个限制针对 cgroup 自身，并不会因为整台 Node 还有空闲 CPU 就自放宽，因此 workload 短时间想使用 2 核仍可能被 throttle。应观察 cpu.stat 的 nr_throttled/throttled_usec，以及应用 p99 latency；这也是生产环境是否设置 CPU limit 的经典 trade-off。

3. OOMKilled 和 kubelet eviction 有什？

OOMKilled 通常表示进程被 Linux OOM killer 在 global 或 memcg 范死；kubelet eviction 是 kubelet 根据 memory.available、nodefs.available 等 eviction signal 主动终止 Pod。前者需要看 kernel/memory.events/cgroup limit，后者看 kubelet event 与 eviction threshold。两者都可能表现为 Pod 重启/消失，但根因和修复式不同。

4. 除文件后磁盘空什有放？

unlink 只是删除目录项并减少 inode link count。如果仍有进程打开该 file object，inode 与 data blocks 仍被引用，所以 df 不会恢复。使用 lsof +L1 或 /proc/<pid>/fd 找 deleted-but-open 文件；关闭/重启进程后空间才释放。不要直接把“rm 成功”等同于 block 已回收。

5. TIME_WAIT 和 CLOSE_WAIT 大量出现分别说明什么？

TIME_WAIT 通常在主动关闭连接的一方，是 TCP 为旧报文和最终 ACK 可靠性保留的状态；大量 TIME_WAIT 不一定是 bug，但可能暴露短连接/端口压力。CLOSE_WAIT 表示对端已经关闭，而本地应用尚未 close socket，长期大量出现通常优先检查应用连接生命周期、goroutine/thread leak。

6. Informer 为什么先 List 再 Watch？

Controller 首先需要有一个一致的初始对象集合，所以 Reflector 通过 List 获取当前 snapshot 和 resourceVersion，再从对应版本开始 Watch 增量事件。这样本地 Store/Indexer 能从完整状态开始，并持续追踪变化。Watch 中或 resourceVersion 过期时，Reflector 会重新建立 watch，必要时 relist；因此 controller 必须接受事件重复、延迟和最一致。

7. DeltaFIFO 为什么不是普通 FIFO？

DeltaFIFO 的 value 是同一个 object key 的 Delta 列表，能够表达 Added/Updated/Deleted/Replaced/Sync 等变化，并在 key 维度聚合多次事件。这样 consumer 处理的是“这个对象发生了哪些变化”，而不是无界重复 event。它同时与 KnownObjects/Store 协作，支持 Replace 后发现删除对象等场景。

8. Controller 为什么 event handler 只 enqueue key，而不是直接理？

Handler 在 informer 分发路径上，应保持快速。把 namespace/name key 放入 rate-limited workqueue 后，worker 可以控制并发、重试、指数退避和去重。reconcile 再从 cache 获取最新对象，所以即使事件被合并也能基于当前状态收敛。这是 Kubernetes controller 可扩展与可靠性的核心设计。

9. Scheduler 为什么不用实时 CPU utilization 度？

Scheduler 的目标首先是做 admission/placement，必须基于可声明、可预测、可审计的资源保证；requests/allocatable 是稳定的契约，而实时 utilization 高度波动且存在采样延迟。若直接用瞬时 CPU 做硬，容易发生抖动和反馈环。实时负载可以通过自定义 scheduler/metrics、autoscaling、descheduler 或 workload-aware placement 辅助，但不能简单替代 requests 模型。

10. LeastAllocated 和 MostAllocated？

LeastAllocated 偏向把 Pod 分散到 request 较低的节点，通常给 burst/headroom 更大的分布；MostAllocated 偏向 bin packing，把 Pod 塞到已经较满但仍可容纳的节点，有利于释放整节点给 autoscaler scale-down。选择取决于 fault domain、workload burst、资源 request 准度、点容速度和成本目，而不是“更先”。

11. Preemption 发生时 Scheduler 在做什？

普通 Filter 找不到 feasible node 时，DefaultPreemption 在候选节点上模拟移除更低 priority 的 Pod，寻找可以容纳高优 Pod 的 victim 集合，并考虑 PDB 等约束。它不会瞬间把目标 Pod 绑定成功：victim 终止需要时间，高优 Pod 可能通过 nominatedNodeName 等状态等待。面试中要继续讨论 disruption、priority abuse 和 starvation。

12. Kubelet PodWorkers 为什么重要？

Kubelet 同时接收 Pod spec 更新、runtime event、probe/status 等异步事件。如果同一个 Pod 的 create/update/terminate 操作行，容易生生命周期。PodWorkers 为每个 Pod 管理串行工作流，把 update 合并并驱动 SyncPod，使实际状态逐步收敛到 desired state。

13. CRI 与 OCI 分别解决什么问题？

CRI 是 Kubernetes kubelet 与高层 container runtime 的接口，定义 RunPodSandbox/CreateContainer/StartContainer 等生命周期 RPC。OCI Runtime Spec 描述一个容器进程如何在 Linux 上运行，包括 namespaces、mounts、cgroup resources；runc/crun 等低层 runtime 执行 OCI spec。containerd 位于中间，通常既实现 CRI，又通过 OCI runtime 真正创建进程。

14. CNI ADD 的典型工作是什么？

Pod sandbox 的 network namespace 建后，runtime 调用 CNI plugin。典型工作包括创建/移动 veth、分配 IP、配置 route、连接 bridge/underlay/overlay、应用 policy 或 eBPF datapath。不同 CNI 架构差异很大，因此面试更应该说明通用 contract 与 packet path，而不是把某个实现当 Kubernetes 固定。

15. WaitForFirstConsumer 解了什？

对 topology-sensitive volume，如果 PVC 创建时就立刻 provisioning，可能把卷建在某个 AZ，之后 Pod 因 CPU/affinity 被调度到另一个 AZ，形成无法 attach 的冲突。WaitForFirstConsumer 延迟 volume binding/provisioning，直到 scheduler 结合 Pod 束出适 topology。它把 storage placement 与 compute placement 协调起来。

16. MIG 和 Time Slicing 最大区别？

MIG 在支持的 NVIDIA GPU 上创建硬件级 GPU instance，提供更明确的显存和计算资源隔离；Time Slicing 主要在时间维度共享同一 GPU，隔离/性能可预测性更弱。MIG 适合多租户稳定 partition，但 profile 和重配置带来碎片与运维复杂度；Time Slicing 更适合开发/低隔景。

17. DRA 相比传统 Device Plugin 的意义是什么？

传统 extended resource/device plugin 模型适合“节点有 N 个同类设备，Pod 请求整数个”的场景。DRA 引入 DeviceClass、ResourceClaim、ResourceSlice 等资源，使设备属性过滤、共享、per-workload configuration 和更复杂的设备选择成为 Kubernetes 原生对象。Kubernetes 文档标注 DRA 自 v1.35 stable，因此 accelerator scheduling 相关面试值得重点关注。

18. TTFT 与 TPOT 分别受什么影响？

TTFT 包含排队、routing、prefill、模型冷启动等，通常是交互体验最敏感的首 token 延迟；TPOT 主要反映 decode 阶段逐 token 生成速度，更容易受到 HBM bandwidth、batch composition、kernel efficiency 影响。优化两者的方法不同，单独追 GPU utilization 可能同时损害 TTFT。

19. 为什么 KV Cache 是推理容量核心？

自回归 decode 每生成一个 token 都需要复用历史 token 的 K/V。缓存这些中间结果避免重复计算，却使显存占用随请求并发和 context length 增长。KV cache 充满时即使算力还有余量，也无法安全接纳更多请求，因此 admission、PagedAttention、prefix cache、eviction 与 autoscaling 都围绕它展开。

20. Continuous Batching 为什么有效？

传统 static batching 要等待一批请求一起完成，短请求会被长请求拖住。Continuous batching 在 token step 级别动态加入/移除请求，让 GPU 持续有足够工作，提高利用率与 throughput。代价是调度复杂度，以及 throughput、queue time、TTFT 之间需要平衡。

21. Inference autoscaling 为什么不应该只看 QPS ?

LLM 单请求成本不是常量：1 个 100-token 请求和 1 个 10000-token 请求差异巨大。QPS 无法表达 input/output token、context、KV cache 和 batch efficiency。因此更好的 signal 是 waiting/running token backlog、queue depth、TTFT SLO、GPU/HBM 状态的组合，并把 node provisioning/model loading 的长冷启动纳入预测。

22. Tensor Parallel 与 Pipeline Parallel 如何取舍 ?

Tensor Parallel 把同一层的矩阵切到多 GPU，通常需要频繁 AllReduce/AllGather，通信敏感但单 token pipeline latency 较低；Pipeline Parallel 按层分 stage，stage 间传 activation，对跨节点更友好但会引入 pipeline bubble/□度复杂度。选择取决于模型大小、GPU topology、网络带宽和延迟目标。

23. NCCL hang 怎么排查 ?

第一步确定所有 ranks 是否启动、是否卡在同一个 collective；开启 NCCL_DEBUG，检查 rank/communicator 日志。第二步检查 GPU/NIC topology、IB/RoCE link、RDMA counters、MTU/PFC/ECN、GPU Xid。第三步验证 peer failure、timeout、容器 network/device mount 和版本兼容。分布式通信问题要避免只盯某个“报错 rank”，因为它可能只是最先观察到故障的一方。

24. GPUDirect RDMA 的价值是什么 ?

传统跨节点 GPU 数据传输可能需要 GPU -> host memory -> NIC，再在对端反向搬运，CPU 和内存 copy 成本高。GPUDirect RDMA 允许 NIC 更直接地 DMA GPU memory，降低 CPU involvement 和 copy，改善带宽/延迟。实际性能仍依赖 PCIe/NIC placement、IOMMU、driver、NCCL 和 fabric。

25. Agent sandbox 为什么普通容器可能不够 ?

普通容器与宿主共享内核，namespace/cgroup 本质是同一 kernel 上的隔离。对执行不可信代码、安装任意包、访问网络的 agent，kernel escape 风险和多租户 blast radius 更敏感。gVisor 通过用户态 kernel 拦截 syscall，Kata/MicroVM 提供更强 VM 边界；选择时要权衡启动延迟、兼容性、密度和隔离。

26. Agent sandbox 如何安全地拿凭证 ?

不要把长期 cloud key 作为环境变量永久注入。更好的架构是 sandbox 以 workload identity 向 secret broker 请求 task-scoped、短 TTL credential，broker 根据 policy/tenant/task，限制 audience/权限。所有请求和使用行为要审计，sandbox 销毁后 credential 自动过期。

27. 如何设计 GPU node health/quarantine ?

将健康信号分层：DCGM/Xid/ECC、PCIe/NVLink、NIC/RDMA、temperature/power、kernel/driver。节点出现高置信度硬件故障后先从 scheduler capacity 中摘除或 taint/cordon，阻止新任务；根据 job 类型 drain/checkpoint/retry。repair workflow 完成 diagnostics 后才能重新入池，避免 flapping 坏节点反复伤害 workload。

28. 如何降低模型冷启动 ?

冷启动要包括 GPU node provision、image pull、model weight download、load/deserialize、CUDA/NCCL init、kernel warmup。优化可使用 warm GPU pool、镜像/权重预取、local NVMe cache、并行下载、分层 cache、snapshot/tensorizer 类方案、staged rollout。关键是测量每个阶段，而不是把“启动慢”视作单一问题。

29. 如何设计 model rollout ?

新版本先创建独立 replica pool，加载权重并完成 readiness + warmup + synthetic validation；再按 1%/5%/25% 等阶段切流，比较 error、TTFT/TPOT、quality proxy 与 GPU efficiency。异常自动停止/rollback。要避免旧/新模型共享不可兼容 cache/session 状态，并确保容量能承受双跑窗口。

30. 如何设计一个可靠的 rate limiter/admission control ?

先区分 request rate limit 与 work/concurrency limit。对 LLM，更适合按 token budget、concurrent sequences、KV cache 或 tenant quota 做 admission。控制器应支持 priority/fairness、bounded queue、deadline/timeout、load shedding，防止 overload 进入系统后形成 retry storm。

19.6 Kubernetes / AI Infra 源码与文档导航

主题	建议入口	学习目标
kube-scheduler	kubernetes/pkg/scheduler/; framework/; backend/queue/	scheduleOne、SchedulingQueue、 framework extension points、 preemption
kubelet	kubernetes/pkg/kubelet/; kuberuntime/	syncLoop、podWorkers、SyncPod、 CRI 调用
client-go informer	client-go/tools/cache/	Reflector、DeltaFIFO、 sharedIndexInformer、processor
controller workqueue	client-go/util/workqueue/	rate limiting、dirty/processing set、 retry
API machinery	apiserver/pkg/endpoints/; registry/	handler chain、authn/authz/admission/storage
containerd CRI	containerd/internal/cri/（版本路径可能 变化）	PodSandbox/container 生命周期与 OCI spec
OCI runtime spec	opencontainers/runtime-spec	LinuxResources、namespaces、 mounts
KubeRay	ray-project/kuberay	RayCluster/RayJob/RayService reconcile
vLLM	vllm-project/vllm	scheduler、KV cache、worker/executor、serving
NCCL	NVIDIA/nccl + 官方 user guide	collectives、transport、topology、 debug

20. 30 个必须能画出来的白板题

- Pod 创建到 Running
- API Server request path
- Informer List/Watch/DeltaFIFO/Indexer
- Controller workqueue/reconcile

- Scheduler queues + framework
- Kubelet SyncPod + CRI
- CNI Pod network path
- CSI volume lifecycle
- Service ClusterIP packet path
- cgroup v2 request/limit mapping
- TCP connection lifecycle
- Linux read()/write() path
- Linux memory reclaim/OOM
- Container namespace/cgroup stack
- GPU software stack
- Device Plugin allocation
- DRA objects and allocation
- MIG/full GPU/time-slicing comparison
- Inference request pipeline
- KV cache lifecycle
- Continuous batching
- TP/PP/DP/EP comparison
- NCCL intra/inter node path
- RDMA / GPUDirect path
- KubeRay RayService lifecycle
- Inference autoscaling loop
- GPU fleet health loop
- Model rollout/canary
- Agent sandbox architecture
- Layered troubleshooting tree

21. 实验清单：把“知道”变成“做过”

实验	步骤概要	你应该记录什么
CPU throttling	容器设 500m limit，运行 stress，多核节点观察 cpu.stat	throttled_usec、latency、node idle 对比
memcg OOM	设置低 memory.max 逐步分配	memory.events、dmesg、Pod status
Informer	写 client-go informer + workqueue	List/Watch、cache sync、重试
Scheduler	自定义 scheduler profile / MostAllocated	节点 request 分布、scale-down candidate
CNI	抓 veth/host packet	namespace、route、NAT path
etcd	制造 fsync latency/大对象（测试环境）	request latency、leader、db size
GPU Operator	安装并观察组件	driver/toolkit/device-plugin/DCGM/MIG
MIG	切 profile 并调度 workload	capacity、隔离、重配置流程
vLLM	不同 concurrency/context benchmark	TTFT、TPOT、tokens/s、KV cache
KubeRay	RayService 多 GPU serving	autoscaling、worker failure
NCCL test	单节点/跨节点 all-reduce	bandwidth、topology、NCCL_DEBUG
Agent sandbox	MicroVM/gVisor 运行受限脚本	startup、syscall compatibility、egress

		policy
--	--	--------

22. Sources / Research References

以下资料用于 2026 岗位趋势与技术状态 research。技术细节仍应在面试前根据目标公司的运行栈和目标 Kubernetes/GPU 版本再次确认。

ID	Source	URL
R1	OpenAI - Software Engineer, Compute Infrastructure	https://openai.com/careers/software-engineer-compute-infrastructure-san-francisco/
R2	Anthropic - Staff+ Software Engineer, Safeguards ML Infrastructure	https://www.anthropic.com/careers/jobs/4778843008
R3	CoreWeave Careers	https://www.coreweave.com/careers
R4	Lambda Careers	https://lambda.ai/careers/open-roles
R5	Together AI Careers	https://www.together.ai/careers
R6	Kubernetes - Dynamic Resource Allocation	https://kubernetes.io/docs/concepts/resource-management/dynamic-resource-allocation/
R7	NVIDIA GPU Operator - MIG	https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/26.3/gpu-operator-mig.html
R8	NVIDIA GPU Operator - DRA Driver	https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/26.3/dra-intro-install.html
R9	Ray on Kubernetes / KubeRay	https://docs.ray.io/en/latest/cluster/kubernetes/index.html
R10	NVIDIA NCCL Documentation	https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/index.html
R11	vLLM Kubernetes Deployment	https://docs.vllm.ai/en/v0.7.0/deployment/k8s.html
R12	Crusoe - Slurm on Managed Kubernetes (2026)	https://www.crusoe.ai/resources/blog/slurm-on-crusoe-managed-kubernetes-how-we-built-managed-gpu-training-infrastructure
R13	Crusoe - AMD MI300X workloads on Kubernetes (2026)	https://www.crusoe.ai/resources/blog/running-ai-workloads-on-amd-gpus-with-skypilot
R14	Lambda at NVIDIA GTC 2026	https://lambda.ai/blog/lambda-at-gtc-2026-building-the-

		superintelligence-cloud
--	--	-------------------------

23. 最后检查：面试前一周 Checklist

- 能在 10 分钟内无资料画出 Kubernetes Pod lifecycle。
- 能从 Pending reason 解 Scheduler queue/filter/score/preemption。
- 能从 requests/limits 讲到 cgroup v2 文件和 kernel behavior。
- 能解释 OOMKilled vs eviction、TIME_WAIT vs CLOSE_WAIT、load vs CPU。
- 能解释 GPU Operator、Device Plugin、MIG、DRA 的角色边界。
- 能解释 TTFT/TPOT/KV Cache/Continuous Batching。
- 能画出 NCCL + NVLink + RDMA 跨节点通信。
- 能做 3 道 AI Infra system design，并给出容量估算。
- 准备 4 个 STAR：事故、性能优化、架构决策、跨团队推动。
- 准备一个完整的 Kubernetes AI inference 项目故事，并有 benchmark/metrics 证据。

最终建议 优先把“系统深度 + 可验证工程项目”做扎实。AI Infra 面试对 ML 数学的要求因岗位而异，但对可靠性、分布式系统、Linux/Kubernetes、性能与硬件意识的要求通常非常真实。

24. 扩展版 Deep Dive: Linux Kernel 与性能诊断

本章把 Linux 从“会用命令”扩展到“能解释内核路径、资源竞争与性能信号”。目标不是记住所有实现细节，而是能够从现象建立假设、选择观测手段，并把结论连接到 Kubernetes 与 AI workload。

CPU 性能问题：从现象到内核调度路径

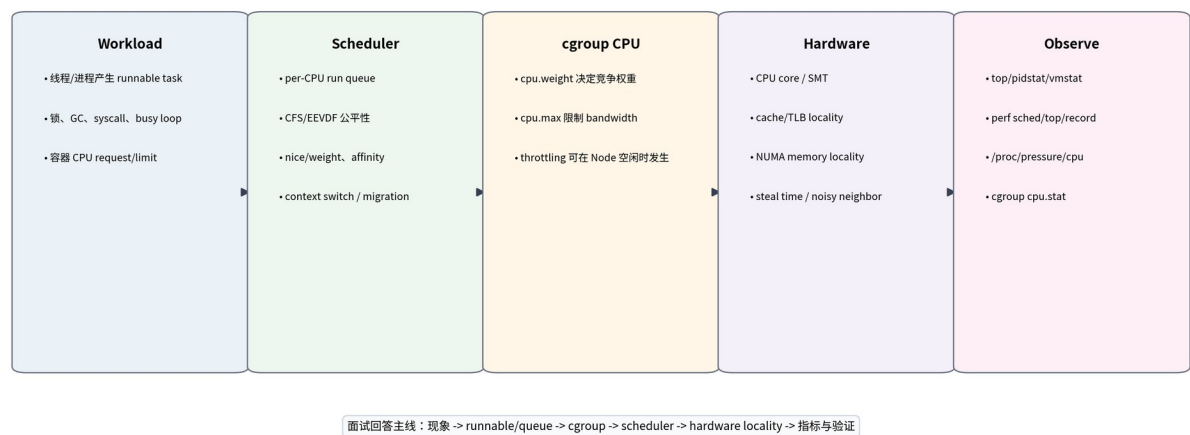


图 24-1：CPU 性能问题的跨层诊断主线。

24.1 CPU Scheduler：面试需要掌握到什么程度

- 每个逻辑 CPU 都维护运行队列。调度器不仅决定“谁运行”，还要处理负载均衡、CPU affinity、NUMA locality、抢占和任务迁移。
- CPU utilization 高表示 CPU 忙；load average 高表示 runnable 或不可中断任务多。两者不同，因此 load 高但 CPU 不高往往需要看 D 状态和 IO 等待。
- cgroup v2 的 cpu.weight 是相对权重，只有竞争时才体现；cpu.max 是 bandwidth ceiling，会产生 throttling。
- 对延迟敏感 workload，需要关注 run-queue latency，而不仅仅是 CPU utilization。PSI、perf sched、runqlat 类 eBPF 工具比单纯 top 更能解释 tail latency。
- AI Serving 中 CPU 仍然重要：tokenization、HTTP/gRPC、request routing、KV metadata、NCCL progress、CUDA launch 都可能受 CPU 调度和 NUMA 影响。

现象	优先假设	验证方式	常见修复
CPU 100%	计算热点 / busy loop / 锁竞争	top -H, perf top, flame graph	优化热点、并行度、算法或锁
CPU 不高但 load 很高	D 状态 / IO wait / blocked task	ps state, vmstat, iostat, wchan	定位存储/内核等待
p99 抖动但平均 CPU 不高	run queue / throttling / IRQ / NUMA	PSI, cpu.stat, perf sched	调 CPU limit、pinning、NUMA、IRQ
容器持续 throttling	cpu.max 过紧	cpu.stat	提高/移除 CPU limit，核对 SLO

		nr_throttled/throttled_usec	
--	--	-----------------------------	--

24.2 Memory：从虚拟内存到 cgroup OOM

内存题最容易被问到 “free 很少是否异常” “OOMKilled 与 eviction 的区别” “page cache 是不是浪费”。回答时建议始终从 virtual memory -> page fault -> reclaim -> cgroup/global pressure -> OOM/eviction 这条链路组织。

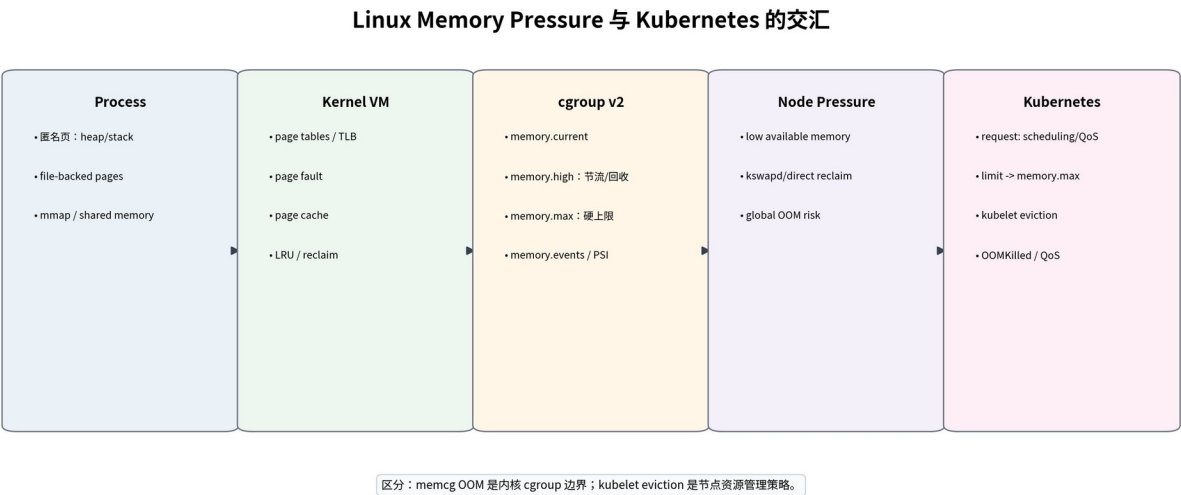


图 24-2：Linux 内存压力、cgroup 与 Kubernetes 的关系。

- page cache 可回收，不能把 “used” 直接等价成不可释放内存；生产上更应该看 MemAvailable、reclaim、swap、PSI。
- memory.high 可以制造压力并触发 reclaim，而 memory.max 是硬上限；达到 max 后分配可能失败并进入 cgroup OOM 路径。
- Kubernetes memory request 首先影响 scheduling 和 QoS；memory limit 通常最终落到 cgroup memory.max。
- 节点 memory pressure 下 kubelet 可能在内核 OOM 之前主动 eviction Pod；因此 OOMKilled 与 Evicted 是两条不同的生命周期。
- AI Inference 的 HBM 与 host memory 同样需要容量模型：模型权重、KV cache、CPU pinned memory、page cache 和 runtime overhead 都可能成为瓶颈。

24.3 Storage / IO：从 VFS 到 NVMe

层次	核心对象/机制	高频指标	典型故障
应用/VFS	fd, inode, dentry, mmap, fsync	open fd, syscall latency	fd leak, inode exhaustion
Page Cache	cached pages, writeback	dirty/writeback, major fault	cache thrash, writeback stall

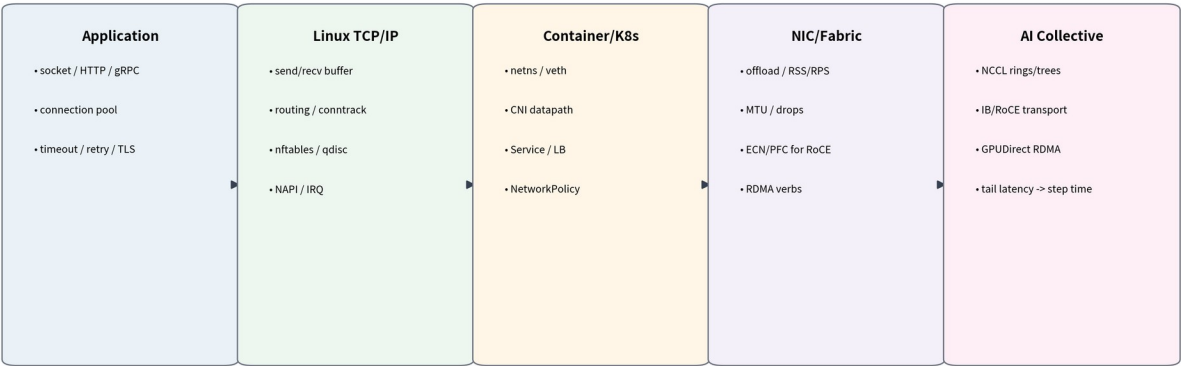
Filesystem	ext4/xfs metadata/journal	fsync latency	metadata contention
Block Layer	bio/request/queue	await, queue depth, IOPS	queue saturation
Device/NVMe	controller/namespace/PCIe	latency, bandwidth, errors	thermal, PCIe link, media error

面试陷阱

iowait 不是“磁盘利用率”。它表示 CPU 空闲时存在未完成 IO；高 iowait 需要结合 device latency、queue depth、application blocking 一起解释。

24.4 Network: packet path 与 tail latency

Kubernetes/AI Workload 的数据包路径



网络面试不仅回答 TCP；AI Infra 还要能从 NIC/PCIe/RDMA/NCCL 一路排查。

图 24-3：普通 K8s 网络到 AI collective communication 的扩展路径。

24.5 Linux 工具选择矩阵

问题	第一组工具	第二组工具	深入工具
CPU hotspot	top/pidstat	perf top	perf record + flame graph
调度延迟	vmstat/PSI	perf sched	eBPF runqlat/offcputime
内存增长	free/smmaps	memory.stat	heap profiler / bpfftrace
IO 慢	iostat	pidstat -d	blktrace/bpfftrace
TCP 慢	ss -ti	tcpdump	eBPF tcp retrans/latency
系统调用异常	strace	perf trace	bpfftrace

25. Kubernetes Internals 扩展：源码级主链路与故障路径

Senior Kubernetes 面试最有价值的能力不是列出组件，而是能把对象生命周期串起来：API 写入 -> watch/cache -> controller/scheduler -> kubelet -> runtime/network/storage -> status 回写，并且知道每个环节如何失败。

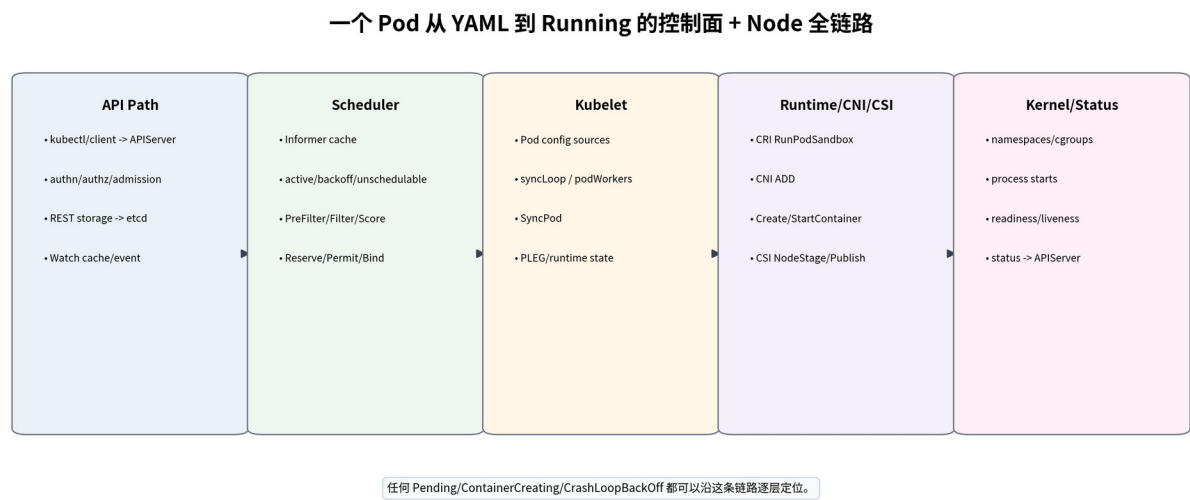


图 25-1：Pod 生命周期源码级主链路。

25.1 kube-apiserver：请求与 Watch 的两个关键路径

路径	核心步骤	常见故障	面试追问
写请求	authn -> authz -> admission -> validation/defaulting -> storage	Webhook timeout, etcd latency, APF throttling	为什么 webhook 能拖慢整个控制面？
读请求	REST handler -> storage/cache -> serialization	大 List, expensive selector	为什么 List 大对象会打爆 APIServer？
Watch	resourceVersion -> watch cache -> stream	410 Gone, slow watcher	为什么 controller 必须能 relist？
APF	FlowSchema/PriorityLevel	queue/reject	如何保护核心控制器请求？

源码阅读建议：先从 genericapiserver 的 handler chain 与 REST storage 抽象理解，不必一开始陷入所有 generated code。面试更关注“请求在哪一层被拒绝/修改/持久化”和“Watch 如何维持增量视图”。

25.2 Informer / Controller：并发、缓存与最终一致性

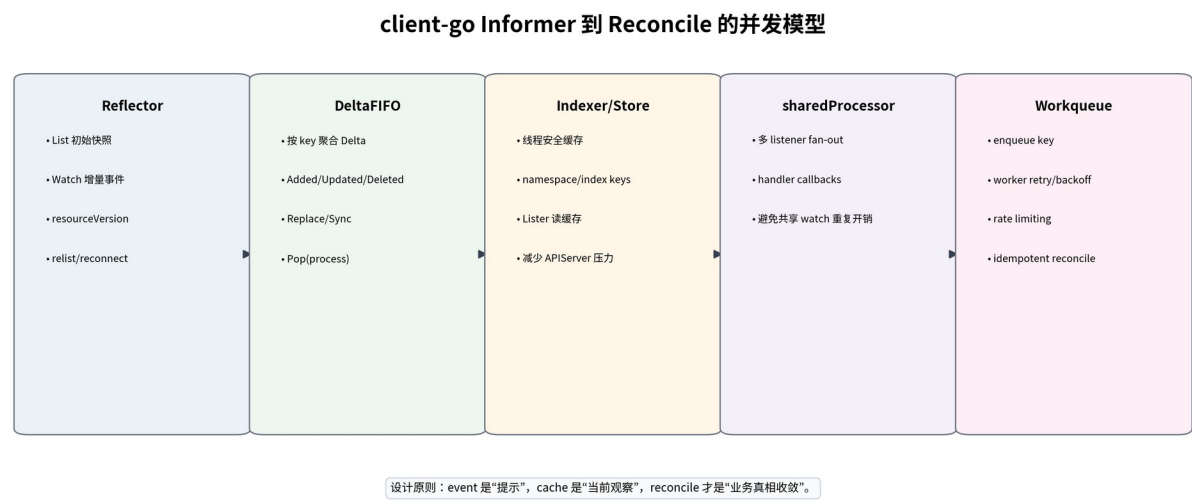


图 25-2: Informer/Controller 的内部协作与并发模型。

高频追问

为什么 handler 不直接做业务？因为 handler 与共享事件分发链路耦合，长耗时/失败会放大影响；enqueue key + rate-limited workqueue 让业务处理可并发、可重试、可限速。

25.3 Scheduler：从 Queue 到 Bind 的失败矩阵

阶段	关键动作	典型失败	后续
Queue	PrioritySort/事件激活	backoff 未到、Unschedulable	等待时间/事件
PreFilter	预计算 Pod state	资源/拓扑信息异常	Unschedulable/Error
Filter	逐 Node 硬约束	Insufficient CPU/GPU、taint、affinity	无可行节点 -> PostFilter
PostFilter	DefaultPreemption 等	找不到可抢占组合	Pending
Score	多个 plugin 加权	偏好而非硬约束	选最高分节点
Reserve/Permit	资源假定/协同批准	gang/插件拒绝	Unreserve + retry
Bind	写 binding / spec.nodeName	API 冲突/超时	binding cycle failure

AI Infra 延伸：GPU 调度常把“Node 可调度”进一步细化为 device selection、topology alignment、gang scheduling、quota/fair-share。传统 Extended Resource 能表达数量，但难表达复杂 device attributes；DRA 正在解决这类需求。

25.4 Kubelet: Node 上的真实状态机

- Kubelet 接收 PodDesiredState 后，不是“一次性启动容器”，而是持续 sync/reconcile；PodWorkers 为每个 Pod 串行化更新，降低并发状态冲突。
- CRI 是 kubelet 与 runtime 的边界；sandbox 先建立 Pod 级网络/namespace 锚点，再创建业务容器。
- PLEG/runtime state 用于发现容器生命周期变化；readiness/liveness/startup probes 决定 service readiness 与 restart 行为。
- Volume Manager 维护 desired/actual state，并协调 CSI attach/mount；Device Manager / DRA plugin 管理设备分配与准备。
- Node 状态由 kubelet 心跳/Lease、NodeCondition、pressure signals 共同体现；控制面 NodeLifecycleController 再进行 eviction/taint 等处理。

25.5 CNI / CSI 故障定位流程

症状	优先看	可能原因	下一步
Pod ContainerCreating 卡住	kubelet events/runtime logs	CNI ADD 失败、IPAM、route	检查 CNI DaemonSet/host netns
跨节点不通	route/MTU/CNI datapath	overlay/underlay、SG/NACL	tcpdump 两端 + node path
PVC Pending	StorageClass/PV/PVC events	binding mode/topology/capacity	scheduler VolumeBinding logs
MountVolume timeout	CSI node/controller logs	attach/mount/secret/device	NodeStage/NodePublish 路径
Volume zone mismatch	PV nodeAffinity	Immediate binding 过早	WaitForFirstConsumer

26. GPU Infrastructure 深入：从 PCIe 拓扑到 Kubernetes 设备生命周期

AI Infra 的关键转折点，是把“GPU=一个可调度资源”升级成“GPU 是一组受拓扑、内存、驱动、fabric、健康状态和集体通信影响的设备”。

GPU 软件与硬件栈：从 Pod 到 HBM/Fabric

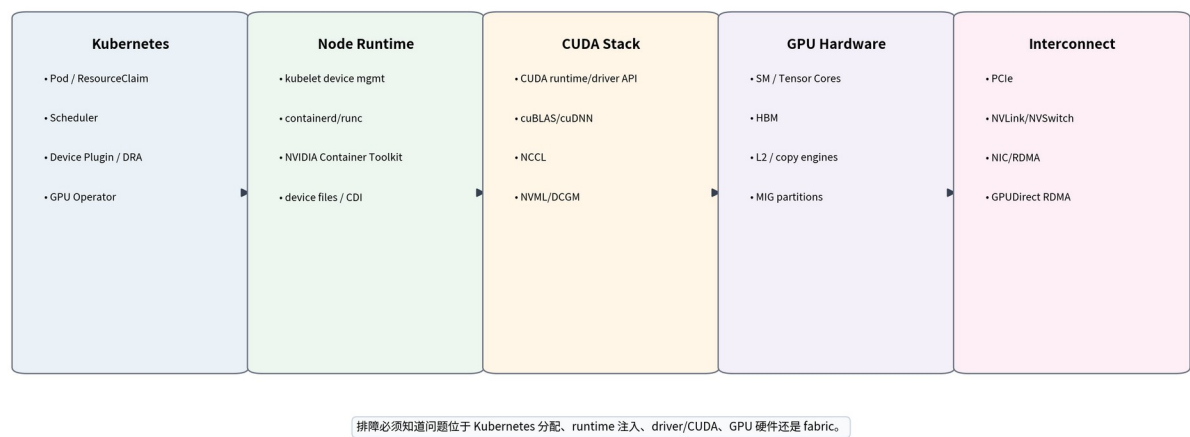


图 26-1：GPU workload 的软件/硬件全栈。

26.1 GPU 资源容量模型

资源	影响	如何观察	典型瓶颈
HBM capacity	决定模型/KV/activation 是否放得下	nvidia-smi, DCGM	OOM, KV capacity
HBM bandwidth	影响 memory-bound kernels	profiler/throughput	decode memory bandwidth
SM compute	影响 GEMM/attention compute	SM active/util	prefill compute saturation
PCIe bandwidth	CPU-GPU / NIC-GPU path	nvidia-smi topo, PCIe counters	model load, host transfer
NVLink/NVSwitch	intra-node GPU collective	topo/NCCL test	tensor parallel all-reduce
NIC/RDMA	inter-node collective	ibstat/ethtool/rdma stats	NCCL hang/low bw

26.2 Device Plugin 与 DRA：面试必须能比较

维度	传统 Device Plugin	DRA
API 表达	Extended Resource 数量，如 nvidia.com/gpu: 1	ResourceClaim/DeviceClass/ResourceSlice，可表达属性与选择
设备选择	主要由 plugin/runtime 内部决定	调度与 driver 协同，可按属性/CEL 选择
动态配置	能力有限，常依赖外部 operator	面向可配置/共享设备设计
拓扑/异构	表达较弱	更适合复杂 accelerator 属性

生态成熟度	长期生产验证	K8s v1.35 DRA stable，但具体 driver 功能成熟度仍需评估

Kubernetes v1.35 中 DRA 已稳定，核心对象包括 DeviceClass、ResourceSlice、ResourceClaim/ResourceClaimTemplate。调度器、kubelet、controller-manager 与 vendor driver 共同完成 advertise -> allocate -> prepare -> consume -> unprepare/cleanup。

26.3 MIG / Time Slicing / MPS / Full GPU 的取舍

方式	隔离	性能可预测性	适合场景	主要风险
Full GPU	强	最高	大模型、稳定低延迟	利用率可能低
MIG	硬件级分区	较高	多租户小模型/固定 profile	profile 碎片、重配置
Time Slicing	时间共享	较低	开发/轻载/容忍抖动	显存与性能隔离弱
MPS	进程并发共享	中	同信任域 CUDA workload	运维复杂、隔离语义不同

26.4 GPU Health / Quarantine 设计

- 采集：DCGM health、Xid、ECC、temperature/power、PCIe replay/errors、NVLink errors、NCCL test。
- 分类：transient workload error vs device degradation vs node/fabric issue。不要把所有 CUDA error 都直接 drain Node。
- 隔离：给 Node/GPU 打 health label/taint，阻止新 workload；必要时迁移/重启/driver reset。
- 验证：运行轻量 GPU diagnostic/NCCL smoke test，成功后再自动解除 quarantine。
- 审计：记录故障次数、组件更换、driver/firmware 版本，避免“反复坏又自动恢复”的 flapping。

27. LLM Inference 深入：容量、调度与性能模型

Inference 面试的关键，是把模型服务理解成“prefill + decode + KV cache + batching + GPU memory/compute + queueing”的资源系统，而不是普通 HTTP 服务。

LLM Inference 请求生命周期

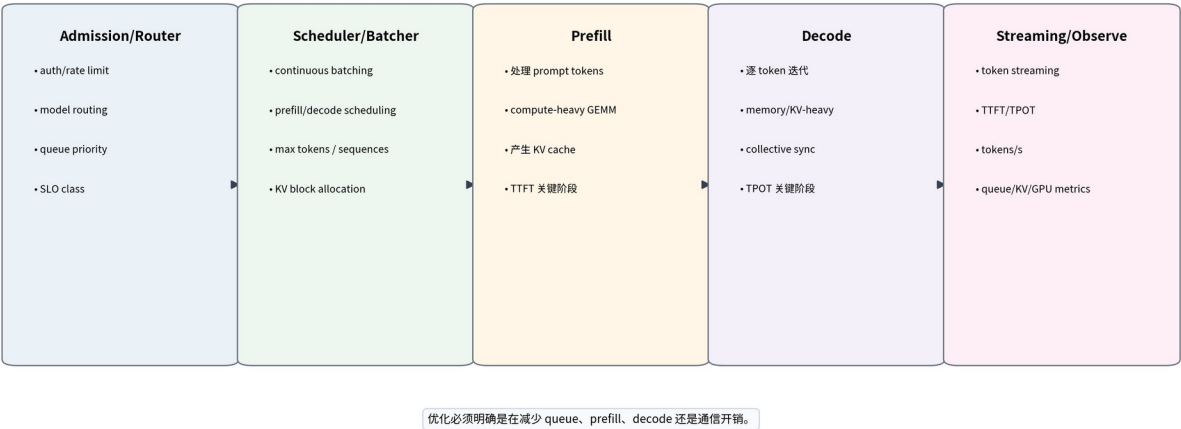


图 27-1：LLM inference 请求的关键阶段。

27.1 模型显存估算

```
Model weights ~= parameter_count * bytes_per_parameter
Example: 70B model
FP16/BF16: 70e9 * 2 bytes ~= 140 GB
INT8:      70e9 * 1 byte  ~= 70 GB
INT4:      70e9 * 0.5 byte ~= 35 GB

GPU memory budget = weights + KV cache + runtime workspace + CUDA/NCCL buffers +
fragmentation/headroom
```

面试时一定强调“模型能放下”不等于“能达到目标吞吐”。KV cache 需要为并发与上下文长度保留显存，而 tensor parallel 还会增加通信和 runtime buffers。

27.2 KV Cache 粗略容量公式

```
KV bytes per token ~= 2 * num_layers * num_kv_heads * head_dim * bytes_per_element
Total KV ~= bytes_per_token * (prompt_tokens + generated_tokens) * concurrent_sequences

注意：GQA/MQA 会显著降低 num_kv_heads；paged KV allocator 会以 block/page 管理，实际还要考虑碎片和预留。
```

变量	增加后影响	系统含义
并发 sequences ↑	KV 线性增长	可能先撞 HBM 而不是 compute
context length ↑	KV 线性增长，prefill 也变重	长上下文需要 admission/配额
output tokens ↑	decode 时间与 KV 增长	占用 slot 更久
TP degree ↑	权重每卡下降，但 collective 增加	容量 vs 通信 trade-off
quantization	权重显存下降	不一定同比降低 KV

27.3 TTFT / TPOT / Throughput 的瓶颈映射

指标异常	优先检查	可能机制
TTFT 高	queue depth、prefill batch、model cold start	排队、长 prompt、加载/编译
TPOT 高	decode batch、KV bandwidth、NCCL	memory bound、collective slow
tokens/s 低	batch utilization、GPU util、communication	批次太小或同步开销
尾延迟抖动	mixed prompt lengths、preemption、network	head-of-line、资源竞争
GPU util 低但延迟高	CPU/router/queue/IO/NCCL stall	GPU 不是瓶颈

27.4 Continuous Batching 为什么有效

- 传统 static batching 等一批请求一起结束，长请求会拖住短请求；continuous batching 允许序列在 decode step 间动态加入/离开。
- 好处是提高 GPU occupancy 和 tokens/s，同时减少空洞；代价是 scheduler、KV allocator、优先级与 fairness 更复杂。
- 需要配合 admission control，否则大量长 context 请求可以耗尽 KV cache，造成 OOM 或极端 queueing。
- 生产上应观测 active sequences、waiting sequences、KV cache utilization、prefill tokens、decode tokens，而不是只看 GPU utilization。

27.5 Autoscaling：三层队列模型

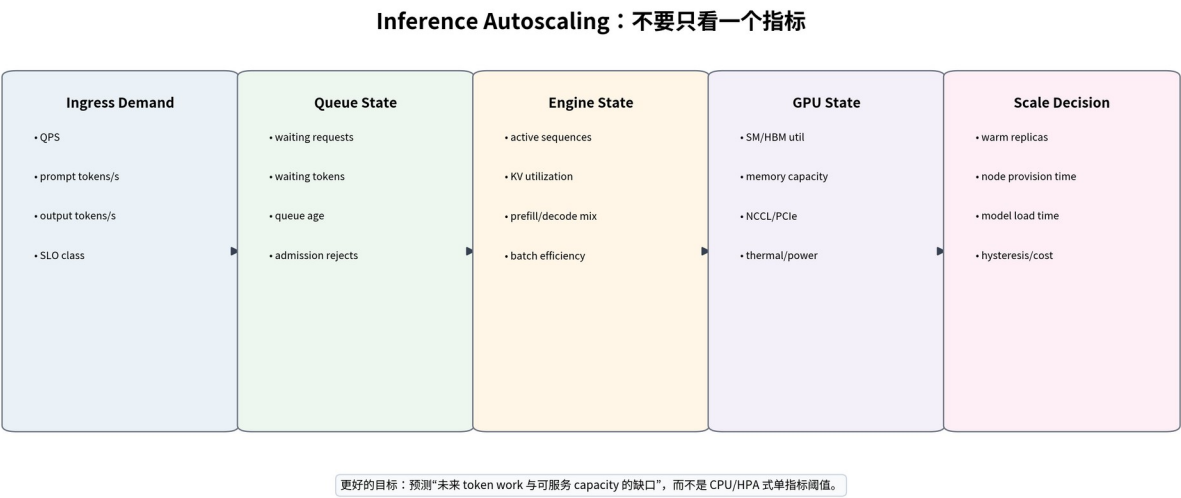


图 27-2：Inference autoscaling 的多层信号。

27.6 Cold Start 分解

阶段	耗时来源	优化
Node provisioning	云实例/GPU capacity	warm pool、reservation、capacity planning
Image pull	大镜像/网络	镜像瘦身、预拉取、local cache
Model fetch	数十/数百 GB weights	本地 NVMe、peer cache、并行下载
Load to host/GPU	disk->RAM->HBM	pinned memory、并行 load、NUMA
Compile/warmup	kernel/JIT/graph capture	预编译、warmup
Readiness	健康检查过严/过早	分阶段 probe、真实 readiness signal

28. Distributed AI: Ray / KubeRay / NCCL / RDMA 深入

分布式 AI 的面试难点在于跨层：Kubernetes 管 Pod/Node；Ray 管 task/actor/placement；NCCL 管 GPU collective；RDMA/IB/RoCE 管 fabric。任何一层错误都可能表现为“任务卡住”。

Distributed AI 多层控制与数据面

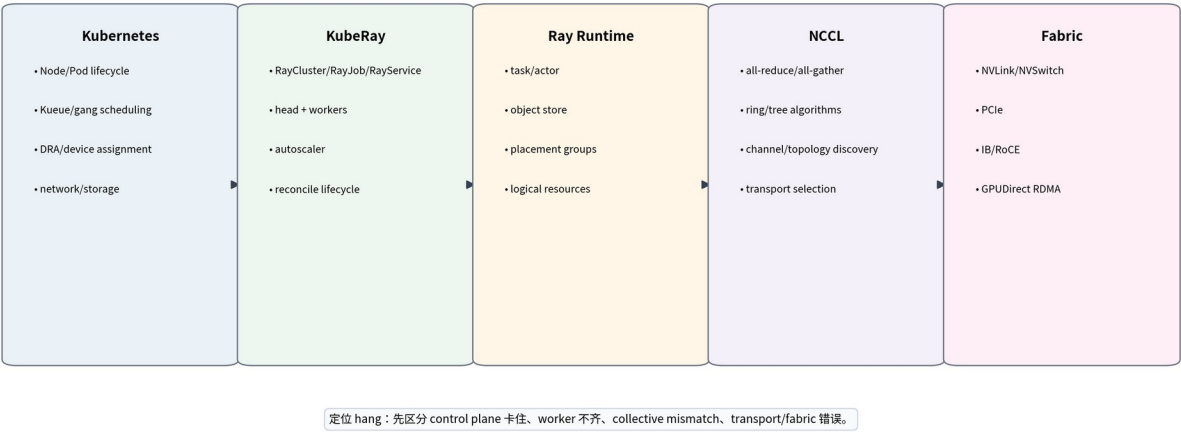


图 28-1：分布式 AI 的五层栈。

28.1 Parallelism 选择

方式	切分对象	主要通信	适合	风险
Tensor Parallel	单层矩阵/张量	频繁 all-reduce/all-gather	单模型跨多 GPU	强依赖低延迟高带宽
Pipeline Parallel	模型 layers/stages	stage 间 activation	模型跨节点/超大模型	pipeline bubble

Data Parallel	请求/副本	参数同步（训练）；推理可独立	横向吞吐	模型需每副本可容纳
Expert Parallel	MoE experts	all-to-all	MoE 模型	流量不均衡/网络压力

28.2 NCCL Hang 排查树

- 29. 确认是否所有 rank 都启动并进入同一个 collective；rank 数量/环境变量/端口配置错误会表现为等待。
- 30. 运行 nccl-tests，先把模型框架变量排除；比较单机 NVLink 与跨机 RDMA bandwidth。
- 31. 检查 NCCL debug logs、interface 选择、HCA/GID、IB/RoCE 配置与 topology。
- 32. 检查 ib_write_bw / ib_write_lat、rdma statistic、ethtool -S、mlxlink 等 fabric 健康。
- 33. 检查 PCIe/NVLink link health、GPU Xid、ECC、driver/firmware 版本。
- 34. 如果偶发 tail stall，关注 congestion/PFC/ECN/CNP、retransmission 与 NUMA/NIC affinity。

28.3 KubeRay 与 Autoscaling

- KubeRay 推荐通过 RayCluster/RayJob/RayService CRD 管理 Ray；RayService 将 serving application 与托管 RayCluster 生命周期连接起来。
- Ray Autoscaler 主要根据 task/actor/placement group 的逻辑资源需求，而不是单纯 CPU/GPU utilization；这与 Kubernetes Cluster Autoscaler/Karpenter 的节点容量层不同。
- 因此扩容链路可能是：Serve queue -> Ray logical demand -> KubeRay 增 worker Pod -> K8s scheduler Pending -> Karpenter/CA 增 Node -> GPU driver/device ready -> worker join。
- 每一层都有自己的 backoff 与 readiness，系统设计中必须避免多个 autoscaler 互相振荡。

29. Agent Runtime / Sandbox 深入：安全模型与虚拟化取舍

AI Agent 的特殊性在于“模型可以生成并执行不可信代码/工具调用”。因此 sandbox 不是单纯 multi-tenant container，而是需要对 kernel attack surface、filesystem、network egress、credentials、resource abuse 和 persistence 建立明确 threat model。

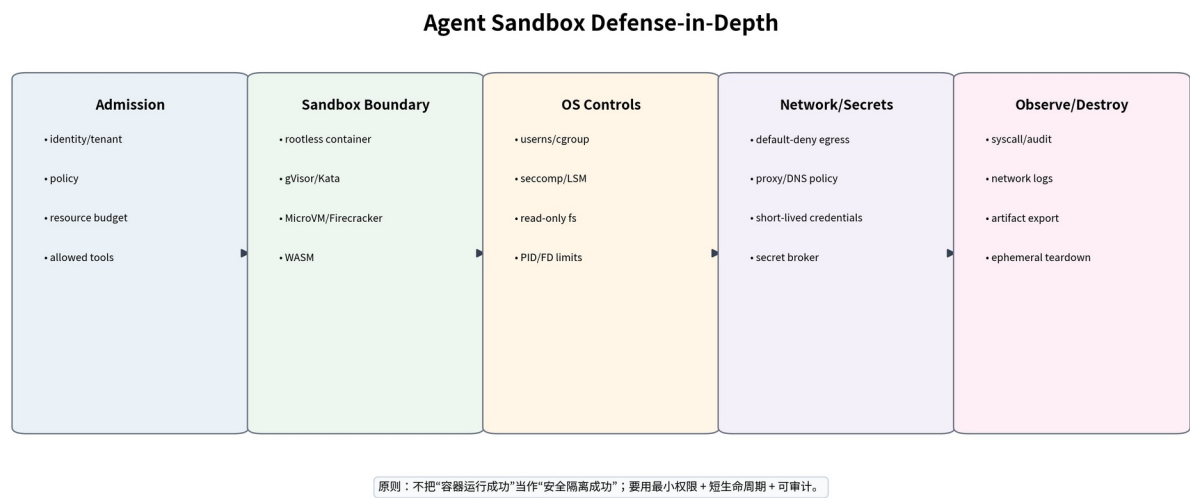


图 29-1：Agent Sandbox 的多层防护。

29.1 Sandbox 技术对比

技术	隔离边界	启动/开销	兼容性	推荐用途
Rootless Container	共享 host kernel + userns	最快/最低	最高	可信或低风险任务
gVisor	用户态 kernel 截获 syscall	低~中	较高	需要比容器更强隔离
Kata Containers	轻量 VM boundary	中	高	K8s 原生强隔离
Firecracker MicroVM	硬件虚拟化 VM	中，通常亚秒级可优化	需要 guest image	高风险代码执行/多租户
WASM	Wasm runtime capability boundary	极快/低	系统调用/生态受限	插件/受限计算

29.2 凭证与网络设计

- 不要把长期云凭证作为环境变量直接注入不可信 sandbox；使用 broker 按任务签发短 TTL、最小 scope token。
- 网络默认 deny；需要访问外部工具时走 egress proxy，由代理根据 tenant/tool/domain/HTTP method 执行策略并记录审计。
- DNS 也属于 egress surface：限制 resolver、阻止访问 metadata service、private control plane 与内部敏感域。
- 文件系统采用 immutable base + ephemeral writable layer；明确允许导出的 artifacts，任务结束销毁 sandbox。
- 资源滥用不仅是 CPU/memory，还包括 PID fork bomb、fd exhaustion、disk fill、network bandwidth、GPU time。

29.3 Sandbox System Design 面试题框架

- Threat model：不可信代码能做什么？是否允许 network/tool/secret？
 - Boundary：container / gVisor / Kata / MicroVM / WASM
 - Provisioning：image, pool, cold start, snapshot
 - Resource isolation：CPU/memory/PID/disk/network/GPU
 - Network policy：default deny + broker/proxy
 - Secrets：short-lived scoped credential
 - Observability：syscall/network/resource/audit
 - Lifecycle：timeout, kill, artifact export, destroy
 - Abuse/escape response：quarantine + forensic + rotate credentials

30. System Design Case Studies：从需求到容量与故障域

30.1 Case A：Multi-tenant LLM Inference Platform

设计维度	关键决策
------	------

SLO	按模型/租户定义 TTFT、TPOT、availability；区分 interactive 与 batch
Routing	model-aware + tenant-aware + locality-aware；支持版本/灰度/容量保护
Admission	token budget、max context、queue age、priority、per-tenant quota
Serving	vLLM/engine pool；continuous batching；GPU topology aware
Autoscaling	queue/token demand + KV state + warm capacity；考虑节点和模型加载延迟
Reliability	跨 fault domain、health quarantine、fallback model/region
Observability	request -> engine -> GPU -> NCCL -> NIC 全链 trace/metrics
Cost	GPU-hour utilization、tokens/\$、idle warm pool、spot/on-demand mix

30.2 Case B: GPU Fleet Scheduler

目标：在异构 GPU fleet 上支持配额、公平性、拓扑、gang、故障隔离和成本优化。面试时应把 Scheduler 分成 admission、queueing、placement、device allocation、node provisioning、runtime readiness 六层。

能力	设计要点
Queue/Fairness	tenant quota、priority、DRF/fair-share、aging
Gang	全部 resources ready 才启动；避免部分占用导致 deadlock
Topology	GPU-NVLink-NIC-NUMA aware；优先满足高带宽通信
DRA/Device	设备属性、MIG profile、健康状态、动态配置
Autoscaler	将 pending gang 转换为节点组需求；capacity type/zone 约束
Fragmentation	bin packing vs topology；MIG profile 碎片；defragmentation
Health	坏卡/坏链路 quarantine，不进入 placement
Preemption	考虑 checkpoint cost、job priority、gang 原子性

30.3 Case C: 1000 并发 Agent Sandbox

问题	设计
Cold start	预热 microVM pool / snapshot / image cache

Isolation	高风险任务 microVM；普通任务 gVisor/rootless 分层
Secrets	broker + short-lived credentials + task identity
Egress	default deny + audited egress proxy
Artifacts	只允许显式 export；object storage 隔离前缀
Resource abuse	cgroup + disk quota + PID/FD + network limit
Scheduling	sandbox class、tenant quota、warm pool locality
Teardown	强制 TTL、kill tree、detach credentials、destroy filesystem

30.4 Case D：跨节点 8xGPU 分布式推理突然慢 40%

35. 先证明是 request 层还是 engine/collective 层：比较 queue time、TTFT、TPOT、tokens/s。
36. 比较单机与跨机 nccl-tests；如果单机正常、跨机慢，聚焦 NIC/RDMA/fabric。
37. 检查 NCCL topology/interface/HCA 选择，确认是否意外走 TCP 或错误 NIC。
38. 检查 ib_write_bw/lat、rdma statistics、PFC/ECN/CNP、switch queue/drop。
39. 检查 GPU-NIC NUMA/PCIe locality、NVLink/PCIe link width、IRQ affinity。
40. 回看变更：driver/NCCL/CUDA/firmware/CNI/kernel/BIOS 版本。
41. 形成 mitigation：quarantine bad node/link、回滚、限制 placement、建立持续 synthetic benchmark。

31. 面试追问库扩展：60 个深度问题与答题要点

31.1 [Linux CPU] 为什么 run queue latency 比 CPU utilization 更能解释 p99？

utilization 只表示忙碌比例；p99 取决于任务等待被调度的时间。需要结合 runnable tasks、CPU affinity、throttling、IRQ、NUMA，并用 PSI/perf sched/eBPF 验证。

31.2 [Linux CPU] CPU request=1、limit=2 在 Node 空闲时最多能用多少？

request 不是上限；limit 决定 ceiling。Node 空闲时理论上可突发到 2 CPU，超过 quota 会 throttling。竞争时 request 对应的 weight 影响相对份额。

31.3 [Memory] memory.high 与 memory.max 的区别？

high 是压力阈值，触发 reclaim/节流；max 是硬上限，无法回收满足分配时可能触发 memcg OOM。

31.4 [Memory] 为什么 Pod RSS 没涨但 Node memory pressure 变高？

考虑 page cache、kernel memory、slab、其他 cgroup、tmpfs、pinned memory；不要只盯单个进程 RSS。

31.5 [IO] fsync 慢为什么会拖慢 etcd?

etcd Raft 提交需要持久化 WAL；storage fsync latency 直接影响 commit latency，进一步影响 API Server 写延迟和 controller throughput。

31.6 [Network] conntrack 满时为什么 Service 新连接失败?

NAT/状态跟踪无法为新连接建立 entry，会 drop 或失败；需要看 nf_conntrack_count/max、连接模式、timeout 与 datapath。

31.7 [API Server] 大量 LIST 为什么危险?

序列化、内存、etcd/cache 读取、network 都可能放大；控制器应 informer/watch 而不是周期性全量 list。

31.8 [Informer] Watch 断开如何保持一致?

使用 resourceVersion 恢复；如果版本过旧/compacted，需要 relist，Replace/Sync 后继续 watch。最终一致依赖幂等 reconcile。

31.9 [Controller] 为什么 workqueue 里通常只放 key?

减少对象复制/过期；worker 处理时从 cache 获取最新状态，天然合并多次 event，支持 retry/rate limit。

31.10 [Scheduler] Filter 和 Score 的本质区别?

Filter 是硬约束决定不可行；Score 是软偏好在可行节点之间排序。

31.11 [Scheduler] MostAllocated 一定更省钱吗?

不一定。它促进 bin packing，但 PDB/topology/daemonset/local storage/requests overprovision 等都可能阻碍 scale-down；还要看 autoscaler 策略和真实利用率。

31.12 [Scheduler] 为什么 GPU gang scheduling 重要?

分布式任务需要同时获得一组 GPU；部分 Pod 启动会占住资源但任务无法前进，导致 deadlock/低利用率。

31.13 [Kubelet] PodWorkers 为什么按 Pod 串行?

同一个 Pod 的生命周期操作存在顺序依赖；串行化避免并发 create/kill/update 导致 runtime state race。

31.14 [CNI] CNI ADD 成功但 Pod 仍无法访问 Service，下一步查什么?

区分 pod L3 通与 Service datapath；查 route/MTU、kube-proxy/eBPF service map、conntrack、policy、DNS。

31.15 [CSI] 为什么 WaitForFirstConsumer 与 topology 有关?

延迟 volume provisioning/binding 到 scheduler 已有 Pod placement 约束时，使 volume 在正确 zone/topology 创建。

31.16 [GPU] GPU utilization 100% 是否代表服务效率高？

不一定。可能被低价值 kernel、通信等待间歇、batch 不合理占满；需要 tokens/s、TTFT/TPOT、HBM、SM active、NCCL 等联合判断。

31.17 [GPU] MIG 为什么比 time slicing 隔离强？

MIG 在硬件上划分 compute/memory 等资源，提供更明确隔离；time slicing 只是时间共享同一 GPU，显存/性能隔离更弱。

31.18 [DRA] DRA 的核心价值是什么？

把设备从简单整数资源升级为可描述属性、claim、动态配置和 driver 协作的 API 模型，适合复杂 accelerator。

31.19 [Inference] 为什么 prefill 与 decode 的优化策略不同？

prefill 更 compute-heavy 且与 prompt 长度相关；decode 逐 token、KV/HBM/通信占比更高。指标上分别显著影响 TTFT 与 TPOT。

31.20 [Inference] 为什么长 context 会伤害整个服务？

prefill work 大、KV cache 占用高、slot 持有久，可能导致 head-of-line、KV exhaustion、其他请求排队。

31.21 [Inference] 为什么 GPU utilization 低但 queue 很长？

可能受 CPU、router、KV capacity、NCCL、memory bandwidth、single-thread bottleneck 或 batch scheduler 限制。

31.22 [Inference] 如何定义 replica capacity？

不能只用 QPS；应按 prompt/output token distribution、SLO、batching、KV capacity、GPU type 建立 tokens/s 与并发容量曲线。

31.23 [Ray] Ray autoscaler 与 Karpenter 有什么区别？

Ray 根据逻辑 task/actor resource demand 扩 worker Pods；Karpenter 根据 Pending Pod/Node requirements 扩基础设施节点，处于不同层。

31.24 [NCCL] NCCL hang 的第一原则是什么？

先判断 rank/collective 一致性与进程是否齐全，再用 nccl-tests 分离框架问题，然后逐层查 transport/topology/fabric。

31.25 [RDMA] RoCE 为什么需要 PFC/ECN 等机制？

RDMA 对丢包/拥塞敏感；RoCE 跑在以太网上，需要 lossless/拥塞控制策略，错误配置会造成性能抖动甚至停滞。

31.26 [Agent] 为什么普通容器不适合所有 Agent code execution？

共享 host kernel，kernel attack surface 仍在；高风险不可信代码可能需要 gVisor/Kata/MicroVM 等更强 boundary。

31.27 [Agent] 为什么 sandbox 应默认无网络？

网络是数据外泄、横向移动、metadata credential theft 的主要面；按需开放经审计的最小 egress 更安全。

31.28 [System Design] 为什么 warm pool 是 AI Infra 的常见设计？

GPU node provisioning + model load 很慢；为低 TTFT/快速扩容保留 warm capacity，用成本换响应速度。

31.29 [Reliability] GPU node quarantine 为什么要自动验证恢复？

单纯重启后 uncordon 容易让坏卡反复进入生产；应通过 diagnostics/synthetic collective 验证再恢复。

31.30 [Cost] AI Infra 成本指标为什么要用 tokens/\$ 而不是 GPU utilization？

utilization 是中间指标；最终目标是满足 SLO 的有效工作量/成本。高 util 但 tokens/s 低仍可能很差。

31.31 [Linux] 什么时候该看 PSI？

当平均 utilization 看不出延迟问题时，PSI 能显示 CPU/memory/IO 资源 stall 对任务的影响；特别适合容器与节点级 pressure。

31.32 [Linux] 为什么 hugepages 可能提升 AI workload？

减少页表/TLB 压力，常用于大内存、DMA/RDMA 场景；但分配和碎片管理更复杂。

31.33 [Linux] NUMA mismatch 如何影响 GPU workload？

CPU thread、host memory、GPU/NIC 若跨 NUMA，会多走 inter-socket link，增加延迟并降低 PCIe/RDMA 吞吐。

31.34 [K8s] Lease 与 NodeStatus 的关系？

Lease 提供轻量高频 heartbeat；NodeStatus 承载更丰富状态，降低 API Server/etcd 更新压力。

31.35 [K8s] PDB 能阻止所有 Pod 中断吗？

不能。PDB 主要约束 voluntary disruption 的 eviction；node crash 等 involuntary disruption 无法阻止。

31.36 [K8s] PriorityClass 为什么可能引发 starvation？

高优先级任务持续进入会抢占/占用资源；需要 quota/fairness/aging，而不是只靠 priority。

31.37 [K8s] TopologySpread 与 podAntiAffinity 如何选择？

TopologySpread 更直接表达跨 zone/node 的分布目标和 maxSkew；AntiAffinity 适合明确“不要共置”约束但计算更重。

31.38 [K8s] etcd compact 与 defrag 有何不同？

compact 移除旧 MVCC history 的可见版本；defrag 回收底层数据库文件空间。

31.39 [K8s] APF 与 client-side rate limit 的区别？

client-side 保护单客户端；APF 是服务器端按流分类、公平排队/限流，保护控制面整体。

31.40 [GPU] Xid error 应如何处理？

先按 Xid 类型和频率分类，关联 ECC/PCIe/driver；高风险设备 quarantine，跑诊断并结合硬件生命周期。

31.41 [GPU] 为什么 GPU Operator 有价值？

自动化 driver/toolkit/device-plugin/DCGM/MIG 等生命周期和一致性，减少 GPU node 手工漂移。

31.42 [GPU] 为什么 GPU scheduling 不能只看 free GPU count？

还要看型号、显存、MIG profile、NVLink/NIC topology、健康、NUMA、作业 gang 和 locality。

31.43 [Inference] 为什么 batch size 不是越大越好？

吞吐会提升到一定点，但 queue/TTFT、KV、HBM、tail latency 会变差；应按 SLO 找 operating point。

31.44 [Inference] 为什么 speculative decoding 可能改善性能？

用 draft 模型预测多个 token，target 验证，可减少大模型 decode step；收益取决于 acceptance rate 与额外算力。

31.45 [Inference] 模型量化的 Infra trade-off？

降低权重显存/带宽，可能提高容量；但 kernel 支持、精度、吞吐、KV dtype 和硬件兼容要验证。

31.46 [Inference] 为什么 model router 要知道 GPU capacity？

同一模型不同 replica 的 KV/queue/warm state 不同；只 round-robin 会造成冷热不均和 SLO 抖动。

31.47 [Ray] Placement Group 解决什么？

把一组资源 bundle 作为调度单元，支持 PACK/SPREAD 等策略，常用于分布式 actor 的 gang/locality。

31.48 [Ray] 对象存储为什么可能成为瓶颈？

大量对象序列化/传输/spill 会消耗 memory/network/storage；要理解 object store memory 与 spill path。

31.49 [NCCL] 为什么 topology discovery 重要？

NCCL 需要选择 GPU/NIC/链路和 ring/tree；错误拓扑或环境变量可能导致走慢路径。

31.50 [RDMA] GPUDirect RDMA 的价值？

NIC 可直接 DMA GPU memory，减少 CPU bounce buffer/复制和延迟，提升跨节点 collective。

31.51 [Agent] 如何限制 fork bomb？

cgroup pids.max、RLIMIT_NPROC、sandbox TTL，并监控 PID growth。

31.52 [Agent] 如何避免 metadata service credential theft？

网络默认 deny，显式阻断 link-local metadata IP，云侧使用 workload identity/短期 token。

31.53 [Agent] 如何审计 Agent 执行?

记录任务 identity、image/hash、命令/工具、network egress、resource metrics、artifact、policy decisions；隐私数据需脱敏。

31.54 [System Design] 如何做 GPU-aware rollout?

小比例新版本/驱动 node pool，synthetic benchmark + canary workload，观测 Xid/NCCL/throughput 后逐步扩。

31.55 [System Design] 如何防 autoscaling oscillation?

分层控制、cooldown/hysteresis、预测启动时间、min warm capacity、避免多个 controller 对同一指标反向调节。

31.56 [System Design] 如何设计跨 AZ inference?

权衡 GPU capacity、network cost/latency、model cache；请求尽量本地，故障时跨域 fallback。

31.57 [System Design] 如何管理模型版本?

immutable artifact、checksum、registry metadata、readiness、traffic split、rollback、兼容 tokenizer/config。

31.58 [System Design] 怎样做 capacity planning?

用真实 prompt/output 分布 benchmark 不同 GPU，建立每 replica token capacity 与 SLO 曲线，再叠加增长/故障 headroom。

31.59 [Observability] AI Infra trace 应覆盖什么?

ingress queue、router、engine queue、prefill/decode、collective、GPU/NIC；最好把 request id 与 batch/replica/device 关联。

31.60 [Observability] DCGM 指标和应用指标如何结合?

GPU 利用率/显存/错误只是资源侧，必须与 TTFT/TPOT/tokens/s/queue 同屏，才能判断是否有效利用。

32. Hands-on Lab：把每个面试点变成可复现证据

Lab	实验步骤	面试产出
CPU throttling	部署 stress container，设置 request=500m/limit=1；观察 cpu.stat、Prometheus throttling、latency。	能解释 request/limit 与 cgroup v2 文件。
Memory OOM	设置 memory.max，通过 allocator 触发 memcg OOM；对比 kubelet eviction。	能区分 kernel OOM vs eviction。
Informer	写一个 client-go informer，打印 List/Watch/handler/workqueue 日志；人为断开 watch。	能解释 relist 与最终一致性。

Scheduler	自定义 scheduler profile: MostAllocated; 构造 Node/Pod requests 比较 placement。	能讲评分与多 plugin 权重。
CNI	抓 Pod->Service->Pod tcpdump; 观察 veth/route/conntrack。	能画 packet path。
CSI	使用 WaitForFirstConsumer StorageClass, 观察 PVC/PV/Pod 调度时序。	能解释 topology binding。
GPU Operator	安装 GPU Operator + DCGM exporter; 采集 GPU metrics。	能解释 operator components。
MIG/DRA	在支持环境配置 MIG 或 DRA ResourceClaim, 观察 device allocation。	能比较 Device Plugin/DRA。
vLLM Benchmark	固定模型/GPU, 改变并发、prompt/output length, 记录 TTFT/TPOT/tokens/s。	建立 capacity curve。
KV pressure	提高 context/concurrency, 观察 KV utilization 与 queue。	能解释显存容量瓶颈。
KubeRay	部署 RayService + autoscaler, 观察 logical demand -> worker Pod -> Node 扩容。	能解释多层 autoscaler。
NCCL	运行 nccl-tests, 比较单机 NVLink 与跨机 RDMA。	能解释 collective/fabric。
Agent Sandbox	实现 default-deny sandbox, 短期 credential broker, 限制 pids/memory/network。	能讲 threat model。
Chaos	kill vLLM pod/GPU node/network fault, 记录恢复时间与 SLO。	把 reliability 变成工程故事。

32.1 建议的 Portfolio Repo 结构

```

ai-infra-lab/
├── terraform/                # GPU cluster / node pools / network
├── kubernetes/
│   ├── gpu-operator/
│   ├── vllm/
│   ├── kuberay/
│   ├── kueue/
│   └── observability/
├── benchmark/
│   ├── loadgen/
│   ├── nccl-tests/
│   └── reports/
├── sandbox/
│   ├── runtime/
│   └── policies/
├── dashboards/
└── docs/
    ├── architecture.md
    ├── capacity-model.md
    ├── incident-runbooks.md
    └── design-decisions.md

```

33. 2026 Research Update：当前官方方向与岗位信号

本扩展版进一步核对了 2026 年公开岗位与官方文档。下面内容适合作为“为什么转 AI Infra”以及“优先学什么”的依据。

来源	2026 信号	对准备的影响
OpenAI Compute Infrastructure	Kubernetes/scheduling、RDMA/NCCL、GPU infra、agent infra、benchmark/observability 同属 Compute Infra 范畴	K8s 深度 + Linux/networking + GPU/collective 是高复用路径
Anthropic Careers	Inference、Inference Deployment、Cloud Inference、Scaling、Compute 相关岗位持续细分	Inference platform 与 fleet/scaling 是独立工程方向
Kubernetes v1.35	DRA stable，面向 accelerator/device claim 与 driver 协同	GPU scheduling 面试应加入 DRA，不只 Device Plugin
NVIDIA GPU Operator 26.x	DRA driver、MIG、DCGM、ComputeDomain 等持续整合	GPU lifecycle/health/topology 需要 operator 化理解
vLLM	官方 Kubernetes 部署与 KServe/KubeRay/LWS/AIBrix 等生态持续扩展	Inference 不应只学单机 engine，还要学 orchestration
Ray/KubeRay	KubeRay 是 Kubernetes 上推荐管理方式，RayCluster/Job/Service + autoscaling	适合 K8s Engineer 迁移到 distributed inference
NCCL Docs	官方 troubleshooting 强调 RDMA bandwidth/latency、rdma stats、mlxlink、RoCE congestion	AI Infra 网络面试需要到 fabric 诊断层

33.1 本轮 Research 参考链接

- https://openai.com/careers/software-engineer-compute-infrastructure-san-francisco/
- https://www.anthropic.com/careers/jobs
- https://kubernetes.io/docs/concepts/resource-management/dynamic-resource-allocation/
- https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/26.7/dra-intro-install.html
- https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/26.3/gpu-operator-mig.html
- https://docs.vllm.ai/en/stable/deployment/k8s/
- https://docs.ray.io/en/latest/cluster/kubernetes/index.html
- https://docs.ray.io/en/latest/serve/autoscaling-guide.html
- https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/troubleshooting/networking_troubleshooting.html

34. 最终准备策略：从 K8s Senior 到 AI Infra Interview-ready

阶段	目标	输出
----	----	----

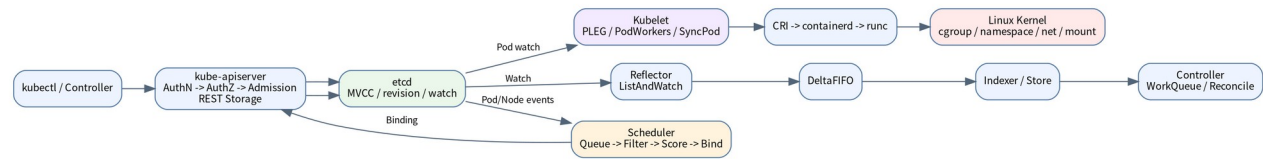
Week 1-2	Linux/Container 排障能跨 kernel/cgroup	4 个实验 + 20 道口述题
Week 3-4	K8s control plane/scheduler/kubelet 源码主链	5 张白板图 + source notes
Week 5-6	GPU stack/device scheduling/health	GPU Operator/DRA/MIG lab
Week 7-8	vLLM inference capacity/performance	benchmark + capacity report
Week 9-10	KubeRay/NCCL/RDMA	distributed demo + NCCL troubleshooting
Week 11	Agent sandbox + security	sandbox prototype + threat model
Week 12	System design/mock/behavior	3 套设计题 + 8 个 STAR stories

最终面试定位

你的标签不应该只是“Kubernetes 工程师学过 AI”，而是“能从 Linux/Kubernetes/Networking 深入到 GPU scheduling、inference performance 和 distributed communication 的 Infrastructure Engineer”。

35. Kubernetes 源码级 Deep Dive II : 从对象变化到真实 Node 状态

这一章把 Kubernetes 面试从“知道组件职责”推进到“能沿调用链定位问题”。面试时不要求背下每一个函数名，但必须能明确：状态从哪里产生、通过什么缓存传播、谁拥有最终写权限、失败后由谁重试，以及数据面何时真正发生变化。



35.1 kube-apiserver : 读写路径与 Watch 路径

写请求的核心是“校验并持久化期望状态”，而不是直接操作 Node。典型路径是 HTTP handler -> authentication -> authorization -> admission -> validation/defaulting -> REST storage -> etcd transaction。读请求还涉及 watch cache、resourceVersion、分页与一致性语义。

- 面试重点 1：为什么大规模 LIST 会伤害 API Server？因为对象反序列化、网络传输、watch cache/etcd 读取、客户端内存都会放大；控制器应尽量依赖 informer 本地缓存。
- 面试重点 2：resourceVersion 不是业务版本号，而是资源存储层的版本/顺序标记；Watch 通过它从一个已知状态继续消费增量。
- 面试重点 3：Watch 断开不是异常设计，而是正常运行路径；客户端必须能 relist/re-watch，并处理过期 resourceVersion。
- 面试重点 4：Admission Webhook 慢会直接进入 API 写路径延迟预算，因此 webhook 的 timeout、failurePolicy、可用性和调用范围都是生产关键点。

35.2 Informer / Controller : 为什么“缓存 + 队列 + 幂等 Reconcile”是黄金组合

Informer 负责把 API Server 的状态高效复制到进程内缓存；Controller 不应该把事件本身当作事实，而应该把事件当作“某个 key 可能需要重新检查”的提示。Reconcile 再从缓存/API 读取当前事实，计算 desired vs observed 的差异。

```
List -> Replace/Sync -> Watch -> DeltaFIFO -> Indexer -> ResourceEventHandler -> WorkQueue(key) -> Worker -> Reconcile(key)
```

机制	解决的问题	面试追问
SharedInformer	多个消费者共享同一份 List/Watch 与缓存	为什么能降低 apiserver 压力？
DeltaFIFO	按 key 合并/组织对象变化并保证处理顺序语义	为什么不是普通 FIFO？
Indexer	本地线程安全对象缓存与二级索引	为什么 handler 里应读 cache？
RateLimitingQueue	失败重试、指数退避、去重	如何避免 hot-loop？
Idempotent Reconcile	重复执行也能收敛到目标状态	为什么分布式控制器必须幂等？

35.3 Scheduler : Scheduling Cycle 与 Binding Cycle 的并发边界

Scheduling Framework 把一次调度分为 scheduling cycle 和 binding cycle。前者选择 Node，通常串行推进以避免复杂竞争；后者执行 Reserve/Permit/PreBind/Bind 等步骤，可以与其他 Pod 的 binding cycle 并发。要能解释 QueueSort、PreFilter、Filter、PostFilter、PreScore、Score、Reserve、Permit、PreBind、Bind 的职责。

```
activeQ.Pop -> scheduleOne -> PreFilter -> Filter -> PostFilter(if needed) -> PreScore -> Score ->
selectHost -> Assume -> Reserve -> Permit -> PreBind -> Bind -> Forget/finish
```

失败路径必须会讲

- Filter 后无可行 Node：进入 PostFilter，例如 DefaultPreemption；仍失败则 Pod 回到 unschedulable/backoff 体系。
- Assume 之后 Bind 失败：scheduler cache 中的 assumed Pod 必须最终被清理，否则会造成“幽灵资源占用”。
- 插件返回 UnschedulableAndUnresolvable 与 Unschedulable 的语义不同，会影响哪些事件能重新激活 Pod。
- Scheduler cache 是乐观缓存；真正事实仍以 API Server 中的绑定结果为准。

35.4 Kubelet : 从 PodSpec 到 Running 不是一个函数调用

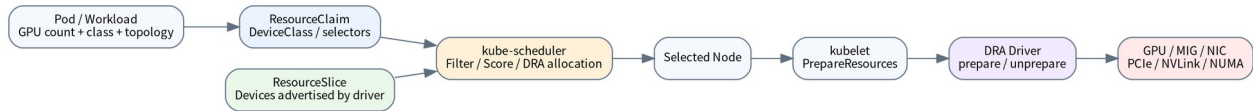
Kubelet 是 Node 上把期望状态变成真实状态的核心状态机。它同时接收 Pod 更新、runtime 状态、probe、volume、PLEG/CRI 事件，并通过 PodWorkers 将同一个 Pod 的同步串行化。

```
config sources -> syncLoop -> dispatchWork -> podWorkers.UpdatePod -> syncPod -> volume
attach/mount -> CRI RunPodSandbox -> CNI -> CreateContainer -> StartContainer -> probes/status
```

- PodSandbox 主要提供共享网络/IPC 等 sandbox 语义；业务 container 在 sandbox 之上创建。
- CNI 失败通常意味着 sandbox 网络未准备好，Pod 可能长时间 ContainerCreating。
- CSI attach/mount 与 container start 有严格依赖关系；volume 未 ready 时不能把容器视为 ready。
- Pod Running 只表示容器运行状态，不等于业务 Ready；readiness probe 决定是否进入 Service endpoints。

36. Scheduler for AI : GPU、Topology、Gang Scheduling 与 DRA

传统 CPU/Memory 调度只需要回答“Node 上还有多少可分配资源”；AI workload 还必须回答“是哪一块 GPU、它与哪块 NIC/NUMA/NVLink 相连、多个 Pod 是否必须同时启动、设备是否健康、资源是否可共享/可重配置”。这使 Scheduler 从 scalar resource placement 走向 topology-aware device placement。



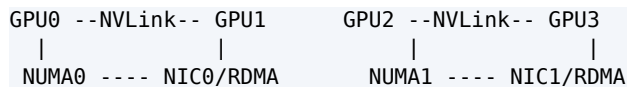
36.1 Device Plugin vs DRA

Extended Resource + Device Plugin 模型简单稳定，但资源请求主要表现为整数计数，例如 `nvidia.com/gpu: 1`；DRA 则把设备能力、选择条件、claim 生命周期和驱动准备过程提升为一等 API。Kubernetes v1.35 起 DRA 已 stable；v1.36 继续扩展 driver、ResourceClaim 与 PodGroup 等能力。

维度	Device Plugin / Extended Resource	DRA
请求模型	整数 scalar resource	ResourceClaim + DeviceClass + selector
设备发现	kubelet device manager	driver 发布 ResourceSlice
调度表达力	主要按数量	可按设备属性/容量/拓扑表达
生命周期	Allocate 后交给 runtime	allocate + prepare + unprepare + cleanup
适合场景	稳定的整卡 GPU	GPU/NIC/FPGA/可重配置设备、复杂共享资源

36.2 Topology-aware GPU placement

GPU 数量相同的两台 Node，性能可能完全不同。必须把 GPU-GPU、GPU-NIC、GPU-CPU/NUMA 拓扑作为容量的一部分。对于 tensor parallel 或多节点 NCCL workload，错误拓扑会让通信穿过慢速 PCIe/QPI/跨 NUMA，甚至失去 GPUDirect RDMA。



- 单机 TP：优先选择 NVLink/NVSwitch 连通性好的 GPU 集合。
- 跨机 TP/EP：GPU 与 RDMA NIC 的 PCIe/NUMA 亲和性会影响 GPUDirect 路径。
- MIG：调度器需要知道 profile，不只是“还有 1 个 GPU”。
- 健康状态：Xid/ECC/链路错误需要进入 quarantine，不能继续作为正常容量。

36.3 Gang Scheduling / Queueing / Quota

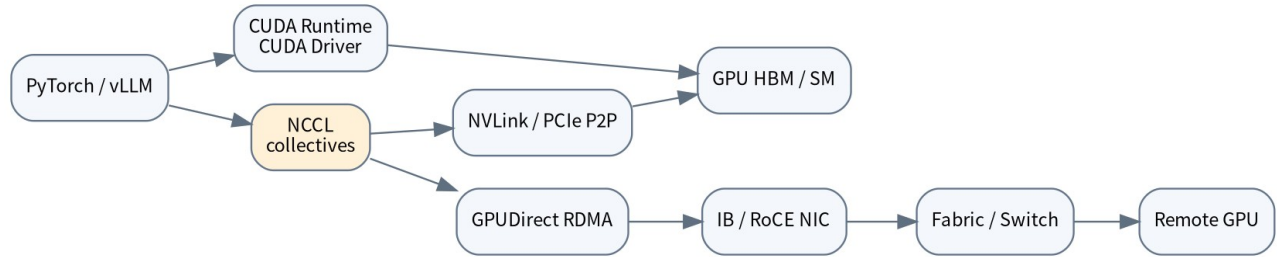
分布式训练或推理常需要 N 个 Pod/GPU 同时可用。若逐个调度，先启动的一部分可能长期占住昂贵 GPU 等待其余成员，形成资源死锁或严重碎片。Gang scheduling 的核心是 all-or-nothing admission；Kueue/Volcano 等系统还会加入队列、配额、优先级和抢占。

面试设计题：100 个团队共享 2000 GPU

- 先做 admission：tenant quota、priority、fair sharing、max running jobs。
- 再做 placement：GPU class、topology、failure domain、gang size。
- 最后做 execution：health check、checkpoint/preemption、node drain、job retry。
- 成本层：优先减少碎片与空置，但不能为了 packing 破坏 topology 或 SLO。

37. GPU / CUDA / NCCL : Infra Engineer 的系统级深度

这一章的目标不是写 CUDA kernel，而是能解释一个 GPU workload 为什么慢、为什么 hang、为什么同样 8 张 GPU 在不同机器上性能差异巨大，以及 Kubernetes/容器层如何影响 CUDA/NCCL。



37.1 GPU 性能模型

瓶颈	典型信号	常见原因	工具
Compute-bound	SM/Tensor Core busy, 高算力利用	大 GEMM/attention, batch 合理	nvidia-smi, Nsight
Memory-bound	HBM 带宽高, compute 未满足	KV 访问、embedding、decode	DCGM, profiler
Communication-bound	NCCL 时间占比高	TP/EP、网络/拓扑差	nccl-tests, NCCL_DEBUG
CPU/input-bound	GPU idle gap	tokenization、Python、数据准备	perf, py-spy, traces
Scheduling/queue-bound	GPU 有空但请求等待	admission/router/replica imbalance	queue metrics, tracing

37.2 NCCL collective 必须理解

AllReduce、AllGather、ReduceScatter、AllToAll 是分布式 AI 的核心通信模式。TP 常大量使用 AllReduce/AllGather；MoE Expert Parallel 常依赖 AllToAll。面试时要能把“模型并行策略”映射到“通信模式”再映射到“网络拓扑与带宽需求”。

- Ring：带宽利用率高、步骤数随 rank 增长，适合大消息。
- Tree：延迟特性更好，拓扑与算法选择会由 NCCL 根据环境决定。
- AllToAll：MoE token dispatch 对网络极敏感，容易暴露 oversubscription 与 tail latency。
- NCCL 会发现 GPU/NIC PCI topology；容器或 VM 中错误的 /sys 拓扑暴露会导致次优路径。

37.3 NCCL Hang 标准排查树

1. 应用层：所有 rank 是否进入同一个 collective？
2. GPU：Xid/ECC/进程是否异常？
3. NCCL：NCCL_DEBUG=INFO，rank/collective 是否一致？
4. 单机：nvidia-smi topo / P2P / NVLink
5. 跨机：NIC selection -> ib_write_bw/lat or rping -> counters
6. GDR：nvidia-peermem / PCIe ACS/IOMMU / GPU-NIC affinity
7. Fabric：IB routing or RoCE PFC/ECN/CNP/drop
8. 再调 NCCL 参数；不要一开始盲目改 env

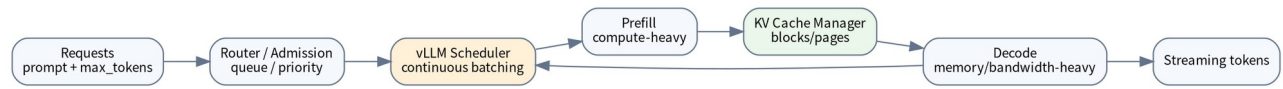
NCCL 2.31 已提供 RAS 与 active diagnostics，官方建议把 diagnostics 作为排查的早期步骤。网络层应区分 InfiniBand 与 RoCE：IB 看 fabric/routing/port health；RoCE 更需要关注 PFC、ECN、CNP、queue drops 与 GID/traffic class。

37.4 GPUDirect RDMA

没有 GDR 时，跨节点 GPU 数据可能走 GPU -> host memory -> NIC -> network；GDR 允许 NIC 直接 DMA GPU memory，减少 CPU involvement 与额外 copy。前提是 GPU/NIC/driver/kernel/module/PCIe topology 都支持正确路径。

38. vLLM Internals : Scheduler、KV Cache、Prefill/Decode 与 Serving 架构

Inference Infra 面试最容易拉开差距的地方，是能否从“部署 vLLM”进一步解释它为什么比朴素 serving 高吞吐、KV Cache 如何决定容量、scheduler 如何形成 batch、长上下文如何制造 head-of-line pressure，以及何时需要 Prefill/Decode disaggregation。



38.1 Prefill 与 Decode 为什么必须分开思考

阶段	主要工作	典型瓶颈	核心指标
Prefill	一次处理 prompt tokens，构建 KV	compute / attention / prompt length	TTFT
Decode	每步生成少量 token，重复读取 KV	HBM bandwidth / KV capacity / scheduling	TPOT/ITL
Serving	多请求共享 GPU	queue + batching + admission	p99 TTFT/TPOT, tokens/s

长 prompt 会放大 prefill 工作量，也会占用更多 KV Cache；若与 decode 混在同一调度器中，可能造成 decode tail latency 抖动。因此 chunked prefill、priority scheduling、prefix caching、disaggregated prefill 都是在不同层面处理“计算与缓存竞争”。

38.2 KV Cache 容量推导

粗略思路：每个 token 在每层需要保存 K 和 V；容量与 layer 数、KV heads、head dimension、dtype bytes 成正比。不同架构有 MHA/GQA/MQA 差异，所以面试时不要死背单一公式，而要先确认模型配置。

```
KV_bytes_per_token ≈ 2(K+V) * num_layers * num_kv_heads * head_dim * bytes_per_element
max_concurrent_tokens ≈ available_KV_memory / KV_bytes_per_token
```

- 模型 weights、CUDA graph/workspace、runtime buffer 与 KV Cache 共同竞争 HBM。
- 提高 gpu_memory_utilization 可能增加 KV 容量，但会降低 OOM 安全余量。
- prefix caching 通过复用相同前缀的 KV 降低重复 prefill 成本，但需要命中率和 cache eviction 策略。
- KV offload/remote cache 会把 HBM 容量问题转换成 PCIe/CPU memory/network latency 问题。

38.3 Continuous Batching

静态 batching 等整批请求结束后再组成下一批，短请求容易被长请求拖住。Continuous batching 在 decode step 边界动态加入/移除 sequence，让 GPU 更持续地有有效工作，同时提升吞吐与利用率。代价是 scheduler 更复杂，且 batch 过大可能恶化单请求延迟。

38.4 Disaggregated Prefill

vLLM 当前提供实验性的 disaggregated prefilling：prefill 和 decode 在不同实例运行，通过 connector 传输 KV。它的主要价值是独立调 TTFT 与 ITL/TPOT，并控制 decode tail latency；官方明确指出它本身不以提升吞吐为目标。生产设计还要计算 KV transfer 的网络成本。

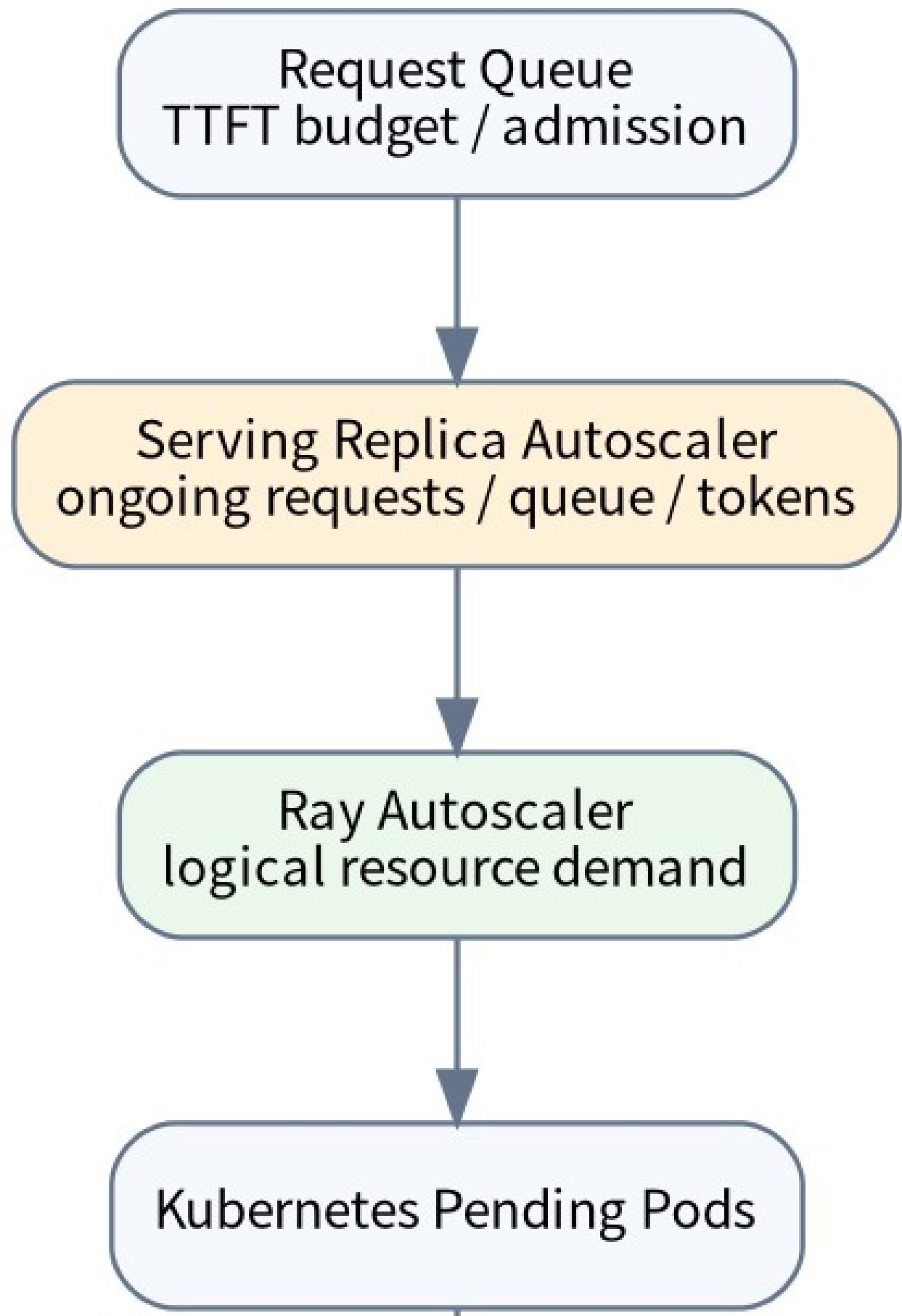
Client -> Global Router -> Prefill Pool -> KV Transfer (NIXL/LMCache/connector) -> Decode Pool -> Stream

什么时候值得考虑？

- SLO 对 TTFT 和 TPOT 都严格，而且 workload prompt/output 分布差异很大。
- Prefill 与 decode 适合不同 GPU/并行策略，希望独立扩缩容。
- 长 prompt 的 prefill 会明显干扰 decode tail latency。
- 已有高带宽 KV transfer 路径，否则拆分可能把问题转移成网络瓶颈。

39. Inference Capacity Engineering : 从单 GPU Benchmark 到全局 Autoscaling

AI Infra system design 不应该从“用 HPA”开始，而应该先定义单 replica capacity envelope，再决定 admission、routing、replica autoscaling、Ray/K8s node autoscaling 和 GPU fleet capacity。



39.1 单 Replica Capacity Envelope

必须用真实 prompt/output 分布压测，而不是只测固定 128/128 token。一个 replica 的容量不是单一 QPS，而是一个多维 envelope：输入长度、输出长度、并发、batch、KV occupancy、TTFT SLO、TPOT SLO、GPU 型号、并行策略共同决定。

指标	什重要	常见误区
tokens/s	更接近 GPU 有效产出	只看 QPS 忽略长短请求差异
TTFT p95/p99	反映排队 + prefill	平均值掩盖 burst
TPOT/ITL p95/p99	反映 decode 服务质量	GPU util 高不代表 TPOT 好
KV occupancy	决定可容纳 active sequences	只看 HBM used 无法解释 cache pressure
queue depth / age	最早暴露容量不足	等 CPU/GPU 100% 才扩容太晚

39.2 Autoscaling 的控制回路

- Fast loop：router/admission 在毫秒~秒级控制并发、拒绝/排队。
- Replica loop：秒~分钟级根据 queue age、ongoing requests、tokens demand 扩 serving replicas。
- Ray node loop：根据 actor/task/placement group 的逻辑资源需求扩 Pod。
- Kubernetes node loop：Pending GPU Pod 驱动 Karpenter/Cluster Autoscaler 创建 Node。
- Cloud capacity loop：GPU quota/instance availability 是最慢且最不确定的一层，因此需要 warm pool 与 capacity reservation。

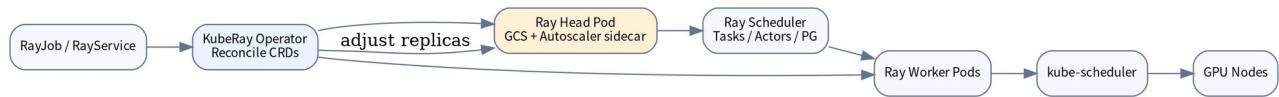
Ray 官方文档明确区分三层 autoscaling：Ray Serve replica、Ray node(Pod)、Kubernetes node。Ray Autoscaler 看的是逻辑 resource request，而不是机器物理 utilization。这和 Kubernetes scheduler 基于 requests 而不是实时 utilization 的思想非常相似。

39.3 防止 Autoscaling Oscillation

- scale-up 触发快，scale-down 保守；使用 stabilization window。
- 指标用 queue age/desired concurrency 等领先指标，而不是仅 GPU utilization。
- 模型冷启动时间必须进入控制器参数，否则扩容动作永远落后于流量。
- 对 burst 使用 warm replicas / warm nodes，而不是无限提高扩容灵敏度。
- 多层 autoscaler 要明确 ownership，避免 Serve、Ray、K8s 三层同时过度补偿。

40. Ray / KubeRay 深入：Actor、Placement Group、Autoscaler 与 Kueue

Ray 的价值在于把分布式任务/actor 的逻辑资源需求变成可调度实体；KubeRay 再把 RayCluster/RayJob/RayService 的生命周期映射到 Kubernetes。面试重点是理解“两套 scheduler”和“三层 autoscaling”如何协作。



40.1 Placement Group

Placement Group 是 Ray 对一组资源 bundle 的原子预留/放置约束，可表达 PACK、SPREAD、STRICT_PACK、STRICT_SPREAD。它与 Kubernetes gang scheduling 类似但处于不同层：Ray 负责 actor/task 的逻辑资源组合，Kubernetes 负责 Pod 到 Node。

40.2 KubeRay Autoscaler 的真实链路

Task/Actor/PG demand -> Ray Head aggregates demand -> Autoscaler sidecar -> update RayCluster replicas -> KubeRay Operator -> create Worker Pod -> kube-scheduler -> maybe create GPU Node -> Ray scheduler places workload

这条链路解释了为什么“Ray 说要扩容”不等于“GPU 马上可用”：中间还有 CR reconcile、Pod scheduling、node provisioning、driver readiness、image/model load。

40.3 Kueue + Ray

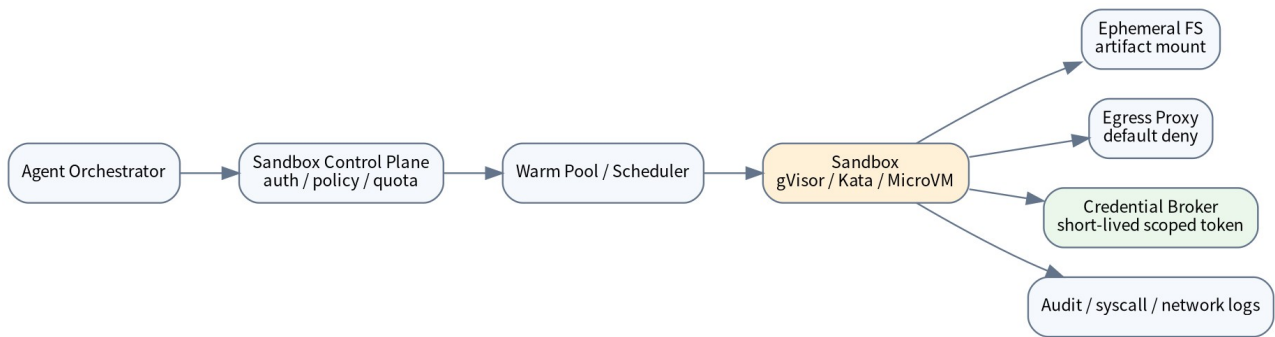
对于批任务/大规模 GPU job，需要把 cluster-level quota/admission 放在 Ray 之前。Kueue 可以对 RayCluster/RayJob 进行 queue/admission，并支持 gang scheduling、priority、quota。RayService 的 managed RayCluster 也可以通过 KubeRay 传播 queue metadata。

常见失败场景

- Ray worker Pod Pending：先看 Kubernetes scheduler reason，不要只看 Ray status。
- Ray actor pending：可能是 Ray logical resource 不满足，即使 K8s Node 有空闲。
- Karpenter 已扩 Node 但 Ray 仍慢：检查 GPU plugin/driver readiness、worker registration、runtime env/model loading。
- scale-down 后频繁 scale-up：检查 idle timeout、Serve replica policy、placement group 与 node consolidation 是否冲突。

41. Agent Runtime Platform : Sandbox、MicroVM、凭证、网络与多租户安全

Agent Infra 的核心风险是“让不可信或半可信代码拥有真实执行能力”。因此 sandbox 不是简单的 Pod，而是一套 execution control plane：身份、镜像/模板、文件系统、网络、凭证、资源限制、审计、生命周期与逃逸防护共同组成安全边界。



41.1 Threat Model

威胁	攻击路径	控制
Container escape	kernel syscall / runtime bug	gVisor/Kata/MicroVM, seccomp, patching
Credential theft	env/file/metadata service	broker + short-lived token + metadata deny
Data exfiltration	arbitrary outbound network	default deny + egress proxy + domain policy
Fork bomb / resource abuse	process explosion / memory bomb	pids.max, cpu.max, memory.max, IO quota
Persistence	write host/path/shared cache	ephemeral rootfs, immutable image, scoped mounts
Lateral movement	service account / cluster network	per-sandbox identity, network segmentation

41.2 Container、gVisor、Kata、MicroVM 怎么选

选择不是“越重越安全”这么简单，而是 isolation、cold start、kernel compatibility、density、observability、运维复杂度之间的权衡。对于任意代码执行，普通共享宿主内核容器通常不是最强边界；MicroVM/Kata 提供更强 VM 边界，gVisor 通过用户态 kernel 拦截系统调用，WASM 适合能力受限的插件型 workload。

41.3 Credential Broker

Sandbox -> request capability X -> Broker verifies sandbox identity + task policy -> mint short-lived scoped token -> audit -> token expires

- 不要把长期 cloud key 注入 env。
- token scope 应与 task/action 绑定，而不是与整个 Agent 身份永久绑定。

- 所有凭证签发与使用应可审计，并能按 sandbox/task 关联。
- 访问 cloud metadata endpoint 默认拒绝，避免 SSRF/metadata credential theft。

41.4 Warm Pool 与安全

Warm pool 能显著降低 sandbox cold start，但复用带来残留数据风险。安全设计必须保证每次分配前后彻底 reset：文件系统、进程、网络 namespace、凭证、tmp/cache、kernel state。若无法证明 reset 完整，宁可销毁实例。

42. AI Infra Observability : 从 GPU 指标到一次请求的 End-to-End Trace

AI Infra 的 observability 必须跨越 HTTP queue、model router、vLLM scheduler、GPU、NCCL、RDMA、Kubernetes Node。只看 DCGM 或只看应用 latency 都无法解释系统瓶颈。



42.1 Golden Signals for Inference

层	核心指标	问题示例
Request	QPS, queue age, p50/p95/p99	是否容量不足/热点模型?
Token	input/output tokens/s, TTFT, TPOT	prefill 还是 decode 慢?
Scheduler	running/waiting seq, batch size	是否 head-of-line/blocking?
KV	usage, eviction, prefix hit	是否 cache pressure?
GPU	SM, HBM, power, temp, ECC/Xid	算力/显存/健康问题?
NCCL	collective duration, timeout	通信是否成为瓶颈?
RDMA	BW, latency, retrans/drop/PFC	fabric 是否拥塞/退化?
K8s/Linux	Pending, PSI, CPU throttling, IO	基础设施是否拖慢?

42.2 一次慢请求怎么 Trace

```
request_id -> gateway queue -> router decision -> replica queue -> prefill span -> KV allocation ->
decode steps -> NCCL spans -> token stream
```

理想情况下 trace 里应该能关联 model/version、GPU node、GPU UUID/MIG profile、replica、prompt/output tokens、batch、KV occupancy、NCCL rank/fabric。这样 p99 变差时才能回答“是 workload 变长、调度排队、GPU 退化还是网络 collective 变慢”。

42.3 SLO 分解

- TTFT budget = ingress + admission + queue + model routing + prefill + first decode + network flush。
- TPOT budget = scheduler wait + decode compute + collective + stream overhead。
- Availability 不只是 HTTP 200；若 token latency 严重超标，对交互式产品等价于不可用。
- 对多模型平台，SLO 应按 model/tier/tenant 分层，避免大模型拖累所有 workload。

43. Staff-level System Design : 6 个完整 Case 的答题骨架

Staff/Senior AI Infra 面试看重的不只是画组件，而是需求澄清、容量模型、故障域、控制回路、成本、可观测性与演进路线。下面的 case 建议每个都能在 35-45 分钟内完整白板。

43.1 Case 1 : Multi-region LLM Inference Platform

- 需求：多模型、多租户、交互式 SLO、区域容灾、GPU 稀缺。
- 入口：global traffic manager 根据 region health/capacity/model locality 路由。
- 区域内：gateway -> admission -> model router -> serving pools。
- 容量：每模型维护 replica capacity envelope；保留 N+1 或 warm pool。
- 容灾：region loss 时 admission 必须保护剩余区域，不能无限 spillover 导致级联故障。
- 数据：模型 artifact 多区域预热；避免故障时所有 Node 同时从远端拉 100GB 权重。
- 指标：region queue age、TTFT/TPOT、GPU capacity headroom、model warm ratio、failover traffic。

43.2 Case 2 : GPU Fleet Scheduler

- 输入：tenant、job priority、GPU class/count、topology、duration estimate、checkpointability。
- Admission：quota/fair share/gang；Placement：topology/binpack/failure domain；Execution：health/drain/preempt。
- 碎片：不仅是 GPU count，还包括 MIG profile、NVLink island、NIC affinity、node shape。
- 抢占：只对可 checkpoint 或低优先级 workload；计算 reclaim benefit 与 restart cost。
- 健康：Xid/ECC/NVLink/NIC degradation -> quarantine -> diagnostics -> controlled rejoin。

43.3 Case 3 : Disaggregated Prefill/Decode Platform

- Prefill pool 优化 TTFT/compute；Decode pool 优化 TPOT/HBM bandwidth。
- Router 根据 prompt length、KV transfer capacity、pool queue 做决策。
- KV transport 必须进入容量模型：bytes/request $\times$ QPS，不能假设免费。
- 独立 autoscaling：prefill demand 与 decode active sequences 使用不同指标。
- 失败语义：KV transfer 失败时 retry prefill、fallback colocated 或 fail fast，需明确。

43.4 Case 4 : 1000 并发 Agent Sandbox

- 目标：<2s startup、强隔离、默认无网、短期凭证、完整审计。
- Control plane：API/policy/quota/scheduler；data plane：sandbox nodes + warm pool。
- 隔离：MicroVM/Kata/gVisor 根据风险 tier；高风险任务不与低风险边界混用。
- 资源：pids/cpu/memory/io/time limit；artifact 通过受控 mount/object store。
- 网络：egress proxy + allowlist + DNS policy；metadata endpoint deny。
- 销毁：每任务强制 teardown，warm reuse 必须证明清理完整。

43.5 Case 5 : NCCL 性能突然下降 40%

- 先建立 baseline：单机 vs 跨机、某些 rank vs 全部、某个 rail vs 全 fabric。
- 检查 workload 是否变了：message size/parallelism/model version。
- GPU topology/P2P/NVLink；GPU-NIC affinity/GDR；NIC bandwidth/latency；fabric counters。

- RoCE 看 PFC/ECN/CNP/drop, IB 看 link/routing/errors。
- 用 nccl-tests 与 ib_write_bw/lat 把应用问题与基础设施问题分离。

43.6 Case 6 : Inference 成本降低 30% 但 SLO 不退化

- 先定义 cost/token、cost/request、GPU-hour utilization 与 SLO, 而不是只追 GPU util。
- rightsizing: 模型量化、GPU class、TP size、batch/concurrency。
- 提高 cache hit、continuous batching、model packing, 但验证 tail latency。
- scale-down 空闲 replicas/nodes; 使用 warm pool 而不是永久大 buffer。
- 分层 SLO: premium 保 headroom, batch/offline 使用 spot/低优先级。
- 用实验数据证明每项节省, 不把成本转移为更高 retry/latency。

44. 120 个追加追问：从 Senior 到 Staff 的追问深度

44.1 [Linux] CFS/EEVDF 下 latency-sensitive workload 为什么可能受 runnable task 数影响？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.2 [Linux] CPU steal time 高时为什么应用优化通常无效？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.3 [Linux] PSI some 与 full 应如何解释？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.4 [Linux] NUMA remote memory 为什么会影响 GPU feeding？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.5 [Linux] THP 对数据库/AI workload 可能有什么利弊？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.6 [Linux] page cache 与 cgroup memory accounting 如何交互？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.7 [Linux] direct IO 与 buffered IO 如何选择？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.8 [Linux] NVMe queue depth 太高为什么可能恶化 p99？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.9 [Linux] softirq 高如何定位到网络 PPS 问题？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.10 [Linux] ephemeral port exhaustion 与 conntrack exhaustion 如何区分？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.11 [K8s] Watch cache 与 etcd watch 的关系是什么？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.12 [K8s] resourceVersion=0 的语义需要注意什么？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.13 [K8s] Admission webhook failurePolicy 如何取舍？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.14 [K8s] APF 如何保护 apiserver 免受 noisy client？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.15 [K8s] etcd leader 频繁变化会产生什么症状？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.16 [K8s] 为什么 controller reconcile 必须基于当前状态而非 event payload？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.17 [K8s] RateLimitingQueue 如何避免失败对象热循环？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.18 [K8s] scheduler Assume 为什么能提高吞吐？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.19 [K8s] assumed Pod 泄漏会出现什么现象？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.20 [K8s] Unschedulable 与 Unresolvable 如何影响 requeue？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.21 [K8s] Permit plugin 能实现什么类型的协同调度？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.22 [K8s] Reserve/Unreserve 为什么必须成对考虑？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.23 [K8s] TopologySpread 的 maxSkew 在节点故障时如何影响 placement？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.24 [K8s] PDB 与 scheduler preemption 的关系？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.25 [K8s] kubelet PLEG 不健康会出现什么症状？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.26 [K8s] Pod Ready 与 ContainersReady 有什么区别？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.27 [K8s] CNI 成功但 DNS 不通怎么分层排查？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.28 [K8s] CSI attach 成功但 mount 失败查哪些层？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.29 [K8s] DRA ResourceSlice 的作用是什么？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.30 [K8s] DRA prepare/unprepare 为什么属于 kubelet/driver 生命周期？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.31 [GPU] GPU SM utilization 与 tensor core utilization 有何差别？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.32 [GPU] HBM bandwidth 和但 SM 不高意味着什么？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.33 [GPU] GPU power cap 如何影响性能？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.34 [GPU] ECC corrected error 是否一定需要 quarantine？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.35 [GPU] Xid error 如何做自动分类与恢复策略？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.36 [GPU] NVLink degraded 如何从指标和 topo 发现？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.37 [GPU] PCIe ACS/IOMMU 为什么可能破坏 P2P？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.38 [GPU] nvidia-peermem 在 GDR 路径中的作用？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.39 [GPU] MIG profile 碎片如何做 defragmentation？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.40 [GPU] Time slicing 为什么不提供显存隔离？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.41 [GPU] MPS 与 time slicing 的适用场景有什么不同？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.42 [GPU] GPU Operator 升如何避免全 fleet 同时不可用？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.43 [GPU] Driver/CUDA compatibility 面试怎么解释？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.44 [GPU] GPU reset 失败后为什么可能必须 reboot node ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.45 [GPU] 为什么 GPU free count 不能代表有效 capacity ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.46 [NCCL/RDMA] AllReduce 为什么对 TP 性能关键 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.47 [NCCL/RDMA] AllToAll 为什么对 MoE 更敏感 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.48 [NCCL/RDMA] NCCL_SOCKET_IFNAME 配错会有什么症状 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.49 [NCCL/RDMA] RoCE GID 选择错误可能发生什么 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.50 [NCCL/RDMA] PFC storm 为什么危险 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.51 [NCCL/RDMA] ECN/CNP 在 RoCE 拥塞控制中的角色 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.52 [NCCL/RDMA] ib_write_bw 正常但 NCCL 慢还查什么 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.53 [NCCL/RDMA] 正常但 tail latency 高应该测什么 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.54 [NCCL/RDMA] 多 rail 网络如何发现某一 rail 退化 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.55 [NCCL/RDMA] 容器中的 /sys 拓扑错误为什么影响 NCCL ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.56 [Inference] 为什么 TTFT 与 prompt length 非线性相关 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.57 [Inference] 为什么 TPOT 对 batch/concurrency 敏感 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.58 [Inference] GQA 为什么能降低 KV Cache ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.59 [Inference] prefix caching 命中率低时为什么可能得不偿失 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.60 [Inference] KV block fragmentation 如何影响 capacity ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.61 [Inference] chunked prefill 如何改善 decode tail latency ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.62 [Inference] speculative decoding 的 acceptance rate 为什么重要 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.63 [Inference] 量化为什么可能降低显存但不一定提升 latency ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.64 [Inference] 长输出请求如何影响其他请求公平性 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.65 [Inference] admission control 应该按 request 还是 token budget ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.66 [Inference] 如何估算一个 replica 的 safe concurrency ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.67 [Inference] 为什么 GPU 100% 但 tokens/s 仍可能低 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.68 [Inference] 为什么 queue age 比 queue depth 更适合作为 SLO signal ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.69 [Inference] 模型 router 为什么需要知道 KV/cache pressure ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.70 [Inference] multi-model packing 最大风险是什么 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.71 [Inference] cold start 如何拆成 node/image/model/compile/cache 阶段 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.72 [Inference] CUDA graph 对 serving 有什么意义 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.73 [Inference] prefill/decode disaggregation 为什么不保证吞吐提升 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.74 [Inference] KV transfer 网络如何进入 capacity planning ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.75 [Inference] 什么时候应该使用不同 GPU 分别跑 prefill/decode ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.76 [Ray] Ray logical resource 与 Kubernetes request 有何不同 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.77 [Ray] Ray actor pending 但 K8s 没有 Pending Pod 为什么？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.78 [Ray] Placement Group STRICT_PACK 什么时候会造成调度困难？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.79 [Ray] Ray object store pressure 会有什么症状？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.80 [Ray] Ray Serve replica autoscaler 与 Ray node autoscaler 如何配合？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.81 [Ray] KubeRay operator 故障时现有 Ray workload 会怎样？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.82 [Ray] RayService upgrade 如何避免流量中断？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.83 [Ray] Kueue admission 在 Ray 前面解决什么问题？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.84 [Ray] 为什么三层 autoscaling 容易振荡？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.85 [Ray] Ray worker startup latency 应拆成哪些阶段？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.86 [Agent] 为什么 sandbox identity 应该是 per execution？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.87 [Agent] 如何限制 Agent 访问企业内部网络？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.88 [Agent] DNS allowlist 为什么不等于安全 egress policy ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.89 [Agent] 如何处理用户上传 artifact 中的恶意文件？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.90 [Agent] 如何防止 sandbox 写满磁盘？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.91 [Agent] 如何设计 wall-clock timeout 与 CPU time limit ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.92 [Agent] MicroVM snapshot warm start 有什么安全风险？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.93 [Agent] sandbox reuse 如何证明没有跨租户残留？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.94 [Agent] 如何审计 shell command、network 与 credential use ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.95 [Agent] 如何给 Agent browser/tool 提供最小权限？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.96 [System Design] 如何定义 GPU fleet 的 headroom ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.97 [System Design] GPU quota 应按 count、GPU-hour 还是 normalized compute ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.98 [System Design] 跨 AZ GPU traffic 为什么昂贵且慢？

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.99 [System Design] 如何设计 model artifact distribution 避免 thundering herd ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.100 [System Design] 如何做 GPU-aware canary rollout ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.101 [System Design] 如何把 Xid error 自动关联到用户请求 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.102 [System Design] 如何设计 capacity reservation 与 spot 的组合 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.103 [System Design] 如何处理云厂商 GPU capacity shortage ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.104 [System Design] 如何避免一个长 prompt tenant 拖垮整个集群 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.105 [System Design] 如何做 tenant-level cost attribution ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.106 [System Design] 如何设计 inference overload protection ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.107 [System Design] 如何定义 degraded mode ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.108 [System Design] 如何做 region failover 又不引发级联过载 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.109 [System Design] 如何做模型版本回滚并处理 KV/cache 不兼容 ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

44.110 [System Design] 如何证明成本优化没有伤害 SLO ?

答题结构：先定义概念和边界，再给出可观测信号；随后沿“应用 -> runtime -> Kubernetes -> Linux/硬件”分层定位；最后给出验证实验、缓解措施与长期修复。不要只报命令或参数。

45. 30 个新增 Hands-on Labs : 面试前真正做一遍

#	Lab	验收标准
1	Linux CPU	制造 CPU contention, 对比 top、vmstat、pidstat、PSI 与 run queue。
2	CPU throttling	在 cgroup v2 设置 cpu.max, 观察 throttled_usec 与应用 p99。
3	MemoryQoS	设置 memory.high/memory.max, 制造 reclaim 与 cgroup OOM。
4	IO	fio 改变 queue depth, 观察 iostat await/util 与 tail latency。
5	Network	制造 conntrack/ephemeral port 压力并区分症状。
6	APIServer	大量 LIST vs informer cache, 比较请求量与延迟。
7	Informer	主动断 Watch, 观察 relist/re-watch 与 resourceVersion。
8	Controller	实现 rate-limited workqueue + idempotent reconcile。
9	Scheduler	自定义 NodeResourcesFit MostAllocated, 观察 placement。
10	Scheduler	构造 unschedulable Pod, 观察 queue/backoff/event reactivation。
11	Preemption	PriorityClass + PDB 场景验证 victim selection。
12	Kubelet	跟踪 Pod 从 Pending 到 Running 的 events/CRI/CNI 日志。
13	CNI	破坏路由/DNS/NetworkPolicy, 按 packet path 排查。
14	CSI	WaitForFirstConsumer + zone topology 实验。
15	DRA	在支持环境部署示例 DRA driver, 观察 ResourceSlice/Claim 生命周期。
16	GPU	采集 DCGM SM/HBM/power/ECC 指标并做 dashboard。
17	MIG	切换 MIG profile, 观察 Kubernetes 暴露资源变化。
18	GPU topology	nvidia-smi topo 与 NUMA/NIC 亲和性记录。
19	NCCL	nccl-tests 单机 vs 跨机建立 bandwidth baseline。
20	RDMA	ib_write_bw/ib_write_lat 建立网络

		baseline。
21	vLLM	不同 input/output 长度压测 TTFT/TPOT/tokens/s。
22	KV Cache	改变 max model len/concurrency, 观察 KV occupancy 与 OOM。
23	Batching	改变 max-num-seqs/batch 参数, 画 throughput-latency 曲线。
24	Prefix cache	构造高/低 prefix hit workload 比较效果。
25	Disagg prefill	两实例 prefill/decode + connector, 测 TTFT/ITL 与 KV transfer。
26	KubeRay	RayService + Serve autoscaling + Ray node autoscaling。
27	Kueue	对 RayJob 做 quota/gang admission。
28	Agent sandbox	用 gVisor/Kata/MicroVM 实现 default-deny network sandbox。
29	Credential broker	实现 5 分钟 scoped token 并记录审计日志。
30	System design evidence	为项目生成 capacity report: SLO、 tokens/\$、failure injection、runbook。

46. 2026 Research Addendum 与版本敏感点

以下内容属于版本敏感知识，面试前应再次核对目标公司的 Kubernetes、NVIDIA driver/CUDA、NCCL、Ray 与 vLLM 版本。不要把某一版本的实验性特性描述成永久稳定接口。

- Kubernetes DRA: v1.35 起 stable; v1.36 继续扩展 driver 与资源表达能力。面试应重点理解 ResourceClaim、DeviceClass、ResourceSlice, 以及 scheduler/kubelet/driver 的职责分工。
- vLLM Disaggregated Prefill: 当前官方文档仍标注 experimental; 主要价值是分别调优 TTFT 与 ITL/TPOT、控制 tail ITL, 并非自动提升 throughput。
- NCCL 2.31: 官方新增/强化 RAS 与 active diagnostics, 并建议优先通过 diagnostics、GPU、network、runtime 层逐层缩小问题范围。
- KubeRay: Ray Autoscaler 根据 task/actor/placement group 的逻辑资源需求扩 Ray worker; Ray Serve、Ray node、Kubernetes node 构成三层 autoscaling。
- AI Infra 的趋势不是 “Kubernetes 被 AI 替代”, 而是 Kubernetes/queueing/device allocation 与 GPU topology/model serving/fabric 进一步融合。

46.1 本轮新增官方资料

<https://kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/>

<https://kubernetes.io/docs/concepts/resource-management/dynamic-resource-allocation/>

<https://kubernetes.io/blog/2026/05/07/kubernetes-v1-36-dra-136-updates/>

<https://kubernetes.io/docs/tutorials/cluster-management/install-use-dra/>

https://docs.vllm.ai/en/latest/features/disagg_prefill/

https://docs.vllm.ai/en/latest/serving/expert_parallel_deployment/

<https://docs.ray.io/en/latest/cluster/kubernetes/user-guides/configuring-autoscaling.html>

<https://docs.ray.io/en/latest/serve/autoscaling-guide.html>

https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/troubleshooting/networking_troubleshooting.html

https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/troubleshooting/gpu_troubleshooting.html

<https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/troubleshooting/diagnostics.html>

47. 最终 Interview Readiness Matrix

领域	Senior 要求	Staff/AI Infra 加分项	自测
Linux	CPU/Memory/IO/Network 故障 分层	NUMA/PCIe/PSI/eBPF/GPU feeding	☐
Kubernetes	APIServer/Informer/ Scheduler/Kubelet/CNI/CSI	源码调用链、queueing、DRA	☐
Scheduler	Filter/Score/Preemption	GPU topology/gang/quota/fairness	☐
GPU	CUDA stack、MIG、DCGM	PCIe/NVLink/GDR/health automation	☐
Inference	TTFT/TPOT/KV/batching	disagg prefill、capacity envelope、cost/token	☐
Distributed	Ray/KubeRay/NCCL	RDMA/RoCE/IB、Kueue、多层 autoscaling	☐
Agent	sandbox 基础	MicroVM/credential broker/egress/audit	☐
System Design	组件与 HA	容量公式、故障域、SLO、成 本、演进	☐
Evidence	能描述项目	有 benchmark、failure injection、runbook、 dashboard	☐

建议完成标准：每个领域至少能回答“是什么 -> 为什么 -> 内部如何工作 -> 怎么观测 -> 怎么故障注入 -> 怎么修 -> 有什么 trade-off”，并能在白板上画出核心数据流。做到这一点，准备目标就从“会使用 Kubernetes”提升到了“能设计和运营 AI Infrastructure”。

48. v3 扩展说明：从“知识地图”进入“可口述的 Staff 面试答案”

本轮目标

这一版不再单纯增加名词，而是把高频主题写成可直接用于 Senior/Staff 面试的回答框架：定义 -> 内部机制 -> 源码路径 -> 故障场景 -> 观测指标 -> Trade-off -> 追问。所有源码函数名都应按目标版本核对，文中重点描述长期稳定的调用关系。



v3.0 终极版
共 128 页

ty

The flowchart illustrates the Kubernetes pod creation process, organized into five main components: 用户提交 (User Submission), API Server, Scheduler, Kubelet, and CRI / containerd / CNI / CSI. The process follows a numbered sequence of steps:

- 用户提交**: kubectl apply -f pod.yaml
- API Server**: 1. POST /pods
- API Server**: 2. AuthN/AuthZ
- API Server**: 3. Admission Ctrl
- API Server**: 4. Validation
- API Server**: 5. Persist to etcd
- API Server**: 6. Watch Event
- Scheduler**: 7. Add to ActiveQ
- Scheduler**: 8. PreFilter
- Scheduler**: 9. Filter (节点筛选)
- Scheduler**: 10. Score (打分)
- Scheduler**: 11. Reserve (Assume)
- Scheduler**: 12. Permit
- Scheduler**: 13. PreBind
- Scheduler**: 14. Bind (POST /bind)
- Kubelet**: 15. Watch Pod
- Kubelet**: 16. SyncLoop
- Kubelet**: 17. SyncPod
- Kubelet**: 18. Create PodSandbox
- Kubelet**: 19. Create Container(s)
- Kubelet**: 20. Start Container(s)
- CRI / containerd**: 21. RunPodSandbox
- CRI / containerd**: 22. CreateContainer
- CRI / containerd**: 23. StartContainer
- CNI / CSI**: 24. CSI Mount
- CNI / CSI**: 25. CNI Add
- CNI / CSI**: 26. Pod Ready

失败与重试路径 (Failure and Retry Path):

- Filter 失败** (Filter Failure) → **UnschedulablePods**
- Backoff (指数退避)** (Backoff (Exponential Backoff))
- PreBind/Bind 失败** (PreBind/Bind Failure) → **Requeue**
- Kubelet 启动失败** (Kubelet Start Failure) → **Restart Backoff**

```

graph TD
    subgraph "2.1 API Server - 请求处理链"
        A[net/http ServeHTTP] --> B[handler.ServeHTTP]
        B --> C[WithAuthentication]
        C --> D[WithAuthorization]
        D --> E[WithAdmission]
        E --> F[restful.RouteFunction]
        F --> G[鉴权校验 / 鉴证ing]
        G --> H[etcd.Store.Create]
        H --> I[etcd.Store.Put]
        I --> J[Watch.Event 分发]
    end

    subgraph "2.2 Informer - ListWatch别 Handle"
        K[Reflector.Run] --> L[ListAndWatch]
        L --> M[DeltaFIFO.Add]
        M --> N[Indexer.Add/Update/Delete]
        N --> O[SharedProcessor.distribute]
        O --> P[EventHandler.OnAdd/Update/Delete]
        P --> Q[WorkQueue.Add]
        Q --> R[Controller.RunWorker]
        R --> S[syncHandler(key)]
    end

    subgraph "2.3 Scheduler - 调度流程"
        T[scheduleOne()] --> U[getHeestPod()]
        U --> V[framework.RunPreFilterPlugins]
        V --> W[findNodesThatFit()]
        W --> X[framework.RunFilterPlugins]
        X --> Y[prioritizeNodes()]
        Y --> Z[framework.RunScorePlugins]
        Z --> AA[selectNode()]
        AA --> AB[bind()]
        AB --> AC[Assume -> Reserve -> Permit]
        AC --> AD[PreBind -> Bind]
    end

    subgraph "2.4 Kubelet - JSyncLoop"
        AE[RunKubelet()] --> AF[syncLoop (goroutine)]
        AF --> AG[pleg.Start()]
        AG --> AH[PodWorkers.UpdatePod()]
        AH --> AI[worker()]
        AI --> AJ[syncPod()]
        AJ --> AK[generateAPIPodStatus()]
        AK --> AL[updateRuntimeUp()]
    end

    subgraph "2.5 containerd - CRI 实现"
        AM[Serve() (gRPC)] --> AN[RunPodSandbox]
        AN --> AO[CreateContainer]
        AO --> AP[StartContainer]
        AP --> AQ[Task.New]
        AQ --> AR[Task.Start]
        AR --> AS[runc.Create]
        AS --> AT[runc.Start]
    end

    subgraph "2.6 CNI - Pod 网络创建"
        AU[kubelet -> CNI Add] --> AV[CNI plugin (e.g. Cilium)]
        AV --> AW[Init NetNS (pause)]
        AW --> AX[Create veth pair]
        AX --> AY[IPAM to bridge]
        AY --> AZ[Route / MTU / Policy]
        AZ --> BA[CNI Result 返回]
    end
  
```

3.1 调度框架阶段

Selection stage: Prefilter, Filter, PostFilter

Distribution stage: Score, Reserve, Permit, Bind, PostBind

3.1 AI 场景扩展点

- ✓ GPU 扩展调度评分 (NVLink / NUMA)
- ✓ Gang Scheduling (or Cuda)
- ✓ Multi-resource scoring (GPU-CPU-MEM)
- ✓ MIG 资源颗粒度划分
- ✓ Topology aware interconnect
- ✓ Avoid noisy neighbors
- ✓ Quota / Fairness

2.2 Default 插件（部分）

阶段	默认插件
Prefilter	NodeResourcesFit, Inout, PostTopologySpread
Filter	NodeResourcesFit, NodeAffinity, PostTopologySpread, InterPodAffinity, NodeUnschedulable, NodeName
PostFilter	DefaultPreemption
PreScore	NodeResourceFit, ImageLocality, InterPodAffinity, NodeAffinity, PostTopologySpread, TaintToleration
Reserve	NodeResourceFit
Permit	DefaultPermit
PreBind	VolumeBinding, NodeResourceFit
Bind	DefaultBinder
PostBind	DefaultPostBind

2.4 自定义插件 skeleton

```

type MyScorePlugin struct {
    Handle framework.Handle
}

func (p *MyScorePlugin) Name() string {
    return "MyScorePlugin"
}

func (p *MyScorePlugin) Score(
    cycleState *framework.CycleState,
    pod *v1.Pod, framework.CycleState,
    nodeName string,
    forNodeAtRevision, ran Status) (
    // 自定义打分逻辑
    )

```

4.1 DRA 组件关系

```

graph TD
    DC[DeviceClass  
(设备设备类型)] --> RC[ResourceClaim  
(Pod 申请)]
    RCT[ResourceClaimTemplate  
(模板)] --> RC
    RC --> RS[ResourceSlice  
(分配结果)]
    RS --> DD[DRA Driver (驱动: nvidia.com/dra)]
  
```

- ✓ 拓扑感知分配
- ✓ 设备健康检测
- ✓ 驱动库配置 (MIG 切片)
- ✓ 共享策略 (time-slicing)
- ✓ 动态热插拔 (未来)

4.2 GPU DRA 流程 (K8s :1.35+ Stable)

1. 管理层创建 DeviceClass (如 nvidia.com/gpu)
2. Pod 模板引用 ResourceClaimTemplate(s)
3. Pod 启动时, Kubelet 内由 DRA 驱动请求分配
4. 驱动返回 ResourceSlice (具体设备/配置)
5. Kubelet 将设备返回 (返回给 Pod 文件 / 环境变量)
6. Pod 运行时可动态扩缩设备 (future)

4.4 与 Device Plugin 对比

特性	Device Plugin	DRA
接口方式	gRPC / Unix Socket	K8s 原生 API
分配粒度	离散设备	任意资源/配置
动态配置	困难	支持 (未来)
拓扑感知	需自实现	标准化
K8s 版本	1.8+	1.35+ (Stable)

5.1 硬件层

- SM (Streaming Multiprocessor)
- Tensor Core (TFLOPS)
- HBM (High Bandwidth Memory)
- PCIe Switch / Root Complex
- NVLink / NVSwitch
- IB NIC / DPU

5.2 软件栈

- CUDA Driver (kernel module)
- CUDA Runtime
- cuBLAS / cuDNN / NCCL
- Framework (PyTorch / TF)
- vLLM / Triton / TensorRT-LLM
- Kubernetes

5.3 GPU 拓扑示例 (DGX Server)

The diagram illustrates a DGX Server topology. Two CPU Sockets (0 and 1) are connected to a PDU Switch. The PDU Switch is connected to a GPU Switch. The GPU Switch is connected to 16 GPUs (GPU0 to GPU15) arranged in two columns. The GPUs are connected via NVLink / NVSwitch. The diagram also shows a PCIe connection between the two GPU columns.

5.4 MIG (Multi-Instance GPU)

The diagram shows a physical GPU (A100 48GB) partitioned into four MIG instances (Multi-Instance GPU). The instances are labeled MIG 1g.5gb, MIG 1g.5gb, MIG 2g.10gb, and MIG 3g.20gb. These instances are connected to CI-0, CI-1, CI-2, and CI-3 respectively.

5.5 Time Slicing

The diagram illustrates time slicing on a GPU. A timeline shows three time slices (TS1, TS2, TS3) executing sequentially. TS1 is Pod A, TS2 is Pod B, and TS3 is Pod C. The timeline is divided into segments by dashed lines, indicating the execution of different pods over time.

7.1 标准回答结构

- 直接回答结论
- 原理解释 (why / how)
- 代码 / 系统说明
- 指标 / 命令 / 规则
- 用词-例句 / 穷举
- 总结-点睛

6.2 Scheduler 流程

- 接收请求 (prompt / params)
- 队列化 (waiting queue)
- 流控批处理 (continuous batching)
- Prefill phase (计算 prompt)
- Decode phase (生成 token)
- 返回 token (stream)
- 清理 / 回收

5.3 KV Cache 组织 (Paged KV Cache)

Logical KVCache 空间

Seq 1	Block 1	Block 2	Block 3	Block 4	Block 5
Seq 2	Block 6	Block 7	Block 8	Block 9	Block 10
...	Block 11	Block 12	Block 13	Block 14	Block 15
Seq M	Block 16	Block 17	Block 18	Block 19	Block 20
...	Block 21	Block 22	Block 23	Block 24	Block 25

5.4 容量计算公式 (示例)

KV Cache 占用 (bytes)

$$= 2 \times L \times L_{kv} \times D_{kv} \times \text{sizeoff} \times B \times T$$

L : 层数 (layers)
 L_{kv} : KV heads
 D_{kv} : head dim
 B : 排批 batch
 T : 批次 batch size

可缓存 Tokens

$$T = \left\{ \sum_{i=1}^L \sum_{j=1}^{L_{kv}} \sum_{k=1}^{D_{kv}} i \right\} \times B \times T$$

M_{kv} : 可用 KV Cache 内存

7.3 参考图要点 (节选)

图 48-1 v3 全栈复习地图与源码级复习路径

层次	面试必须做到	不要停留在
Linux	解释调度、内存回收、TCP/IO 数据路径，并能用工具证明瓶颈	只会 top/free/df
Kubernetes	从 API Object 到 Queue/Cache/Reconcile/Bind/SyncPod 串起源码调用链	只会 YAML 和 kubectl
GPU	解释 GPU 拓扑、设备发现、MIG/DRA、NUMA/RDMA 对 placement 的影响	只会 nvidia-smi
Inference	能推导 KV Cache/并发/吞吐，并解释 scheduler 的 admission/preemption	只会部署 vLLM
Distributed	能诊断 NCCL hang、RDMA path、collective mismatch 与网络拥塞	只会设置 NCCL_DEBUG
System Design	把 SLO、容量、调度、弹性、故障域、成本串成闭环	只画组件框图

49. Kubernetes 源码深挖：API Server / Informer / Scheduler / Kubelet

49.1 API Server：一次 Pod CREATE 的完整调用链

```

HTTP POST /api/v1/namespaces/<ns>/pods
-> genericapiserver / REST handler
-> authentication
-> authorization
-> mutating admission
-> validation
-> validating admission
-> REST storage strategy
-> etcd3 storage Create / guaranteedUpdate
-> watch cache / broadcaster
-> clients observe resourceVersion progression

```

- 面试关键点 1：API Server 是无状态控制平面入口，但它依赖 etcd 作为事实存储；本地 watch cache 是 `/Watch` 扩展机制，不是事实源。
- 面试关键点 2：Admission 发生在持久化之前；Mutating 可能改变对象，Validating 再决定是否允许。
- 面试关键点 3：resourceVersion 是并发控制与 Watch 增量消费的重要基础。不要把它解释成简单的数据库自增 ID。
- 面试关键点 4：高 QPS 场景要区分 LIST 风暴、WATCH fan-out、etcd 写延迟、APF 排队、serialization/response 带宽。

故障现象	优先确认	常见根因	可观测信号
kubectl create 延迟高	APIServer request latency 分位数	webhook 慢、etcd fsync 慢、APF 排队	apiserver_request_duration_seconds、webhook latency、etcd WAL fsync
controller 大量 relist	Watch 连接是否频繁断开	RV 过旧、网络中断、apiserver 重启、client timeout	watch error、410 Gone、reflector relist
API QPS 正常但 etcd CPU 高	求型象大小	大对象、高 churn CR、	etcd db size、backend

49.2 Informer : 从 ListAndWatch 到 Handler，再到 WorkQueue

```

SharedInformerFactory.Start()
-> SharedIndexInformer.Run(stopCh)
-> Controller.Run()
-> Reflector.Run()
  -> ListAndWatch()
    -> List() initial state
    -> Watch() incremental events
  -> DeltaFIFO.Add/Update/Delete/Replace
-> processLoop()
-> DeltaFIFO.Pop(processFunc)
-> HandleDeltas()
  -> Indexer.Add/Update/Delete
  -> sharedProcessor.distribute()
    -> processorListener -> ResourceEventHandler
  
```

为什么 Controller 通常不在 Handler 里直接做重活？

Informer 回调的职责应该尽量轻：提取 key、写 WorkQueue。真正的外部 API 调用/重试/幂等逻辑放在 worker/reconcile。这样能解耦事件接收速度和业务处理速度，并利用 rate-limiting queue 控制失败重试。

组件	核心数据结构/职责	面试追问
Reflector	List+Watch、RV、重连/重列	Watch 断了如何恢复？为什么需要 relist？
DeltaFIFO	按 key 聚合 Delta，连接 producer 和 consumer	为什么不是直接 channel？如何处理多次 update？
Indexer/Store	线程安全本地对象缓存与索引	Lister 为什么通常不访问 API Server？
sharedProcessor	把同一事件 fan-out 到多个 listener	慢 handler 是否拖垮其他 handler？
WorkQueue	去重、延迟、rate limit、重试	Add/Done/Forget/AddRateLimited 生命周期是什？

49.3 Scheduler : 从 Pending Pod 到 Binding 的源码级心智模型

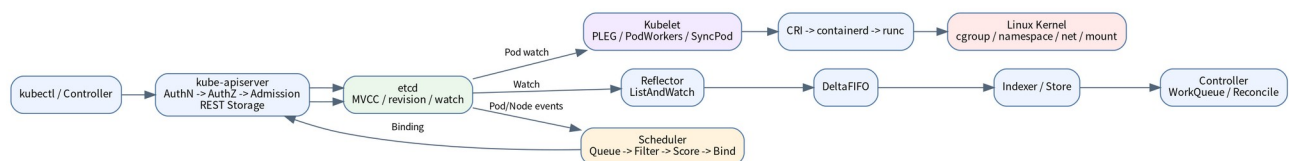


图 49-1 Kubernetes 核心组件源码链路 (v2 图复用并补充讲解)

```

scheduler.Run()
-> schedulingQueue.Run()
-> scheduleOne(ctx)
  -> NextPod() / activeQ.Pop()
  -> sched.SchedulePod()
    -> schedulingCycle()
      -> RunPreFilterPlugins
      -> findNodesThatFitPod / RunFilterPlugins
      -> prioritizeNodes / RunPreScorePlugins / RunScorePlugins
  
```

```

-> selectHost
-> assume()
-> Reserve / Permit
-> bindingCycle() [async]
-> PreBind
-> Bind
-> PostBind

```

- Assume 是 scheduler cache 中的乐观更新：先假定 Pod 已占用目标 Node 资源，避免并发调度周期把同一份资源重复分配。
- Scheduling cycle 失通常回到列； binding cycle 失败需要 Unreserve，并恢复 cache 状态。
- Filter 解决“能不能放”，Score 解决“更愿意放哪里”；PostFilter 常见实现是 preemption。
- AI 集群不能只看 CPU/Memory：GPU 型号、MIG profile、NVLink domain、NUMA、NIC proximity、gang co-scheduling 都可能影响 placement。

阶段	典型问题	AI 场景扩展
PreFilter/Filter	资源、亲和性、Taint、Topology、Volume	GPU/DRA Claim 可满足性、作业最小资源集
PostFilter	无节点可用后的抢占	高优先级训练/推理任务抢占时的 gang 语义
Score	多插件归一化+权重	NVLink/NIC/NUMA proximity，降低跨机通信
Reserve/Permit	预留与等待	Gang scheduling barrier / cohort
Bind	写 Binding/Pod nodeName	设备分配结果与节点最终状态的一致性

49.4 Kubelet : SyncLoop / PodWorkers / CRI 的真实边界

```

RunKubelet()
-> Kubelet.Run()
-> syncLoop()
  -> config updates / PLEG / runtime events / housekeeping
  -> HandlePodAdditions/Updates/Reconcile
  -> podWorkers.UpdatePod()
    -> per-pod worker
      -> SyncPod()
        -> admit / status / volumes / cgroups
        -> container runtime SyncPod
          -> RunPodSandbox
          -> CreateContainer
          -> StartContainer

```

常见错误理解

Kubelet 不是“看到 Pod 就直接 docker run”。它持续把期望状态与节点运行时状态收敛，PodWorkers 保证同一个 Pod 的 SyncPod 串行化，CRI 把 Kubernetes 对容器生命周期的需求转给 container runtime。

现象	Kubelet 侧排查顺序	常见根因
ContainerCreating 很久	Events -> kubelet log -> runtime -> CNI/CSI	镜像拉取、CNI ADD、CSI Mount、sandbox 创建
Pod Running 但不 Ready	readiness probe / endpoints / app logs	探针失败、依赖未就绪、网络策略
频繁重启	container status / exit code / OOM /	OOMKilled、liveness、应用 crash、node

	probes	pressure
节点 NotReady	kubelet heartbeat / runtime / disk / network	runtime hang、磁盘满、CNI 故障、control-plane connectivity

50. Staff 面试回答模板：20 个高频问题完整追问树

50.1 为什么 Linux load average 很高但 CPU utilization 不高？

标准回答：先定义 load average 是可运行任务与不可中断睡眠任务的平均压力，而不是 CPU 百分比。然后区分 runnable 与 D-state；如果 CPU 不高但 load 高，优先查 IO wait、存储延迟、NFS/、和不可中核路

高频追问：追问：如何证明？答：top 看 state，vmstat 看 r/b，pidstat -w/-d，ps wchan，iostat -x，perf/ebpf 看阻塞。

50.2 CPU request 与 limit 在 Linux 核分生效？

标准回答：request 主要影响调度权重（v1 cpu.shares / v2 cpu.weight），limit 通过 bandwidth quota（v1 CFS quota/period，v2 cpu.max）限制可消耗 CPU。request 不是硬上限，limit 才可能导致 throttling。

高频追问：追问：节点有空闲 CPU 为什么仍 throttling？因为 quota 是 cgroup 局部 bandwidth 约束，与节点总空闲并不矛盾。

50.3 为什么 memory request 不等价于 memory limit？

标准回答：request 首先是调度与 QoS 入；limit 是硬上限。cgroup v2/MemoryQoS 可把 request 映射为保护语义（如 memory.min/low），但这取决于 K8s 版本和配置。

高频追问：追问：容器 OOM 与 node OOM 如何区分？看 cgroup memory.events/OOMKill、kubelet status 与 kernel dmesg。

50.4 Informer 为什么要有本地 cache？

标准回答：目的是把大量 controller 的读从 API Server 移到共享本地 cache，并用 List+Watch 保持最终一致。Handler 收到事件后通常只写 workqueue，reconcile 再基于 cache 计算。

高频追问：追问：cache 不是强一致怎么办？reconcile 设计成幂等，必要写操作使用 resourceVersion/patch 突重试。

50.5 Scheduler 为什么需要 Assume？

标准回答：为了并发调度时乐观占用 scheduler cache，避免多个 Pod 都认为同一份节点资源可用。Bind 之后事实状态由 API Server/Kubelet 收敛。

高频追问：追问：Bind 失败怎么办？执行 Unreserve/forget assumed state，并重新入队。

50.6 NodeResourcesFit 的 MostAllocated 为什么能帮助降成本？

标准回答：它促进 bin packing，把 request 集中到更少节点，给 autoscaler 创造真正空节点。前提是 requests 准确，且要控制 PDB、拓扑、峰值与资源碎片。

高频追问：追问：为什么 AI GPU 集群不能仅用 MostAllocated？因为 GPU 拓扑和 gang 约束可能比简单密度更重要。

50.7 GPU Device Plugin 与 DRA 的本质区别是什么？

标准回答：Device Plugin 把设备暴露成扩展资源，Pod 通常按整数资源计数；DRA 使用 ResourceClaim/DeviceClass/ResourceSlice 等 API 表达更灵活的设备属性、选择和分配。

高频追问：追问：为什么 DRA 更适合复杂加速器？因为它能表达设备属性与动态选择，而非只有 vendor.com/gpu: 1。

50.8 MIG 与 Time-Slicing 怎么选？

标准回答：MIG 提供硬件级分区和更强隔离，适合多租户/SLO；time-slicing 提高共享率但隔离较弱，适合容忍抖动的开发/批理。

高频追问：追问：MIG 的代价？固定 profile 造成碎片，重配置复杂，某些跨实例能力受限。

50.9 为什么 GPU utilization 高不代表推理效率高？

标准回答：利用率只表示 GPU 在采样周期内忙，不直接代表有用 token throughput。可能在做低效 kernel、通信、memory-bound 工作或小 batch。要结合 tokens/s、TTFT、TPOT、HBM、SM occupancy 和 queue depth。

高频追问：追问：如何识别 memory-bound？看 HBM bandwidth、kernel profiling、算术强度与 tensor core utilization。

50.10 vLLM 为什么能比朴素 serving 提高吐？

标准回答：核心来自 continuous batching、PagedAttention/KV block 管理、调度器动态拼 batch、prefix caching/chunked prefill 等机制，提高 GPU 利用和显存效率。

高频追问：追问：吞吐提升是否会伤害 latency？会，需要控制 max_num_batched_tokens、max_num_seqs 与 admission。

50.11 KV Cache 容量怎么估算？

标准回答：近似与层数、KV heads、head dimension、token 数、dtype bytes 和并发序列数成正比。工程上应先得到每 token KV bytes，再用可用显存除以每请求上下文占用，保留模型权重和 runtime headroom。

高频追问：追问：GQA/MQA 为什么省 KV？因为 KV head 数少于 query head 数。

50.12 Prefill 与 Decode 为什么要分开看？

标准回答：Prefill 对长 prompt 做高并行矩阵计算，通常 compute-heavy；Decode 每步只生成少量 token，更 latency-sensitive，往往更 memory/launch-bound。它们的最佳 batching 策略不同。

高频追问：追问：Disaggregated Prefill 的收益和代价？可独立优化 TTFT/ITL，但增加 KV transfer、路由与容量协调复杂度。

50.13 NCCL AllReduce hang 第一反是什？

标准回答：先排除 rank/collective 调用不一致和进程失败，再确认 topology、NVLink、NIC/RDMA、MTU/PFC/ECN、GPU Xid/ECC、driver/NCCL 境一致性。NCCL 2.31 可先跑 RAS/active diagnostics。

高频追问：追问：为什么一个 rank 错误会拖住所有 rank？Collective 需要所有 rank 按一致序列参与，缺席 rank 会形成全局等待。

50.14 RoCE 丢包为什么比普通 TCP 更敏感？

标准回答：RDMA 追求低延迟/低 CPU，RoCE 通常依赖精心配置的拥塞/无损网络策略。PFC 配置错误可出现 pause storm，ECN/DCQCN 不造成吐和尾延。

高频追问：追问：怎么定位？交换机 PFC/ECN counters + NIC counters + RDMA perf + NCCL tests。

50.15 Ray/KubeRay 与原生 Deployment 有什么不同？

标准回答：RayCluster 有 head/worker 拓扑和分布式任务语义，KubeRay Operator 负责 CR 的生命周期收敛；Ray Serve 又增加 deployment/replica/router 的应用层伸缩。

高频追问：追问：为什么会出现双重 autoscaling？Ray worker、K8s pod、node autoscaler 可能都有控制回路，需要明确责任边界。

50.16 Inference autoscaling 为什么不能只按 GPU utilization？

标准回答：GPU utilization 是滞后且不区分负载类型。更好的信号包括 queue depth、active sequences、tokens backlog、TTFT/TPOT、KV pressure、arrival rate，再结合 warmup/cold-start。

高频追问：追问：如何避免振荡？不同层采用不同时间常数、hysteresis、cooldown 和 capacity headroom。

50.17 如何设计 GPU-aware model rollout？

标准回答：先校验模型/driver/runtime compatibility，预拉权重并暖机，按小流量 canary 比较 TTFT/TPOT/error/tokens/s/GPU memory，再逐步扩大。回滚必须考虑缓存格式与正在执行请求。

高频追问：追问：为什么不能只看 HTTP error rate？性能回退、OOM、KV thrash 可能在 error rate 之前出现。

50.18 Agent Sandbox 的最小安全边界是什么？

标准回答：至少要有独立 execution identity、文件系统隔离、资源限制、default-deny egress、短期凭证、审计、超时和销毁/reset。高风险不可信代码优先考虑 MicroVM/Kata/gVisor 等更强隔离。

高频追问：追问：为什么 rootless container 仍不等于强 sandbox？共享 host kernel 仍是关键攻击面。

50.19 100k QPS LLM 平台怎么做容量设计？

标准回答：先把 QPS 转为输入/输出 token rate，再结合模型在目标 GPU 上的可持续 tokens/s 与 latency SLO，计算 replica/GPU 数；再加入峰值系数、失败冗余、滚动升级和 region headroom。

高频追问：追问：为什么直接用 benchmark 最大吞吐会低估容量？生产需要 latency SLO、request length distribution、fragmentation 与故障余量。

50.20 AI Infra 如何做成本优化而不伤 SLO？

标准回答：把成本拆成 GPU-hour、idle headroom、KV/模型内存效率、cross-AZ/network、storage/model loading、失败重试；任何优化必须用 TTFT/TPOT/error/availability 对照验证。

高频追问：追问：最常见错误？单纯提高 GPU utilization，结果 queueing 和 tail latency 化。

51. GPU / CUDA / NCCL / RDMA : 从硬件数据路径到故障定位

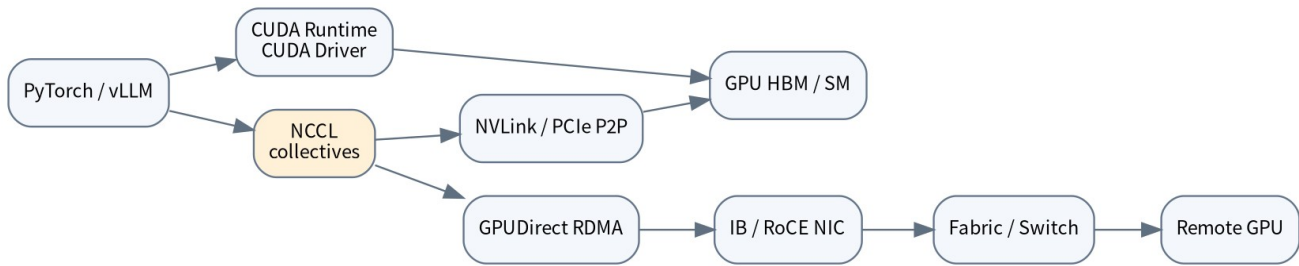


图 51-1 GPU 数据路径 : CPU/PCIe/NVLink/NIC/RDMA

51.1 GPU 性能四个资源维度

维度	瓶颈表现	关键指标	常见优化
Compute	SM/Tensor Core 和	SM active、Tensor Core utilization、kernel duration	更大 batch、kernel fusion、量化/更高效 kernel
HBM/Memory	带宽/容量受限	HBM BW、memory used、cache miss	Paged KV、量化、GQA、offload
Interconnect	GPU-GPU 通信慢	NVLink BW、NCCL bus BW	topology-aware placement、collective tuning
Host/Network	CPU/NIC/RDMA 成瓶颈	PCIe BW、NIC counters、RDMA latency	NUMA pinning、GPUDirect、NIC proximity

51.2 NCCL Collective : 必须能口述的语义

Collective	语义	典型 AI 使用	错误模式
AllReduce	所有 rank 得到同果	Data Parallel gradient sync	rank 数/顺序不一致 -> hang
AllGather	每个 rank 收集所有 rank 的分片	Tensor/sequence parallel	链路瓶颈、内存压力
ReduceScatter	Reduce 后把结果分片给各	DP/FSDP	分片大小不一致
AllToAll	每个 rank 向所有 rank 发送不同分片	MoE Expert Parallel	网络 fan-out、拥塞、尾延迟
Broadcast	root 向所有 rank 发送	参数/metadata 分发	root 异常、链路不对称

NCCL 2.31 面试更新

NCCL 2.31 文档新增/强化了 RAS diagnostics 与 active diagnostics。RAS 检查 GPU inventory、driver、ECC、NVLink、NCCL 环境一致性；active diagnostics 会实际验证 P2P 等通信路径。排障时可以把它放在 nccl-tests 和底层交换机/NIC 诊断之前。

51.3 NCCL Hang 标准故障树

1. Application correctness
 - all ranks entered same collective?
 - count / datatype / order consistent?
 - any rank crashed or OOM?
2. GPU / driver
 - Xid / ECC / NVLink down?
3. Topology

- nvidia-smi topo -m
- PCIe ACS / NUMA / NIC affinity
- 4. Transport
 - IB/RoCE device up? GID? MTU? RDMA counters?
- 5. Fabric
 - PFC pause storm? ECN marking? packet loss?
- 6. NCCL
 - RAS diagnostics / active diagnostics / NCCL_DEBUG
- 7. Reproduce
 - nccl-tests pairwise -> single-node -> multi-node

52. vLLM Internals : Scheduler、KV Cache Manager 与性能旋钮



图 52-1 vLLM 推理引擎与 KV Cache 关键路径

52.1 Scheduler 需要解决什么？

- Admission：哪些 waiting request 本轮可以进入执行。
- Token budget：max_num_batched_tokens / max_num_seqs 限制单轮工作量。
- Chunked prefill：长 prompt 可被分块，与 decode 共享执行机会，改善 head-of-line blocking。
- Preemption：KV 不足或调度资源不足时，运行请求可能回到 waiting，再恢复。
- Async scheduling：减少 CPU scheduler 造成的 GPU bubble，但会改变并发与调试复杂度。

参数/机制	影响	调优方向
max_num_batched_tokens	单 iteration token 工作预算	高 -> 吞吐可能更好，但 decode latency 可能变差
max_num_seqs	单 iteration 活跃序列数	高 -> 并发提高，但 KV/调度压力增加
chunked prefill	长 prompt 分片	降低长 prefill 对 decode 的阻塞
prefix caching	重复 prefix 命中复用 KV	适合重复 system prompt / shared context
preemption	KV 紧张时让出资源	频繁发生通常意味着 capacity/admission 不合理

52.2 Hybrid KV Cache Manager：为什么面试值得关注

- 前 vLLM 设计中 KVCacheManager 是 scheduler 与 KV 管理系统的边界。
- Hybrid KV Cache Manager 可为不同 attention 类型维护不同 KV group，并由 coordinator 分配果。
- 这意味着“KV cache = 一块简单连续显存”是过时理解；真实系统要管理 block、prefix reuse、引用计数、不同 attention 类型以及未来的 offload/tiering。
- 最新文档也出现多层 KV offload/tiering 设计：GPU/CPU/存/网络之间可以形成层级缓存，但代价是传输延迟和复杂性。

面试推导：

$KV \text{ bytes / token} \approx 2(K,V) \times \text{layers} \times \text{kv_heads} \times \text{head_dim} \times \text{dtype_bytes}$

$\text{request KV} \approx KV \text{ bytes/token} \times (\text{prompt_tokens} + \text{generated_tokens})$

$\text{max concurrent sequences} \approx \text{allocatable_KV_bytes} / \text{expected_KV_per_request}$

生产容量必须再乘：fragmentation factor、prefix hit ratio、preemption headroom、P95/P99 context distribution。

53. Ray / KubeRay : Operator Reconcile 与多层弹性控制

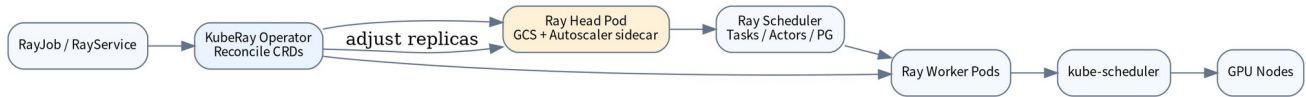


图 53-1 KubeRay / Ray Serve 控制平面与数据平面

RayCluster CR

- > KubeRay controller watch
- > reconcile desired head/worker groups
- > create/update Services, Pods, Config
- > observe Pod / Ray status
- > update CR status

RayService

- > manages RayCluster + Serve application
- > health/upgrade logic
- > traffic continues through stable service endpoint

控制回路	控制对象	时间尺度	风险
Ray Serve autoscaler	Serve replicas	秒级~分钟	请求指标噪声、冷启动
Ray autoscaler	Ray workers	分钟级	worker 启动慢、资源碎片
K8s scheduler/Kueue	Pod admission/placement	秒级	gang/queue head-of-line
Node autoscaler/Karpenter	GPU nodes	分钟级	云容量不足、GPU warmup
Model loader/cache	权重与 engine warmup	分钟级	thundering herd、共享存储瓶颈

Staff 级重点

不要让所有控制器都追同一个信号。推荐把 request-level scaling、worker-level scaling 和 node-level capacity 分层，并让上层的时间常数更短、下层更慢；否则非常容易产生 oscillation。

54. 六个 Staff-Level AI Infra System Design 完整答题框架

54.1 设计一个多租户 LLM Inference Platform

- SLO : TTFT/TPOT/availability/region
- Tenant : quota、priority、max context、rate limit
- Router : model/version/tenant aware + load aware
- Serving : vLLM/Ray Serve , GPU pool 分层
- Scheduler : GPU type/MIG/DRA/topology
- Autoscaling : queue/tokens backlog + warm pool
- Observability : per-tenant token/cost/SLO
- Failure : GPU Xid、node loss、region failover

答题顺序：先 SLO/规模/约束 -> 流量和容量模型 -> 控制平面/数据平面 -> 状态与一致性 -> failure domain -> observability -> cost/trade-off。

54.2 设计 10k GPU Training Fleet Scheduler

- 队列与 quota : team/project/fair share
- Gang : all-or-nothing admission
- Topology : NVLink domain / rack / IB fabric
- Preemption : checkpoint-aware
- Node health : GPU/NVLink/NIC quarantine
- Capacity : reserved + spot + multi-region
- Artifact : dataset/model checkpoint locality
- Recovery : elastic/restart/checkpoint

答题顺序：先 SLO/规模/约束 -> 流量和容量模型 -> 控制平面/数据平面 -> 状态与一致性 -> failure domain -> observability -> cost/trade-off。

54.3 设计 Agent Sandbox Platform

- Threat model : untrusted code/tool/browser
- Isolation : MicroVM/gVisor/Kata tier
- Identity : per-execution short-lived token
- Network : default deny + broker/proxy
- Storage : ephemeral workspace + artifact scan
- Resource : CPU/mem/pid/io/time quotas
- Warm pool : snapshot/reset correctness
- Audit : commands/network/credential usage

答题顺序：先 SLO/规模/约束 -> 流量和容量模型 -> 控制平面/数据平面 -> 状态与一致性 -> failure domain -> observability -> cost/trade-off。

54.4 设计 GPU Fleet Observability 平台

- Inventory : GPU/driver/NVLink/NIC/NUMA
- Health : Xid/ECC/thermal/throttle
- K8s : Pod/claim/node/scheduler events
- Serving : TTFT/TPOT/tokens/s/KV pressure
- Distributed : NCCL/RDMA diagnostics
- Correlation : request -> pod -> GPU -> NIC
- Automation : quarantine/drain/remediation
- Cost : GPU-hour/tenant/model

答题顺序：先 SLO/规模/约束 -> 流量和容量模型 -> 控制平面/数据平面 -> 状态与一致性 -> failure domain -> observability -> cost/trade-off。

54.5 设计跨 Region Inference Failover

- 正常流量 : locality first
- 容量 : N+headroom / reserved failover pool
- State : 模型、配置、router metadata
- Routing : health/SLO driven failover
- Protection : admission + degraded mode
- Data : 避免跨区 KV 依赖

- Recovery : gradual fallback
- Cost : standby vs recovery time

答题顺序 : 先 SLO/规模/约束 -> 流量和容量模型 -> 控制平面/数据平面 -> 状态与一致性 -> failure domain -> observability -> cost/trade-off。

54.6 设计模型权重分发与冷启动优化

- Artifact format/version/content hash
- Object store + regional cache
- Node local cache / image layer cache
- Parallel download 与 bandwidth shaping
- Preload / warm pool
- Engine compile/warmup
- Thundering herd protection
- Cache eviction 与 consistency

答题顺序 : 先 SLO/规模/约束 -> 流量和容量模型 -> 控制平面/数据平面 -> 状态与一致性 -> failure domain -> observability -> cost/trade-off。

55. 实战 Runbook : 从症状到根因的跨层排障

55.1 TTFT 突然从 500ms 升到 4s

42. 入口 : 确认是否所有模型/tenant/region 都受影响, 拆 queueing vs compute。
43. Router : queue depth、load balance 是否倾斜。
44. vLLM : waiting/running、prefill tokens、chunked prefill、KV pressure/preemption。
45. GPU : util、HBM、clock/throttle、Xid。
46. Node : CPU throttling、NUMA、PCIe。
47. Network : 如果 disaggregated/remote KV, 检查 RDMA/NIC。
48. Capacity : arrival rate 是否超过可持续 token throughput。

55.2 GPU utilization 95% 但 tokens/s 下降 40%

49. 先验证 workload mix 是否变长 prompt/更长 output。
50. 看 kernel/SM/HBM, 不要只看 utilization。
51. 检查 batch size、max_num_batched_tokens、KV fragmentation/preemption。
52. 检查 NCCL/TP 通信占比。
53. 检查 thermal/power clock。
54. 对比同模型基准, 分离模型配置变化。

55.3 Multi-node vLLM/NCCL 偶发 Hang

55. 检查 collective 顺序/rank crash。
56. NCCL RAS/active diagnostics。
57. GPU Xid/ECC/NVLink。
58. nvidia-smi topo -m / NIC locality。
59. RDMA counters、MTU/GID。
60. Fabric PFC/ECN/pause。

61. nccl-tests 缩小到 pair/node/fabric。

55.4 Pod Pending 但集群明明有空 GPU

- 62. kubectl describe Pod 看 scheduler reason。
- 63. 确认 resource name / DRA claim / device class。
- 64. Node taint/affinity/topology。
- 65. MIG profile 是否碎片化。
- 66. Gang/queue 是否未 admission。
- 67. Scheduler extender/plugin/DRA controller 健康。
- 68. 节点 GPU health 是否被 quarantine。

55.5 Agent Sandbox 出现跨租户数据残留风险

- 69. 立刻禁用 sandbox reuse/warm snapshot pool。
- 70. 确认 workspace mount、tmp、shared cache、browser profile。
- 71. 审计 credential broker token 生命周期。
- 72. 检查 snapshot/reset 是否清理 process/network state。
- 73. 采用新实例 per execution 作为安全基线，再逐项恢复 warm optimization。

56. 新增 20 个进阶 Hands-on Lab（总实验数提升到 50+）

编号	主题	完成标准
Lab 31	Scheduler Framework Plugin	写一个 Score Plugin，根据自定义 node label + resource usage 打分，并记录每个 extension point latency。
Lab 32	Scheduler Queue	制造 Unschedulable Pod，观察 activeQ/backoff/事件激活行和 scheduler logs。
Lab 33	Informer	写 SharedInformer + RateLimitingQueue controller，模拟 Watch 断线和对象快速更新。
Lab 34	Kubelet/CRI	用 crictl 跟踪 sandbox/container lifecycle，并把事件与 kubelet 日志。
Lab 35	cgroup v2	手动实验 cpu.weight/cpu.max/memory.max/memory.high/pressure。
Lab 36	DRA	部署一个 DRA 示例 driver，创建 DeviceClass/ResourceClaim，观察 allocation status。
Lab 37	MIG	切换 MIG profile，测量碎片和不同 workload 的隔离性。
Lab 38	GPU topology	采集 nvidia-smi topo -m 与 lspci/numactl，构建 GPU-NIC-NUMA 拓扑表。
Lab 39	NCCL	nccl-tests 做 NVLink vs PCIe vs multi-node 基准。
Lab 40	NCCL diagnostics	运行 NCCL 2.31 RAS/active diagnostics 并保存健康基线。

Lab 41	RoCE	采集 NIC/RDMA/PFC/ECN counters，做 congestion 故障注入（测试环境）。
Lab 42	vLLM batching	改变 max_num_batched_tokens/max_num_seqs，画 TTFT/TPOT/tokens/s 曲。
Lab 43	KV pressure	长上下文压测，观察 KV usage、preemption、prefix cache hit。
Lab 44	Chunked Prefill	构造长短 prompt 混合流量，对比开启/关闭 chunked prefill。
Lab 45	Distributed inference	KubeRay + vLLM 两节点推理，记录 NCCL/RDMA 与 serving latency。
Lab 46	Autoscaling	按 queue depth/tokens backlog 做 HPA/KEDA controller，观察 oscillation。
Lab 47	GPU failure	模拟节点 drain/GPU unhealthy，验证 router 和 workload recovery。
Lab 48	Agent sandbox	实现 default-deny egress + credential proxy + wall-clock timeout。
Lab 49	Model cache	实现 node-local model cache 与预热，测 cold-start P50/P99。
Lab 50	End-to-end observability	把 request id 关联到 model pod/GPU/NCCL/node，形成一次完整 trace。

57. 源码导航清单：面试前应该实际读哪些目录

项目	建议目录/模块	阅读目标
Kubernetes	pkg/scheduler, pkg/scheduler/framework, pkg/scheduler/backend/queue	scheduleOne、framework extension points、scheduling queue/cache
client-go	tools/cache	Reflector、DeltaFIFO、SharedIndexInformer、Indexer
Kubelet	pkg/kubelet, pkg/kubelet/pod_workers.go, kuberuntime	SyncLoop、PodWorkers、SyncPod、CRI runtime manager
containerd	core/runtime, plugins/cri	CRI service -> task -> shim/runtime 边界
runc	libcontainer	OCI spec -> namespace/cgroup/process
vLLM	vllm/v1/core/sched, vllm/v1/core/kv_cache_manager.py	scheduler、preemption、KV allocation/prefix cache
KubeRay	ray-operator/controllers/ray	RayCluster/RayService reconciliation
NCCL	user guide + tests	collective contract、transport、RAS/diagnostics
Linux	kernel/sched, mm, net, block	只需形成关键路径，不要求背代码

58. 2026-08 Research Update：版本敏感知识

- vLLM 最新 Scheduler 配置继续强化 async scheduling、chunked prefill 等能力；参数含义和默认值变化较快，面试前应以目标版本文档为准。
- vLLM Hybrid KV Cache Manager 当前明确分为 KVCacheManager -> Coordinator -> SingleTypeKVCacheManager 层次，用来处理不同 attention 类型的 KV group。

- NCCL 2.31 提供 RAS diagnostics 与 active diagnostics ；前者偏配置/健康一致性，后者实际验证数据通信路。
- NCCL Collective 的契约要求各 rank 以匹配的 count/datatype 调用相同 collective ；违反可导致 hang、crash 或数据损坏。

官方资料（2026-08-31 检索）

- <https://docs.vllm.ai/en/latest/api/vllm/config/scheduler/>

- https://docs.vllm.ai/en/stable/design/hybrid_kv_cache_manager/

- <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/troubleshooting/diagnostics.html>

- <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/collectives.html>

59. v3 Interview Readiness Matrix ：何时算“准备好”

领域	基础合格	Senior 合格	Staff/AI Infra 合格
Linux	会的工具	能解释内核机制并定位 CPU/mem/io/net	能跨 NUMA/PCIe/RDMA/GPU 分析性能
K8s	会部署/排障	能口述组件内部机制和源码链	能设计 scheduler/controller/large-scale control plane
GPU	会用 nvidia-smi	懂 CUDA/MIG/topology/operator	懂 fleet scheduling/health/RDMA/NCCL
Inference	会部署模型	懂 prefill/decode/KV/batching	能做容量/路由/autoscaling/SLO trade-off
Distributed	知道 NCCL/Ray	能做 multi-node inference	能诊断 collective/fabric 并设计恢复
Agent	知道 sandbox	懂 namespace/cgroup/gVisor/Kata	能设计 identity/network/credential/warm-pool 安全边界
System Design	画架构	给容量与故障策略	能量化 SLO/cost/headroom，并解释控制回路

最终建议

面试前不要只继续阅读。用 Lab 31-50 中至少完成 8 个，并为每个实验保存：架构图、命令、指标截图、故障注入、结论和可复现 README。面试中的可信度往往来自“我实际测过什么”，而不是“我读过什么”。

v3 Expanded Edition | Updated 2026-08-31 | Kubernetes -> GPU Platform -> Inference -> Agent Runtime

60. 最后 7 天：Mock Interview 冲刺计划

时间	主题	任务
D-7	Linux + K8s Internals	用白板口述 Pod 生命周期、Informer、Scheduler、Kubelet；做 2 个 Linux 故障题
D-6	GPU + CUDA	复盘 NUMA/PCIe/NVLink/MIG/DRA；完成 GPU topology

D-5	vLLM	口述 Prefill/Decode/KV/Batching ; 完成一次长上下文 KV 压测
D-4	NCCL/RDMA	做 NCCL hang 故障树；复盘 RAS/active diagnostics 与 RoCE counters
D-3	Ray/KubeRay	口述 reconcile 和多层 autoscaling ; 设计一个 RayService rollout
D-2	System Design	限时 45 分钟完成多租户 Inference Platform + 10k GPU Scheduler 两题
D-1	行为面试 + Portfolio	每个项目准备背景/决策/失败/指标/结果；停止新增知识点，只复盘弱项

面试当天回答原则：先确认约束，再给结论；每个结论至少补一个机制、一个指标、一个 trade-off。遇到不会的源码细节，不猜具体函数名，明确说明版本差异并回到稳定的组件边界。

60.1 最后自检

- 能否在 10 分钟内画出 Pod 从 API Server 到 Running 的完整路径？
- 能否解释 requests/limits 到 cgroup v2 文件的映射？
- 能否解释 GPU Pod 为什么可能 Pending，使 nvidia-smi 看起来有空 GPU？
- 能否从 TTFT 升高一路排到 queue -> vLLM -> KV -> GPU -> NCCL/RDMA？
- 能否推导一个模型在目标 GPU 上的大致并发和容量，而不是只引用 benchmark？
- 能否说清 Agent Sandbox 的 threat model、identity、network、credential 与 reset 边界？
- 能否给出至少 3 个亲自做过的实验和数据结果？

53. 最终版说明：面向 Senior / Staff Kubernetes & AI Infrastructure 面试

最终版定位

最终版以“可解释、可设计、可排障、可落地”为目标。每个主题都尽量连接 Linux 内核、Kubernetes 控制面、GPU/网络数据面、LLM serving 与真实生产 SLO。建议按章节末的追问树做口述练习，而不是背诵名词。



Senior Kubernetes -> AI Infrastructure Interview Handbook

面试深度扩展版 v3.0 (120+ 页完整版)



v3.0 终极版

共 128 页

Kubernetes Internals

Linux Kernel

GPU & CUDA

NCCL & RDMA

vLLM Internals

Ray & KubeRay

Agent Runtime

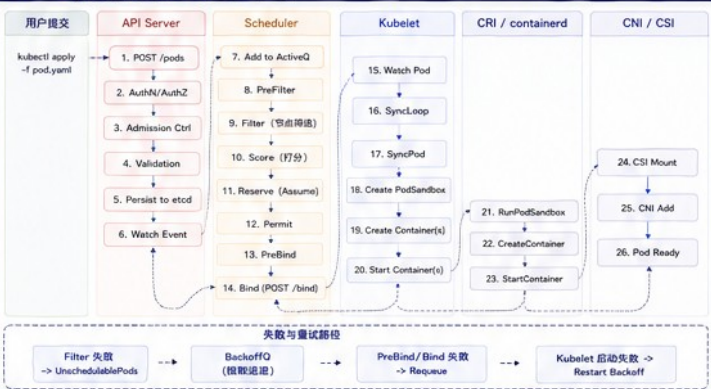
System Design

Observability

0. 知识地图 (升级版)



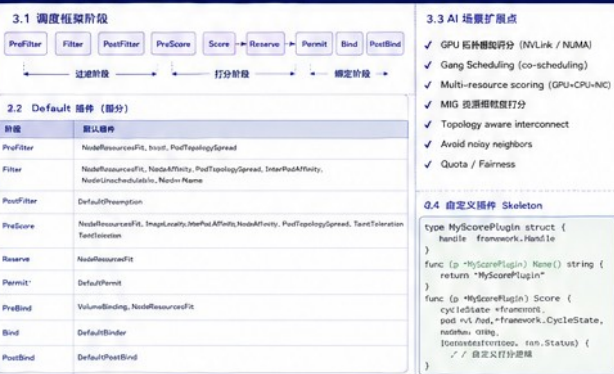
1. Pod 生命周期源码级流程图 (从提交到运行)



2. K8s 各核心组件源码调用链 (关键路径)



3. Scheduler 深入: 插件机制与扩展 (含 AI 场景)



4. DRA (Dynamic Resource Allocation) 与 GPU 调度



5. GPU 架构与系统

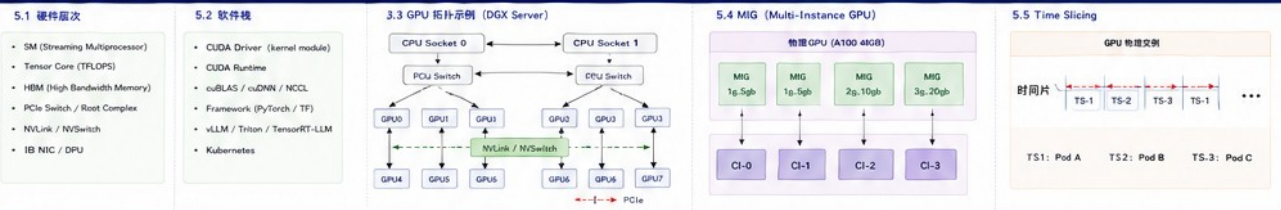


图53-1 全栈AI Infrastructure 面试知识地图

能力层	必须掌握	Staff 要求
Linux	CPU/Memory/IO/TCP/cgroup/namespace	能够从症状定位到 kernel/cgroup/network 数据路径
Kubernetes	API Server/etcd/Informer/Scheduler/Kubelet/CNI/CSI	能解释源码边界、一致性、失败恢复、扩展点和容量瓶颈
GPU Platform	CUDA/driver/MIG/DRA/topology/DCGM	能设计 GPU placement、隔离、健康治理与容量模型
Distributed GPU	NCCL/NVLink/RDMA/IB/RoCE	能定位 hang/性能退化，并理解 collective 对拓扑和网络的要求
LLM Serving	vLLM/KV cache/prefill/decode/batching	能把 tokens、显存、并发、TTFT/TPOT 转成容量与调度策略
Platform Design	autoscaling/rollout/multi-region/agent sandbox	以 SLO、故障域、成本、安全驱动架构 trade-off

54. Linux 性能工程终极复习：从指标到 Kernel 路径

54.1 CPU : runqueue、CFS/EEVDF、context switch 与 throttling

Application thread -> runnable -> per-CPU runqueue -> scheduler picks task -> context switch

Kubernetes request -> cpu.weight (v2)

Kubernetes limit -> cpu.max (v2) -> bandwidth throttling

- 面试时先区分 CPU utilization、load average、run queue length 与 throttling。它们描述的是不同层面的压力。
- 高 system CPU 往往指向 syscall、network packet processing、filesystem、page fault、锁竞争或 eBPF/iptables 等内核路径。
- 高 context switch 不一定是问题；关键是 voluntary/non-voluntary switch 是否与吞吐下降和 scheduler latency 同时出现。
- NUMA 系统要关注 CPU affinity、memory locality、IRQ affinity 和 GPU/NIC PCIe locality。

症状	优先指标	命令/工具	典型根因
CPU 100%	user/system/steal、runq	top, pidstat -u, mpstat, perf	忙循环、锁竞争、syscall、虚拟化 steal
load 高 CPU 低	r/b、D state、iowait	vmstat, ps, iostat, bpftrace	IO/NFS/块设备等待、不可中断睡眠
throttling	nr_throttled/throttled_usec	cat cpu.stat, container metrics	CPU limit 太低、短 burst 被 quota 截断
单核热点	per-CPU util、IRQ	mpstat -P ALL, /proc/interrupts	单线程、RSS/IRQ 不均、软中断热点

54.2 Memory : page fault、reclaim、PSI、OOM 与 cgroup v2

virtual address -> page table -> TLB -> physical page

minor fault: page already available / mapping fix

major fault: storage IO required

pressure -> reclaim -> direct reclaim / kswapd -> OOM if progress impossible

概念	面试表述	关键证据
RSS	进程当前驻留物理内存，不等于全部虚拟地址空间	/proc/<pid>/smaps, ps
Page Cache	文件数据的内核缓存，可在压力下回收	free, /proc/meminfo, cachestat
Anonymous	heap/stack 等匿名页，通常比 page cache 更难无损回收	smaps, memory.stat
PSI	CPU/Memory/IO 压力导致任务停顿的时间比例	/proc/pressure/*, cgroup pressure
cgroup OOM	某 cgroup 超过可用上限/无法 reclaim	memory.events, dmesg, pod status
Node OOM	全局内存压力触发内核选择 victim	dmesg, node metrics, kubelet events

Staff 级追问

如果 Pod 未达到 memory.max 仍然性能变差，要考虑 memory.high、reclaim、working-set 扩大、page cache 抖动、NUMA remote memory、THP/page fault 以及 node pressure。

54.3 IO 与文件系统：VFS -> page cache -> block layer -> device

read/write -> syscall -> VFS -> filesystem -> page cache -> block layer -> NVMe/EBS/local SSD

fsync path adds durability latency; remote volume adds network + storage backend latency

问题	第一层判断	深入定位
df 有空间但写不了	inode/readonly/quota	df -i, mount, dmesg, project quota
删文件空间没释放	open fd still holds inode	lsof +L1
iowait 高	await/queue depth/util	iostat -x, pidstat -d, blktrace/bpf
fsync 慢	device/WAL/backend latency	fio fsync test, storage metrics
容器层 IO 慢	overlays/copy-up/ephemeral disk	container runtime, mountinfo, node disk

54.4 TCP/Linux Networking : 一次 request 的数据路径

NIC -> driver/NAPI -> softirq -> skb -> routing/netfilter -> socket receive queue -> userspace
userspace send -> socket -> TCP -> routing/netfilter -> qdisc -> driver -> NIC

现象	检查	常见解释
SYN timeout	route/SG/NACL/firewall/listen backlog	路径不可达、ACL、conntrack、server overload
大量 TIME_WAIT	连接创建率、主动关闭方	正常短连接生命周期或连接复用不足
大量 CLOSE_WAIT	应用 fd 是否关闭	peer 已关闭但应用未 close
retrans 高	NIC/drop/MTU/拥塞	网络丢包、buffer、MTU、ECMP/链路问题
softirq 高	NET_RX/NET_TX、RSS/RPS	包速率高、单队列热点、iptables/conntrack 开销

55. Kubernetes Control Plane 源码专章

55.1 API Server : Request Pipeline、APF 与 Watch Cache

HTTP -> Authentication -> Authorization -> APF -> Mutating Admission -> Validation -> Validating Admission -> REST storage -> etcd

Read/List/Watch may leverage watch cache; writes persist through storage layer.

- APF (API Priority and Fairness) 是控制 API Server 过载的重要机制：先分类 flow，再按 priority level 分配并发和排队。
- 大规模集群要特别关注 LIST/WATCH 行为：控制器 restart storm、CRD 大对象、频繁更新 status 都可能放大 apiserver/etcd 压力。
- Webhook 是常见尾延迟与可用性风险点；必须设置 timeout、failurePolicy，并避免 webhook 自身依赖被它拦截的资源。

瓶颈层	症状	指标/日志	作
APF	429/排队	apiserver flowcontrol metrics	调 priority level / seat / client QPS
Admission	create/update tail latency	webhook duration/reject	缩短 timeout、HA、fail-open/closed 权衡
Serialization	CPU 高、大对象慢	request size/CPU profile	对象大小 / 无意义 status
etcd	write latency、watch lag	fsync/commit/db size	独立磁盘、compaction/defrag、控制 churn
Watch	controller relist 风暴	watch termination/410	稳定连接、合理 resync、版本兼容

55.2 etcd : Raft、WAL、Snapshot、Compaction 与一致性

client write -> leader -> WAL append/fsync -> Raft replicate -> quorum ack -> apply -> backend DB

linearizable read may consult leader/quorum; watch consumes committed revision stream

主题	必须说清	常见误区
Raft	leader + majority quorum 保证日志一致性	不是“所有节点都写成功才返回”
WAL	顺序持久化是写延迟关键	只看 CPU，不看 fsync
Snapshot	缩短日志恢复长度	snapshot 不是 compaction 的替代
Compaction	删除旧 revision 的历史可见性	不等于立即释放磁盘空间
Defrag	整理 backend DB 释放文件空间	不应无脑高频执行

面试故障题

API Server p99 突然升高但 QPS 没变：应同时检查 etcd fsync、leader changes、disk latency、webhook、APF queue、对象大小和节点网络，而不是只重启 apiserver。

55.3 Informer / Controller : 最终一致、幂等与 WorkQueue

ListAndWatch -> DeltaFIFO -> Indexer -> Handler -> key -> RateLimitingQueue -> worker -> Reconcile -> API write

设计原则	为什么	实现方式
Handler 量	避免阻塞 event fan-out	只做过滤和 enqueue key
Reconcile 幂等	事件可能重复/丢失语义要靠状态收敛	每次从期望/实际状态重新计算
使用 cache 读	减轻 API Server	Lister/Indexer
写时并发控制	cache 可能滞后	resourceVersion/patch/conflict retry
失败退避	避免坏对象热循环	AddRateLimited/Forget

56. Scheduler 源码专章：Queue -> Framework -> Cache -> Bind

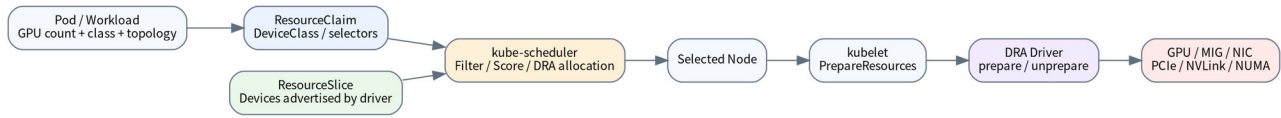


图 56-1 Scheduler + GPU-aware scheduling 关键路径

56.1 Scheduling Queue 与重试

集合	作用	Pod 迁移条件
activeQ	前可度， QueueSort 决定 Pop 顺序	新 Pod、事件激活、backoff 到期
backoffQ	失败后的延迟重试	backoff deadline 到期
unschedulablePods	等待集群条件改变	Node/Pod/PVC/资源事件、超时 flush

- 调度重试不是忙轮询，而是 backoff + event-driven requeue。
- 插件可通过 QueueingHint 等机制缩小哪些事件真正值得重新尝试，从而减少无效调度。
- 大规模集群要关注 pending pods 数量、scheduling latency、plugin duration、unschedulable reason 分布。

56.2 Framework Extension Points 的回答方式

Extension Point	问题	典型插件/行为
PreEnqueue	Pod 是否应进入 active scheduling	SchedulingGates
QueueSort	哪个 Pod 先调度	PrioritySort
PreFilter	为 Filter 预计算	resources/affinity/topology state
Filter	节点是否满足硬约束	NodeResourcesFit, Affinity, Taints, Volumes
PostFilter	无可用节点后的恢复路径	DefaultPreemption
PreScore/Score	候选节点偏好	resources, topology, image, affinity
Reserve	乐观预留外部/内部状态	resource reservation
Permit	允许等待/拒绝	gang/co-scheduling style barrier
PreBind/Bind	绑定前动作/最终绑定	VolumeBinding/DefaultBinder
PostBind	绑定后通知	side effects/metrics

56.3 NodeResourcesFit 与资源策略

Filter: $\text{requested} + \text{nodeInfo.requested} \leq \text{allocatable}$

Score: $\text{LeastAllocated} \mid \text{MostAllocated} \mid \text{RequestedToCapacityRatio}$

Final node score = $\text{sum}(\text{plugin_score} * \text{plugin_weight})$

策略	目标	风险	AI 适用性
LeastAllocated	spread	节点难 scale-down、成本高	通用但不考虑 GPU topology
MostAllocated	bin pack	热点/碎片/峰值 headroom	适合 CPU 批任务和部分 inference
RequestedToCapacityRatio	自定义曲线	配置复杂	可表达保留 headroom
Topology-aware custom	通信 locality	实现/观测复杂	训练/多 GPU inference 价高

56.4 Preemption、Priority 与 Gang Scheduling

- Preemption 的核心问题不是“能不能杀低优先级 Pod”，而是选择 victims 后是否真的能让高优先级 Pod 满足所有 Filter。

- 分布式训练需要 gang 语义：如果 8 个 worker 只能启动 3 个，往往整体无进展且浪费 GPU。
- Kueue/Volcano 等上层批调度方案解决 quota、cohort、admission、gang 等需求；要避免与 kube-scheduler 的任重。

57. Kubelet / CRI / containerd / OCI 源码与排障专章

57.1 Kubelet 收模型

Pod source update -> syncLoop -> PodWorkers.UpdatePod -> per-Pod worker -> SyncPod
SyncPod -> admit/resources/cgroups/volumes -> runtime SyncPod -> sandbox + containers -> status reconciliation

子系统	职责	典型故障
PodWorkers	串行化同 Pod 更新	stuck sync / long operation
PLEG/runtime events	观察容器状态变化	runtime event lag
VolumeManager	attach/mount desired vs actual	ContainerCreating, mount timeout
EvictionManager	node pressure 处理	DiskPressure/MemoryPressure eviction
StatusManager	Pod status 上报	Ready/phase 状态滞后

57.2 CRI 与 OCI 边界

Kubelet -> CRI gRPC -> containerd CRI plugin -> containerd task -> shim -> runc -> OCI bundle -> namespaces/cgroups -> process

- CRI 是 Kubernetes 与 container runtime 的 API 界；OCI runtime spec 描述容器进程/namespace/cgroup/mount 等低层配置。
- Pod Sandbox 通常提供网络 namespace 和 Pod 级基础环境；容器加入该 sandbox。
- 排障要分层：Kubelet event -> CRI call -> containerd/shim -> runc -> kernel cgroup/ns。

57.3 requests/limits 到 cgroup v2

Kubernetes	cgroup v2	意义
requests.cpu	cpu.weight	竞争时相对 CPU 权重
limits.cpu	cpu.max	CPU bandwidth hard cap
limits.memory	memory.max	内存硬上限
memory QoS protection	memory.min / memory.low / memory.high (视配置)	保护/节流而非简单 hard limit
PID limit	pids.max	防 fork bomb
Block IO policy	io.weight / io.max	IO 权重/限速 (runtime/平台配置相关)

58. Kubernetes Networking 深挖 : CNI、Service、conntrack、eBPF

58.1 Pod 网络创建链路

Kubelet RunPodSandbox -> runtime creates netns -> CNI ADD -> IPAM -> veth/ENI -> route -> policy -> return CNI result -> Pod network ready

层	关键问题	常用证据
NetNS	Pod 是否获得正确 namespace/interface	nsenter, ip link/addr/route
IPAM	IP 是否耗尽/冲突	CNI logs, subnet/IP pool
Datapath	veth/ENI/eBPF/overlay 是否通	tc, bpftool, ip route
Policy	NetworkPolicy/SG 是否阻断	policy map, iptables/nft/eBPF
DNS	CoreDNS/ndots/search path	dig, resolv.conf, CoreDNS metrics

58.2 Service 数据面

实现	数据路径	优缺点
iptables	规则链 DNAT	成熟，但规则规模与更新成本可能上升
IPVS	virtual server hash/table	更明确的 LB 数据结构
eBPF	hook/map 驱动服务转发	可减少 iptables/conntrack 依赖，但实现更复杂
Cloud LB	外部 provider 数据面	与 Service controller/cloud integration 相关

58.3 高频网络面试故障树

- Pod A cannot reach Pod B
1. DNS? -> resolve destination
 2. route/interface? -> src netns / dst netns
 3. policy/SG/NACL?
 4. service NAT/conntrack?
 5. MTU/fragmentation?
 6. application listen/backlog?
 7. packet capture both ends

59. Kubernetes Storage 深挖 : CSI、PVC/PV、Topology 与数据一致性

59.1 CSI 生命周期

PVC -> provisioning -> PV -> scheduler VolumeBinding/topology -> ControllerPublish(attach) -> NodeStage -> NodePublish(mount) -> container

阶段	控制面/节点面	常见问题
Provision	CSI controller	quota 、StorageClass 参数 、backend latency
Bind	scheduler + PV controller	WaitForFirstConsumer/topology
Attach	controller service	multi-attach 、cloud API timeout
Stage/Publish	node plugin	filesystem 、mount 、device path
Resize/Snapshot	controller + node	fs resize 、snapshot consistency

AI Infra 关联

模型权重加载常成为 GPU cold-start 的关键路径。需要把对象存储/共享文件系统/local NVMe/cache prefetch 与 Pod placement 、image/model warmup 一起设计。

60. GPU Architecture / CUDA 深挖

60.1 面试需要掌握的硬件层次

组件	作用	Infra 关联
SM	行 warps/threads	compute utilization / occupancy
Tensor Core	矩阵/AI 加速	FP16/BF16/FP8 throughput
HBM	高带宽显存	weights + KV cache + activations
L2/cache	减少 HBM 流量	kernel efficiency
PCIe	CPU/GPU/NIC 互连	host transfer / GPUDirect topology
NVLink/NVSwitch	GPU-GPU 高速互连	tensor parallel / collective
NIC/RDMA	跨节点通信	NCCL multi-node

60.2 CUDA execution 模型

CPU host launches kernel -> grid -> blocks -> warps -> threads
Memory hierarchy: registers -> shared memory/L1 -> L2 -> HBM
Performance = compute efficiency + memory efficiency + launch/synchronization overhead

- Infra 面试不要求手写 CUDA kernel，但要理解为什么小 batch、频繁 kernel launch、低 arithmetic intensity 会让 GPU utilization 看起来“忙但不高效”。
- Pinned memory、DMA、asynchronous copies、streams 决定 host-device transfer 与 compute overlap。
- 多 GPU 时，CUDA device context、peer access、NCCL communicator 与 topology 都影响最终性能。

60.3 GPU OOM 不是只有一种

类型	特点	处理
Weight OOM	模型加载就失败	量化/TP/更多 GPU/CPU offload
KV OOM	并发/上下文增长后失败	admission、KV blocks、max sequences
Fragmentation	总 free 看似足够但大块分配失败	allocator/pooling/重启/调参
Transient peak	prefill/compilation/temporary buffer 峰值	headroom、batch/token budget
MIG capacity	profile 固定导致资源不匹配	重新规划 profile / admission

61. GPU Scheduling 最终专章：Device Plugin、DRA、MIG、Topology

61.1 Device Plugin vs DRA

维度	Device Plugin	DRA
资源表达	展源，如 nvidia.com/gpu: 1	DeviceClass/ResourceClaim/ResourceSlice 描述属性与选择
分配时机	scheduler 根据 extended resource capacity	claim 与 device allocation 进入调度流程
动态配置	有限	更适合动态参数/属性
复杂设备	表达能力有限	更适合 accelerator/NIC/组合设备
K8s 状态	长期成熟	Kubernetes 1.35 起 stable

2026 重点

DRA 已在 Kubernetes v1.35 stable。面试应理解 DeviceClass、ResourceClaim、ResourceSlice 以及 scheduler/kubelet/driver 如何共同完成 advertise -> allocate -> prepare -> cleanup。

61.2 GPU placement 目标函数

Good placement = feasible device + minimal fragmentation + topology locality + fairness + failure-domain policy + future schedulability

- 单卡 inference：优先考虑显存容量、GPU 型号、MIG profile、NUMA/NIC locality。
- 多卡单机：优先让 tensor-parallel ranks 落在 NVLink/NVSwitch 良好连通的 GPU 集合。
- 多机训练/推理：还要让 GPU 与 RDMA NIC 拓扑合理，并减少跨 rack/oversubscribed fabric。
- 长期平台要避免只优化当前 Pod：GPU 型号/MIG profile 碎片会影响后续大作业可调度性。

61.3 MIG / Time-Slicing / MPS / Full GPU

模式	隔离	利用率	适合
Full GPU	最高	视 workload	大模型/强 SLO
MIG	较强硬件隔离	高，但 profile 固定	多租户 inference
Time-Slicing	较弱	提升轻负载共享	开发/批量小任务
MPS	进程级并发共享	适合 CUDA 并发	受平台与 workload 限制

62. NCCL / NVLink / RDMA / RoCE 终极排障专章

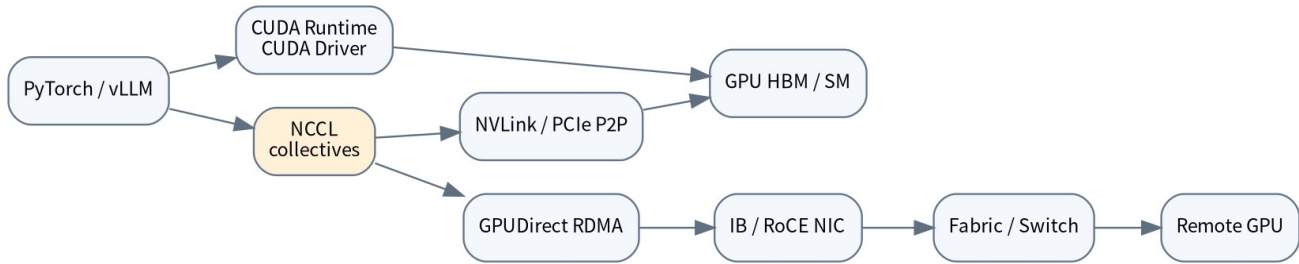


图 62-1 GPU-to-GPU / GPU-to-NIC 数据路径

62.1 Collective 基础

Collective	语义	常见用途
AllReduce	每 rank 聚合后都获得结果	data parallel gradient
AllGather	收集所有 rank 数据	tensor/model parallel
ReduceScatter	聚合并切分	ZeRO/parallel algorithms
AllToAll	每 rank 与所有 rank 交换不同数据	MoE expert parallel
Broadcast	root 向所有 rank 分发	初始化/参数同步

62.2 Hang 排障顺序

74. 确认所有 rank 进程存活、collective 调用次序一致，没有某 rank 提前异常。
75. 运行 NCCL 2.31 RAS diagnostics，比较 GPU inventory、driver、ECC、NVLink、NCCL_* 配置一致性。
76. 运行 active diagnostics / nccl-tests 验证真实 GPU 通信路径。
77. 单机先隔离 NVLink/PCIe；跨机再隔离 NIC/RDMA/IB/RoCE。
78. 检查 GPU Xid/ECC、NIC errors/drop、RDMA counters、switch PFC/ECN、MTU。
79. 最后再分析 collective algorithm/protocol/topology 是否导致性能问题。

层	关键工具/指标	典型问题
GPU	nvidia-smi/DCGM/Xid/ECC	GPU reset/ECC/PCIe error
NVLink	nvidia-smi topo/link counters	link down/错误拓扑
NCCL	NCCL_DEBUG/RAS/active diagnostics	rank mismatch/env mismatch
RDMA	ibstat/rdma/perftest/NIC counters	QP/path/GID/MTU
RoCE fabric	PFC/ECN/CNP/switch buffer	pause storm/congestion
Application	rank logs/timeout	collective order mismatch

62.3 RoCE vs InfiniBand

维度	RoCE	InfiniBand
底层	Ethernet	IB fabric
拥塞/无损	常依赖 ECN/PFC 等精细配置	IB 原生流控/拥塞机制
生态	复用数据中心 Ethernet	专用 HPC/AI 网络生态
运维	网络团队需精细调优	专用 fabric 工具链
常见问题	PFC storm、ECN 配置、MTU	link/subnet manager/path

63. vLLM Internals 最终专章：Scheduler、KV Cache、Prefill/Decode



图 63-1 vLLM Serving / Scheduler / KV Cache 心智模型

63.1 请求生命周期

HTTP request -> tokenize -> request queue -> scheduler admission -> KV block allocation -> prefill -> decode iterations -> detokenize/stream -> release KV blocks

阶段	瓶颈	关键指标
Queue	arrival > service capacity	queue depth/wait time
Prefill	长 prompt compute	TTFT, prefill tokens/s
KV allocation	显存容量/碎片	KV utilization, evictions/preemption
Decode	memory/launch/serial token generation	TPOT/ITL, decode tokens/s
Streaming	network/client backpressure	stream latency, disconnect

63.2 SchedulerConfig 面试关注点

配置	影响	Trade-off
max_num_batched_tokens	每 iteration token budget	大 -> throughput, 高峰 TTFT/latency 可能变差
max_num_seqs	并发 sequence 数	高 -> concurrency/显存压力
enable_chunked_prefill	长 prefill 拆分	降低 decode starvation, 但调度更复杂
async_scheduling	减少 GPU scheduling gap	并发状态管理更复杂
scheduler_policy	请求优先级/顺序	公平 vs latency vs throughput
scheduler_reserve_fullIsl	避免 chunked prefill 过度 admission	可能降低短期利用率, 换取稳定 KV

2026 文档要点

vLLM 当前 SchedulerConfig 公开了 async scheduling、chunked prefill、max_num_batched_tokens、max_num_seqs、scheduler_policy，以及 Hybrid KV Cache Manager 开关等关键控制项。

63.3 Hybrid KV Cache Manager

层	职责
KVCacheManager	Scheduler 与 KV subsystem 的统一接口
KVCacheCoordinator	协调一个或多个 KV cache group 的分配/prefix cache
SingleTypeKVCacheManager	针对 full attention/sliding window/Mamba 等类型实现块管理
KVCacheGroup	把具有兼容 KV layout 的层分组管理

- 面试不需要背源码每个函数，但要能解释为什么不同 attention 类型需要不同 KV 管理逻辑。
- Prefix caching 的价值来自复用共享 prompt 的 KV blocks；命中率取决于请求前缀分布与 cache eviction。
- KV cache 不仅是“剩余显存”；它直接决定可同时 active 的 context 数，是 admission control 的核心资源。

63.4 KV Cache 容量估算

Approx KV bytes per token $\sim 2 * \text{num_layers} * \text{num_kv_heads} * \text{head_dim} * \text{dtype_bytes}$

Per request KV $\sim \text{per_token_bytes} * (\text{prompt_tokens} + \text{generated_tokens})$

Concurrency ceiling $\sim \text{usable_KV_memory} / \text{expected_per_request_KV}$

- GQA/MQA 通过减少 KV heads 显著降低 KV cache。
- 生产估算必须使用 request length distribution，而不是单一 max context。
- 还要给 CUDA graphs、temporary buffers、allocator fragmentation 和峰值 prefill 留 headroom。

64. LLM Inference Capacity Engineering : 从 Token 到 GPU 数量

64.1 四个基本量

量	定义	工程意义
Arrival rate	requests/s 或 input/output tokens/s	需求强度
Service rate	每 replica 可持续 tokens/s	供给能力
Concurrency	同时 active requests/sequences	Little's Law / KV 占用
Latency SLO	TTFT/TPOT/E2E p95/p99	容量不能只看最大吞吐

`rough replicas = peak_required_token_rate / sustainable_token_rate_per_replica`
`production replicas = rough * headroom * failure_factor * rollout_factor`

64.2 为什么 benchmark 结果不能直接当生产容量

- benchmark 请求长度分布通常更简单；生产 prompt/output length 更长尾。
- 最大 throughput 点往往牺牲 TTFT/TPOT。
- 生产还有冷启动、模型切换、故障、rolling update、节点碎片、网络抖动。
- 因此需要建立 latency-throughput frontier，而不是只记录一个 tokens/s 数字。

64.3 Admission Control

信号	用途	风险
queue depth	检测 backlog	只看请求数忽略 token 大小
queued tokens	更接近工作量	需要准确估算 output
KV free blocks	显存 admission	可能忽略 compute bottleneck
TTFT/TPOT	直接 SLO feedback	滞后信号
active sequences	并发压力	不同 sequence 长度差异大

65. AI Inference Autoscaling : 多控制回路设计

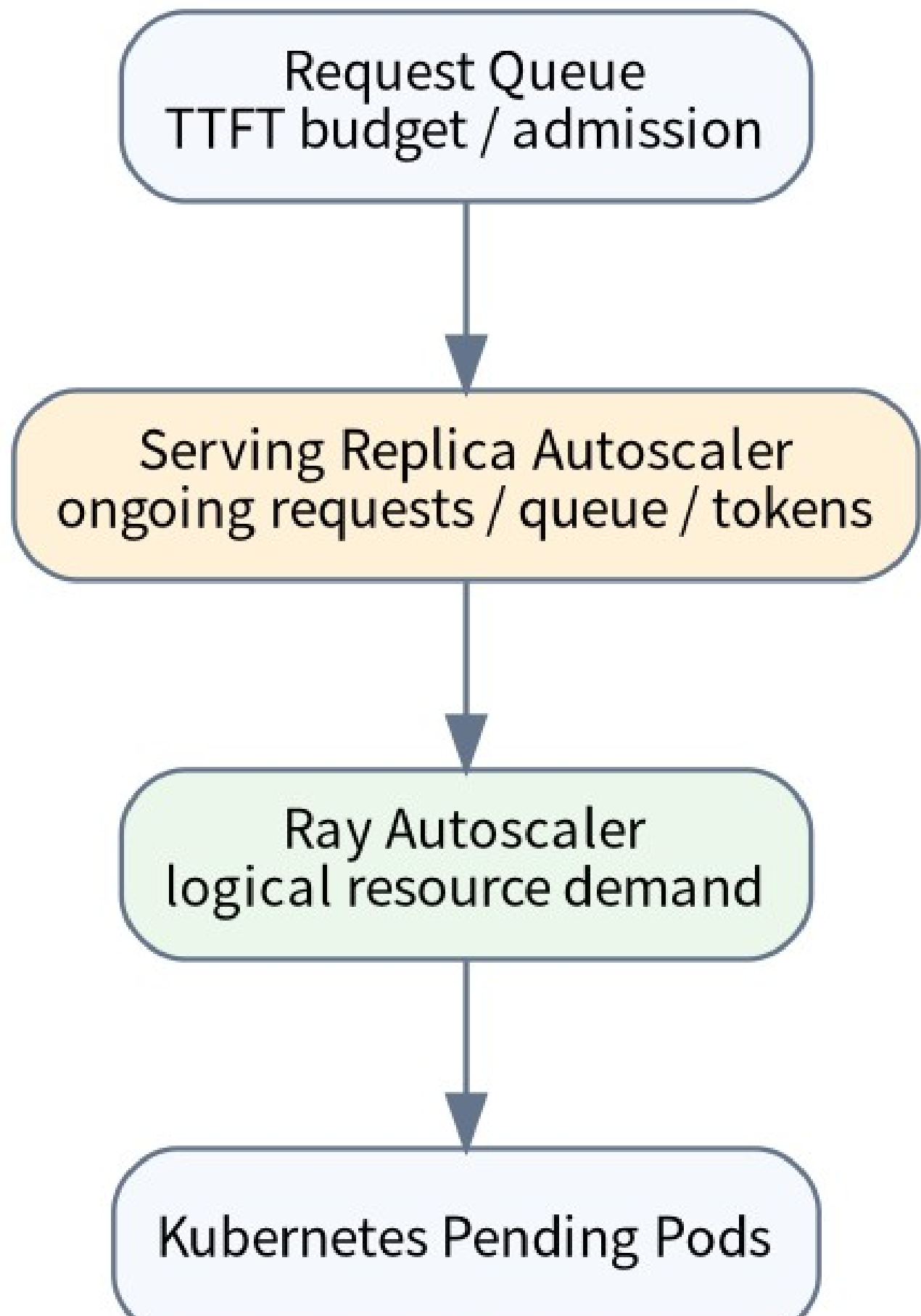


图 65-1 Request/Replica/Ray/Kubernetes/Node 多层 Autoscaling

层	控制对象	推荐信号	时间尺度
Request Router	流量分配/admission	queue/tokens/SLO	秒级
Serving Replica	vLLM/Ray Serve replica	queue, active seq, TTFT/TPOT	秒~分钟
Ray Worker	worker Pods	logical resource demand	分钟
Kubernetes	Pod placement	requests/claims	事件驱动
Node Autoscaler	GPU nodes	pending pods/capacity	分钟级
Cloud capacity	GPU quota/instance supply	provider availability	分钟~小时

避免振荡

不要让所有层同时用同一个短周期 utilization 指标。上层应快速做 admission/replica 调节，下层 node autoscaler 更慢，并使用 cooldown、hysteresis、warm pool 与 capacity headroom。

66. Ray / KubeRay / Ray Serve 深挖

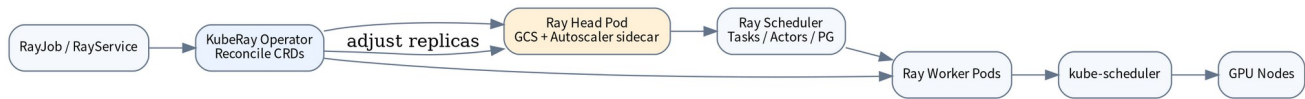


图 66-1 KubeRay Operator / RayCluster / RayService / Autoscaling

66.1 KubeRay Operator Reconcile 心智模型

Watch RayCluster/RayJob/RayService CR -> enqueue key -> reconcile -> compare desired vs actual -> create/update/delete head/worker Services/Pods -> status update

CRD	主要用途	面试重点
RayCluster	长生命周期 Ray 集群	head/worker groups 、autoscaler
RayJob	提交 job 并管理集群生命周期	job completion/failure cleanup
RayService	Ray Serve + cluster HA/upgrade	zero-downtime rollout 、serve status

66.2 Ray Autoscaler 与 K8s Node Autoscaler

- Ray Autoscaler 根据 Ray logical resources、tasks/actors/placement groups 的需求扩 worker pods。
- K8s node autoscaler 看到 pending worker pods 后再扩 GPU nodes。
- 这会产生天然的两级延迟：Ray demand -> Pod pending -> cloud node ready -> GPU/driver ready -> Ray worker ready。
- 优化 cold start 要覆盖 image/model cache、GPU node bootstrap、driver/operator、Ray worker registration。

67. AI Agent Runtime / Sandbox 最终专章

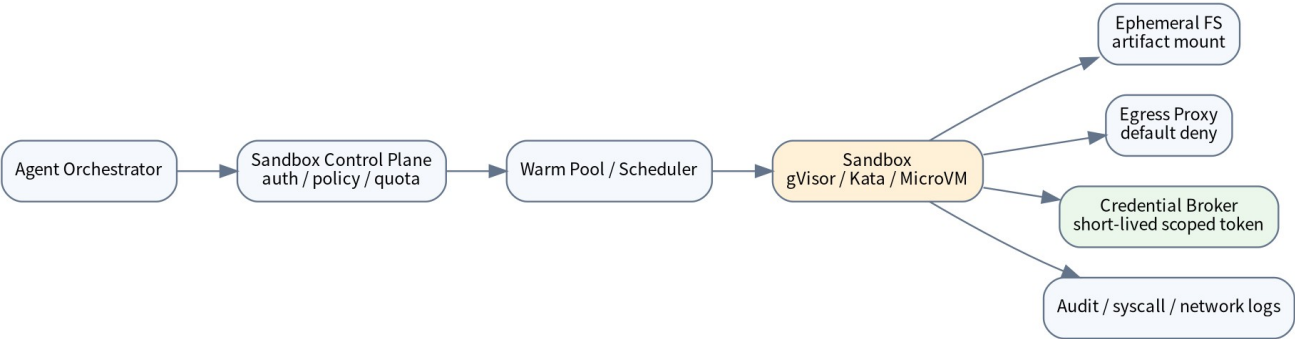


图 67-1 Agent Runtime Platform : Sandbox / Broker / Network / Audit

67.1 Threat Model

威胁	例子	防护
Host escape	恶意代码利用 kernel/runtime 漏洞	MicroVM/Kata/gVisor、patching、seccomp
Credential theft	读取 cloud token/API key	credential broker、短期 token、scope
Data exfiltration	任意 egress 上传数据	default-deny egress + allowlist/proxy
Resource abuse	fork bomb/crypto mining	cgroup CPU/mem/pids/io、timeout
Persistence	写入共享卷/缓存污染	ephemeral FS、reset、immutable base
Cross-tenant	side channel/共享缓存	强隔离、tenant-aware placement

67.2 Sandbox 技术选择

技术	隔离	启动	兼容性	适合
Container/rootless	中	最快	最高	低风险工具执行
gVisor	中高	快	较高	不可信 Linux workload
Kata	高	中	高	K8s 内强隔离
Firecracker/MicroVM	高	较快	VM 级	高风险 agent sandbox
WASM	高（能力沙箱）	快	受限	插件/脚本/受限 runtime

67.3 Warm Pool 与安全 Reset

- 80. 预启动 sandbox 减少 cold start，但必须保证 tenant 切换前彻底 reset。
- 81. 清理 filesystem、process、network namespace、credential、tmp/cache、runtime state。
- 82. 高风险场景更倾向销毁 MicroVM 而不是复用长期实例。
- 83. 所有工具调用和外部网络访问应产生可审计事件。

68. AI Infrastructure Observability : 从 HTTP Request 到 GPU Fabric

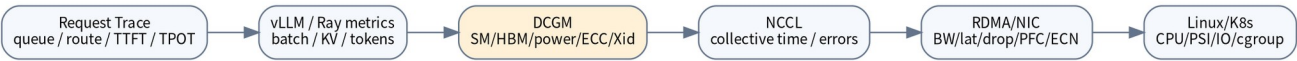


图 68-1 AI Infra 端到端 Observability 层级

层	核心指标	典型告警
API/Router	QPS 、 queue 、 429 、 E2E latency	queue growth / rejection
Inference	TTFT 、 TPOT 、 tokens/s 、 active seq	latency SLO breach
KV/Memory	KV utilization 、 eviction/preemption 、 HBM	KV pressure/OOM
GPU	SM/HBM/power/temp/ECC/Xid	GPU unhealthy
NCCL	collective duration 、 timeouts	hang/perf regression
RDMA/NIC	throughput/drop/PFC/ECN/CNP	fabric congestion
Kubernetes	pending/scheduling/kubelet/node	placement/capacity
Linux	PSI/CPU/IO/TCP	node bottleneck

Golden Debugging Rule

任何 SLO 问题都先定位“排队、计算、内存、通信、节点、控制面”中的哪一层，再向下 drill-down；不要一开始就进入最底层抓包或 perf。

69. Staff-Level System Design 通用答题模板

步骤	必须回答	示例
1. Clarify	QPS/token、模型大小、SLO、region、tenant	p99 TTFT < 500ms?
2. Capacity	GPU 型号、per-replica throughput、KV	需要多少 GPU/region
3. Data Plane	router -> serving -> GPU/fabric	请求与 token 数据路径
4. Control Plane	deployment/scheduler/autoscaler/operator	模型/资源生命周期
5. Failure Domains	GPU/node/AZ/region/provider	冗余与降级
6. Observability	SLO + saturation + health	TTFT/TPOT/KV/NCCL
7. Security	tenant/network/credential/model data	isolation
8. Cost	utilization/headroom/spot/cache	cost/token
9. Trade-offs	latency vs throughput vs cost	为什么选这个方案

69.1 Case A : 100k QPS Multi-Region Inference Platform

84. 把请求转换为 token workload，并基于真实 length distribution 估算 region GPU 容量。
85. Global traffic manager 根据 region health/capacity/latency 路由。
86. Region 内 router 做 model/version/tenant aware routing 和 admission。
87. Serving fleet 预留 N+1/N+2 headroom，模型权重通过本地 cache/warm pool 减少冷启动。
88. Node autoscaling 与 cloud GPU quota 形成慢控制回路；突发先靠 warm capacity 和 admission。
89. 跨 region failover 要考虑 capacity shock，不能假设另一 region 有无限 GPU。

69.2 Case B : 多租户 GPU Platform

- 资源模型：full GPU/MIG/DRA、tenant quota、priority、preemption、fair sharing。
- 隔离：namespace/RBAC + node/GPU isolation + network + secrets。
- 调度：GPU SKU、MIG profile、topology、gang、failure domain。
- 成本：idle GPU、fragmentation、reserved/spot mix、chargeback/showback。

69.3 Case C : Distributed Training Platform

- Gang admission：资源全部满足后再启动。
- Checkpoint：对象存储/并行文件系统、恢复时间、checkpoint cadence。
- 网络：NCCL topology、RDMA、rack placement、fabric oversubscription。
- 故障：rank failure、node/GPU quarantine、job restart/elastic strategy。

69.4 Case D : Secure Agent Execution Platform

- MicroVM/gVisor/Kata sandbox pool，按风险等级路由。
- Credential broker 使用短期、最小权限 token；sandbox 无长期云凭证。
- Egress proxy/default deny，支持 domain/API allowlist 与审计。
- Sandbox lifecycle: create -> inject task -> execute -> collect output -> revoke credentials -> destroy/reset。

70. 五套跨层 Troubleshooting Runbook

70.1 Pod Pending 但集群“看起来有 GPU”

90. `kubectl describe pod`：确认 `Unschedulable reason`。
91. 检查 `request/DRA claim/MIG profile/GPU SKU` 是否匹配。
92. 检查 `taint/toleration`、`affinity`、`topology/gang/quota`。
93. 检查 `scheduler cache` 与 `Node allocatable/device advertisement`。
94. 检查碎片：有总 GPU 但没有满足 `topology/profile` 的连续设备集合。
95. 确认 `autoscaler` 是否能创建正确 GPU node type，并检查云侧 `quota/capacity`。

70.2 vLLM TTFT p99 突然升高

96. 先看 `queue wait` 是否上升；若是，说明 `arrival` 超过 `service capacity/admission` 太宽
97. 区分 `prefill compute` 变慢还是 `scheduler` 等待变长。
98. 检查 `prompt length distribution` 是否变化。
99. 检查 `max_num_batched_tokens/max_num_seqs/chunked prefill` 配置。
100. 检查 `HBM/KV pressure`、`preemption/eviction`、`GPU clocks/power/thermal`。
101. 多 GPU 时检查 `NCCL collective latency` 与 `NIC/fabric`。

70.3 NCCL Job 随机 Hang

102. 收集所有 `ranks` 日志，确认是否某 `rank crash/timeout`。
103. 执行 `RAS diagnostics` 比较软件/硬件一致性。
104. 执行 `active diagnostics/nccl-tests` 验证 `data path`。
105. 查看 `Xid/ECC/NVLink`，跨机查看 `RDMA/NIC/switch`。
106. 检查 `collective sequence mismatch`、`timeout` 和 `workload skew`。

70.4 Node CPU 低但 Pod latency 高

107. 查看 `CPU throttling`：`cgroup cpu.stat`。
108. 查看 `PSI / load / D-state`，排除 `IO` 或 `memory pressure`。
109. 查看 `network retrans/conntrack/softirq`。
110. 查看 `application queue/GC/lock`。
111. 按 `request trace` 把 `latency` 分解到 `client/network/app/storage`。

70.5 Agent Sandbox 发生可疑外联

112. 立 `revoke sandbox credential / terminate sandbox`。
113. 保存审计日志、`network flow`、`process tree`、`filesystem diff`。
114. 确认 `egress policy` 为什么允许该目标。
115. 检查 `credential broker scope/TTL` 与 `token usage`。
116. 按 `tenant/host image/runtime version` 搜索相同 `IOC`，必要时 `quarantine node`。

71. Hands-on Lab 最终清单：从 K8s 到 AI Infra

Lab	目标	输出证据
L1 Scheduler trace	观察 Filter/Score/Bind	scheduler logs + events
L2 Informer	写 client-go informer/controller	queue retry metrics
L3 cgroup v2	验证 request/limit 映射	cpu.max/weight/memory.max
L4 CNI	跟踪 Pod netns/veth/route	ip/nsenter/tcpdump
L5 CSI	观察 PVC -> mount	events + node mount
L6 GPU Operator	部署并采集 DCGM	GPU dashboard
L7 MIG	创建 profile 并调度 workload	MIG inventory + pods
L8 DRA	创建 DeviceClass/Claim	allocation objects
L9 vLLM	压测不同 batching 参数	TTFT/TPOT/tokens/s curve
L10 KV Cache	估算与实测并发上限	memory vs concurrency
L11 KubeRay	RayService + autoscaling	worker scale timeline
L12 NCCL	单机/多机 nccl-tests	bandwidth/topology
L13 RDMA	perftest + counters	latency/bandwidth
L14 Agent Sandbox	default-deny + broker	egress/credential audit
L15 Full System Design	搭建 mini inference platform	design doc + benchmark

72. 面试现场命令与指标 Cheat Sheet

领域	常用命令	回答重点
CPU	top, mpstat, pidstat, perf	user/system/runq/throttle/context switch
Memory	free, vmstat, smaps, memory.stat, PSI	RSS/cache/reclaim/OOM
IO	iostat -x, pidstat -d, fio	await/queue depth/fsync
Network	ss, ip, ethtool, tcpdump, conntrack	state/retrans/route/drop
K8s	kubecttl describe/get events/logs	先事件再组件日志
Scheduler	scheduler metrics/logs	unschedulable reason/plugin latency
GPU	nvidia-smi, DCGM	HBM/SM/ECC/Xid/topology
NCCL	NCCL_DEBUG, RAS diagnostics, nccl-tests	rank/path/collective
RDMA	ibstat, rdma, ib_write_bw	link/GID/MTU/errors
vLLM	serving metrics/benchmark	TTFT/TPOT/tokens/KV/queue

73. 最终 60 道高价值追问题（用于 Mock Interview）

117. 为什么 scheduler 使用 requests 而不是实时 utilization？如果想做 utilization-aware scheduling 有什么风险？
118. 为什么 Assume 能提高 scheduler 并发？它与 Reserve 有什么区别？
119. 一个 Pod 为什么会在 unschedulablePods 与 backoffQ 之间移动？
120. 如何证明 NodeResourcesFit 不是最终唯一决定节点的插件？
121. Preemption 如何选择 victim？为什么 PDB 只是一部分约束？
122. DRA 如何改变 GPU 资源表达方式？
123. 如何设计一个 topology-aware GPU Score plugin？
124. 为什么 MIG profile 会造成资源碎片？怎么衡量？
125. 为什么 GPU node autoscaling 比普通 CPU node 更难？
126. 如何处理云厂商 GPU capacity 不可用？
127. API Server APF 如何防止 noisy client 拖垮控制面？
128. 为什么 webhook 可能形成控制面循环依赖？
129. 为什么 etcd fsync 延迟会放大成集群级症状？
130. Informer cache 过旧时 controller 为什么仍能最终收敛？
131. WorkQueue 去重与 rate limit 各解决什么问题？
132. Kubelet 为什么需要 PodWorkers？
133. PLEG/runtime event lag 会表现成什么？
134. ContainerCreating 卡住时如何区分 CNI 与 CSI？
135. 为什么 cgroup CPU request 是权重而 memory request 不是完全同类机制？
136. memory.high 与 memory.max 的行为差异是什么？
137. Linux PSI 比单纯 utilization 多提供了什么信息？
138. 为什么 load average 可以因为 IO 增加？
139. 为什么 CLOSE_WAIT 是应用问题而 TIME_WAIT 常常不是？
140. conntrack table 满会如何影响 K8s Service？
141. eBPF datapath 为什么可能降低 iptables 大规模规则成本？
142. GPU HBM utilization 高但 tokens/s 低可能是什么原因？
143. 什么是 arithmetic intensity？为什么对 GPU 性能重要？
144. PCIe locality 如何影响 GPU + NIC RDMA？
145. NVLink/NVSwitch 对 tensor parallel 有什么价值？
146. AllReduce 与 AllToAll 的网络流量特征为什么不同？
147. NCCL hang 与 NCCL slow 应分别怎么排？
148. RAS diagnostics 与 active diagnostics 的区别是什么？
149. RoCE 为什么需要关注 PFC/ECN？
150. PFC pause storm 如何影响整个 rack？
151. vLLM continuous batching 为什么提高吞吐？
152. chunked prefill 如何减少 decode starvation？
153. Prefix cache 什么时候几乎没有收益？
154. KV Cache 为什么常成为 concurrency ceiling？
155. 如何估算一个 70B 模型在某 GPU 上的显存预算？
156. 量化会怎样影响 weight memory、throughput 与质量？

157. 为什么 TTFT 和 TPOT 需要分别扩缩容？
158. 为什么只按 GPU utilization autoscale 容易失控？
159. 如何设计 queue-based autoscaling 而不发生 oscillation？
160. Ray Autoscaler 与 K8s node autoscaler 如何协作？
161. Ray placement group 与 gang scheduling 有什么联系？
162. KubeRay Operator reconcile 失败如何恢复？
163. RayService rollout 如何避免同时丢失旧模型和新模型？
164. 模型 cold start 的时间花在哪里？
165. 如何设计 model weight cache？
166. 多 region inference failover 为什么会产生 capacity shock？
167. 如何在 SLO 和 GPU 成本之间设置 headroom？
168. 如何定义 cost per token？
169. 如何识别 GPU fleet 中的 bad node？
170. 如何设计 GPU quarantine 与 automatic remediation？
171. Agent sandbox 为什么必须使用短期凭证？
172. 为什么 default-deny egress 对 agent 特别重要？
173. MicroVM warm pool 的安全 reset 要包含什么？
174. 如何给 agent tool execution 做审计？
175. 如何在系统设计面试中说明 failure domain？
176. Staff engineer 在 incident 中如何把跨层证据组织成假设树？

74. 最终速查矩阵

场景	第一指标	第二指标	第三层深入
Pod Pending	scheduler reason	requests/claim/topology	autoscaler/cloud capacity
Pod Creating	events	kubelet/runtime	CNI/CSI
CPU latency	throttle/PSI	runq/context switch	perf/eBPF
Memory latency	PSI/reclaim	memory.stat	NUMA/page fault
Network latency	retrans/RTT	softirq/conntrack	packet path
GPU slow	SM/HBM/power	kernel/profile	topology/NCCL
TTFT high	queue wait	prefill tokens	KV/GPU capacity
TPOT high	decode throughput	batch/active seq	GPU memory/communication
NCCL hang	rank health	RAS/active diagnostics	RDMA/fabric
Agent incident	sandbox terminate	credential revoke	network/audit forensics

75. 最终 30 天面试冲刺计划

天数	主题	每天产出
1-4	Linux CPU/Memory/IO/Network	每天 10 题口述 + 1 个故障实验
5-9	K8s API/etcd/Informer/Scheduler/Kubelet	画 5 张源码调用图 + 源码阅读
10-12	CNI/CSI/cgroup/containerd	3 个端到端实验
13-16	GPU/CUDA/MIG/DRA	GPU 平台设计 + 设备调度实验
17-20	NCCL/RDMA/NVLink	nccl-tests + 网络故障树
21-24	vLLM/KV/Prefill/Decode	benchmark + capacity spreadsheet
25-26	Ray/KubeRay/Autoscaling	RayService 与双层扩缩容
27	Agent Sandbox	threat model + sandbox design
28	System Design	2 套 45 分钟白板
29	Full Mock	Linux+K8s+AI Infra 90 分钟
30	复盘	只复习错题和薄弱调用链

建议

每道题尽量形成 2 分钟版本和 10 分钟版本。2 分钟版用于先给结构化答案；面试官继续追问时，再下钻源码、指标、命令、trade-off。

76. 最终版 Research / Reference Checklist

- Kubernetes 官方文档：Scheduling Framework、Dynamic Resource Allocation、Kubelet/CRI、API Priority and Fairness、cgroup v2。
- Kubernetes 源码：kubernetes/kubernetes 的 scheduler、kubelet、apiserver、client-go/cache/workqueue。
- NVIDIA：CUDA、GPU Operator、MIG、DCGM、NCCL 2.31 diagnostics、RDMA/GPUDirect 资料。
- vLLM 官方：SchedulerConfig、Hybrid KV Cache Manager、distributed inference、metrics/benchmark。
- Ray/KubeRay 官方：RayCluster/RayJob/RayService、KubeRay autoscaling、Ray Serve。
- 生产准备时必须按你要面试公司的实际 GPU 型号、Kubernetes 版本、云平台和 serving stack 再做一次定向校准。

版本注意

本手册强调稳定概念与工程方法；具体函数名、feature state、默认参数会随版本变化。面试前应以目标岗位正在使用的 Kubernetes / CUDA / NCCL / vLLM / Ray 版本为准。

77. : 把 Kubernetes 深度变成 AI Infrastructure 的优势

最终目标不是“从 Kubernetes 行去做 AI”，而是把 Linux、Kubernetes、网络、分布式系统的基础设施深度，延伸到 GPU 度、分布式通信、LLM serving 与 Agent Runtime。

- 遇到任何问题先画数据路径和控制路径。
- 先定义 SLO 与资源，再谈架构。
- 用指标和实验验证假设，而不是只靠经验判断。
- 系统设计必须同时讨论 reliability、performance、security 与 cost。
- 源码学习重点是组件边界、状态机、队列/cache、失败恢复与关键扩展点。